B.TECH CSE · DATABASE MANAGEMENT SYSTEMS

Relational Algebra

Complete Solved Question Bank

Classical relational algebra only — selection, projection, rename, union, intersection, difference, Cartesian product, join and division. No SQL. No aggregates. Every expression derived, every schema stated.

Coverage. 12 parts · 3 mandatory schemas (Sailors–Boats, Library, Hospital–Prescription) · 240+ solved questions · pattern catalogue · "not expressible" proofs · error clinic · practice paper with answer key · coverage checklist. Modelled on **Database System Concepts** (Silberschatz, Korth, Sudarshan), **Fundamentals of Database Systems** (Elmasri & Navathe) and **Database Management Systems** (Ramakrishnan & Gehrke).

Compiled for university examination preparation · MANIT Bhopal pattern

How to use this book

This book is a **question bank**, not a textbook. It is organised the way an examiner thinks: one **logical pattern** per family of questions, then every surface form of that pattern, then the traps that separate a 6-mark answer from a 10-mark answer.

The one idea that carries the whole subject. Nearly every "hard" relational algebra question is one of exactly five shapes:

1. Existence ("at least one", "some", "a")
→ join, then project.

2. Non-existence ("none", "never", "not")
→ `π(R) − π(matches)`.

3. Universality ("all", "every", "each")
→ division, or `π(R) − π(counter-example)`.

4. Extremum ("highest", "oldest", "latest")
→ `R − π(σ_{R1.a < R2.a}(ρ_{R1}R × ρ_{R2}R))`.

5. Comparison of two records ("same", "more than", "pairs")
→ self-join with `ρ` plus an explicit `θ` predicate.

6. Counting / summing ("exactly 3", "more than 20", "total")
→ **not expressible** in classical RA. Say so, then give the extended-algebra form.

Conventions used throughout.

- Relations are written in italics: $T(\text{Sailors})$, $T(\text{Reserves})$. Attribute names are upright: $A(\text{sid})$, $A(\text{sname})$.
- String constants are green: $\sigma_{A(\text{color})=\,\textbf{red}}$.
- **Set semantics** is assumed everywhere. A relation is a **set** of tuples; π and every other operator remove duplicates. This is the classical definition and it is what university answer scripts are marked against.
- Schema of a relation $T(\text{R})$ is written $T(\text{R})(A_1, A_2, \ldots, A_n)$.
- $\perp$ denotes the distinguished **no-value** marker used when a textbook schema has a NULL-able column (for example `ReturnDate`). Classical relational algebra has no NULLs; we model "unknown" by an explicit constant or by an explicit extra relation, and we say which one we are using.
- Every solution states (i) the expression, (ii) the intermediate relations **with their schemas**, (iii) why it is correct, (iv) the trap.

On attributing questions. Questions marked **(SKS §4.3 pattern)**, **(EN Ch. 6 pattern)** or **(RG §4.2 pattern)** follow the standard pattern of that chapter; they are **textbook-standard**, not verbatim quotations. Only questions whose wording is genuinely canonical (for example "find sailors who have reserved all boats") are described as classic. No question in this book claims to be a verbatim reproduction of a printed exercise.

The twelve parts, and what each one fixes in your head

Part	Purpose
1	Short notes, operator reference, schema of every operator, the 20-pattern catalogue
2	Basic university-level questions: selection, projection, rename, one-relation and two-relation queries
3	Joins, self-joins, set operations, negative ("never / without / except") queries
4	Division and universal quantification; deriving division from the five primitive operators
5	Maximum, minimum, ranking, ties, group-wise extremes, k-th highest
6	Sailors & Boats: the full 37-question classical bank

7	Books, Authors and Borrowing: full and simplified schema, 33 questions
8	Hospital & Prescription: the five original exam questions plus 22 more
9	Advanced mixed questions that combine three or more techniques
10	Query equivalence, wrong expressions, counterexamples, error clinic
11	Practice paper — questions only, exam timing
12	Answer key, marking scheme, coverage checklist

A warning about the phrase "write the relational algebra". Marks are lost far more often for an ill-typed expression than for a wrong idea. Before you write anything, write down the **schema you want** (which attributes must the answer have?) and the **schema you have**. Every operator in this book is accompanied by its schema rule for exactly this reason — see Part 1.3.

Contents

Part 1 — Short notes and operator reference 3
Part 2 — Basic university-level questions 10
Part 3 — Joins, self-joins, set operations and negative queries 16
Part 4 — Division, "all", "every", "only" and universal conditions 22
Part 5 — Maximum, minimum, ranking, ties and group-wise comparisons 29
Part 6 — Sailors and Boats — complete solved question bank 39
Part 7 — Books, Authors and Borrowing — complete solved question bank 47
Part 8 — Hospital and Prescription — complete solved question bank 55
Part 9 — Advanced mixed questions combining multiple techniques 65
Part 10 — Query equivalence, incorrect expressions, counterexamples and exam traps 69
Part 11 — Practice paper — questions only 78
Part 12 — Answer key, marking scheme and coverage checklist 80

Part 1 — Short notes and operator reference

What this part fixes. The exact notation, arity and **schema rule** of all nine classical operators; what "union compatible" means; how division is defined; what set semantics costs you; which operators are **not** allowed in this book and why; and the 20 logical patterns from which every exam question in Parts 2–9 is built.

1.1 What relational algebra is, and why the examiner asks for it

Relational algebra is a **procedural** query language: an expression is a tree of operators, each of which takes one or two relations and returns one relation. Because every operator returns a relation, expressions nest arbitrarily. That closure property is what makes the algebra composable, and it is also why a single operator applied to the wrong operand type — a relation where an attribute is expected, say — makes the whole answer worth zero.

Two things are tested when a university paper asks for "the relational algebra" of a query:

1. **Can you decompose an English sentence into logical quantifiers?** ("all", "only", "no", "at least two", "maximum", "same as").
2. **Can you build a well-typed expression tree** whose root schema is exactly the schema the question asks for, using only the permitted operators?

The second is where most marks leak. A student answers "names of sailors who reserved a red boat" with $\pi_{\text{bname}}(\sigma_{\text{color} = \textbf{red}}(Boats))$ — correct in spirit, wrong in schema, and it never touches `Reserves`. The fix is mechanical: **write the target schema first, then the source schema, then bridge them.**

1.2 The three schemas used throughout

Every question in this book belongs to one of three families. They are defined here once; Parts 6, 7 and 8 restate them together with sample instances.

A. Sailors, Boats, Reserves — the Database System Concepts running example

Relation	Schema, keys and foreign keys
Sailors	(“sid”, “sname”, “rating”, “age”); primary key “sid”. “rating” is an integer 1–10; “age” is in years
Boats	(“bid”, “bname”, “color”); primary key “bid”
Reserves	(“sid”, “bid”, “day”); primary key (“sid”, “bid”, “day”); “sid” → Sailors, “bid” → Boats

The composite key on `Reserves` matters. It says a sailor reserves a given boat **at most once per day**, but may reserve it again on another day. Consequently $\pi_{\text{sid, bid}}(Reserves)$ is a **set** with duplicates removed: a sailor who reserved boat 103 on three days appears once. “At least two reservations” is therefore **not** the same as “at least two boats”; see Q6.21 and Q6.37.

B. Library — Books, Authors and Borrowing

Two versions are used, and every question states which one applies.

Full schema (the default).

Relation	Schema, keys and foreign keys
Book	(“BookID”, “Title”, “Publisher”, “Category”); PK “BookID”
Author	(“AuthorID”, “AuthorName”); PK “AuthorID”
WrittenBy	(“BookID”, “AuthorID”); PK (“BookID”, “AuthorID”); FKs to Book and Author
Member	(“MemberID”, “MemberName”); PK “MemberID”
Borrow	(“MemberID”, “BookID”, “BorrowDate”, “ReturnDate”); PK (“MemberID”, “BookID”, “BorrowDate”); FKs to Member and Book. “ReturnDate” $= \perp$ means **not yet returned**

Simplified schema. $Books\ (\text{BookID}, \text{Title})$, $BookAuthors\ (\text{BookID}, \text{Author})$, $Borrow\ (\text{MemberID}, \text{BookID})$ — no dates, no publisher, one author name per row, and the key of `Borrow` is (“MemberID”, “BookID”). In the simplified schema a member borrows a given book at most once, so “borrowed twice” is not merely hard — it is meaningless.

C. Hospital and Prescription — the university exam pattern

Relation	Schema, intended keys and foreign keys
Doctor	(“SSN”, “FirstName”, “LastName”, “Specialty”, “YearsOfExperience”, “PhoneNum”); PK SSN
Patient	(“SSN”, “FirstName”, “LastName”, “Address”, “DOB”, “PrimaryDoctor_SSN”); PK SSN “PrimaryDoctor_SSN” → Doctor.SSN
Medicine	(“TradeName”, “UnitPrice”, “GenericFlag”); PK TradeName
Prescription	(“Id”, “Date”, “Doctor_SSN”, “Patient_SSN”); PK Id FKs “Doctor_SSN” → Doctor, “Patient_SSN” → Patient
Prescription_Medicine	(“Prescription_Id”, “TradeName”, “NumOfUnits”); PK (“Prescription_Id”, “TradeName”); FKs to Prescription and Medicine

Two attributes are named “SSN”: `Doctor.SSN` and `Patient.SSN`. In an unqualified expression this is an ambiguity error, not a stylistic preference. Every solution in Part 8 renames the relations — $\rho_D(Doctor)$, $\rho_P(Patient)$ — or qualifies explicitly the moment both appear. The same applies to “FirstName” and “LastName”, which both relations own.

1.3 Operator reference: arity, schema, semantics

Throughout, $R\ (A_1, \ldots, A_n)$ and $S\ (B_1, \ldots, B_m)$ are relations, and $\text{attr}\ (R)$ denotes the attribute set of R.

Operator	Arity	Schema rule	Semantics
$\sigma_\theta(R)$	1	$\text{attr} = \text{attr}(R)$	Keeps the tuples satisfying θ. Never removes attributes.
$\pi_{i_1,\ldots,i_k}(R)$	1	$\{i_1, \ldots, i_k\} \subseteq \text{attr}(R)$; result has k attributes in the written order	Keeps the named columns and **removes duplicate rows** (set semantics).
$\rho_X(R)$	1	same attributes, new relation name X	Renames the relation. $\rho_{X\ (a_1,\ldots,a_n)}(R)$ renames relation **and** attributes positionally.
$R \cup S$	2	requires $n = m$ and $\text{dom}(A_i) = \text{dom}(B_i)$ for all i; names come from the left operand	Tuples in R or S or both. Duplicates collapse.
$R - S$	2	same union-compatibility rule	Tuples in R that are **not** in S. Not commutative.
$R \cap S$	2	same rule	Tuples in both. Derived: $R - (R - S)$.
$R \times S$	2	$\text{attr}(R) \cup \text{attr}(S)$; $n \cdot m$ attributes, $\lvert R\rvert \cdot \lvert S\rvert$ tuples	Every tuple of R paired with every tuple of S. Illegal if the two share an attribute name and neither is renamed.
$R \underset{\theta}{\bowtie} S$	2	as $\times$, then σ_θ	$\sigma_\theta(R \times S)$ — a **theta join**. Both copies of a compared attribute survive.
$R \bowtie S$	2	$\text{attr}(R) \cup \text{attr}(S)$, shared names merged once	Natural join: equates every pair of same-named attributes, then removes the duplicate copy.
$R \div S$	2	requires $\text{attr}(S) \subseteq \text{attr}(R)$; result attributes $= \text{attr}(R) - \text{attr}(S)$	Tuples t such that for **every** tuple s in S, t concatenated with s is in R.

The schema rule is the examinable part. For every intermediate relation you create, write its schema beside it, exactly as every worked solution in this book does.

1.4 Selection: the details that are actually examined

Selection takes **rows** that satisfy a condition. The condition may reference any attribute of the operand and may be built from $\wedge$, $\vee$, $\neg$ and the comparisons $=, \neq, <, \leq, >, \geq$.

$$\sigma_{\text{rating}\ >7 \wedge\ \text{age}\ <40}(Sailors) \quad \sigma_{\text{color}\ =\ \textbf{red}\ \vee\ \text{color}\ =\ \textbf{green}}(Boats)$$

Two identities are worth memorising because they justify the simplifications in Part 10:

$$\sigma_{\theta_1}\big(\sigma_{\theta_2}(R)\big) = \sigma_{\theta_1 \wedge \theta_2}(R) \quad \sigma_{\theta_1}(R \cup S) = \sigma_{\theta_1}(R) \cup \sigma_{\theta_1}(S)$$

The second holds for union, intersection and difference — selection distributes over all three set operations — but **not** over projection or division.

Trap / common mistake. $\sigma_{\text{rating} = \text{high}}(Sailors)$ is a type error: "rating" is numeric. In this book string constants are always green and quoted, numeric constants never are. Also, σ cannot compare attributes of **different tuples** unless those tuples were first brought together by a product or a self-join — see Part 3.

1.5 Projection: duplicate elimination is the whole point

$\pi_{\text{color}}(Boats)$ does **not** return one row per boat. It returns the **set of colours that occur**: if sixty boats are red, the output contains exactly one **red**. This is not a technicality, it is the reason $\pi_{\text{sid}}(Reserves)$ can stand in for "sailors who have made a reservation" with no DISTINCT keyword.

Consequences used repeatedly below:

- $\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(Reserves)$ is exactly "sailors with no reservation", because both operands are **sets of sids** with the same domain.
- $\pi_{\text{sid, bid}}(Reserves)$ loses the day. Anything that needs the day must project **after** the day has been used, never before.
- Idempotence: $\pi_{\text{sid}}\big(\pi_{\text{sid, bid}}(Reserves)\big) = \pi_{\text{sid}}(Reserves)$.

1.6 Rename: the three forms

Form	Effect
$\rho_X(R)$	the relation called R is called X; attributes unchanged
$\rho_{X\,(a_1,\ldots,a_n)}(R)$	relation renamed to X **and** the i-th attribute renamed to a_i, in schema order
$\rho_{a_1 \leftarrow b_1, a_2 \leftarrow b_2}(R)$	only the listed attributes are renamed, which is what you need when the relation name must stay

Self-joins **require** a rename on at least one side; otherwise $Sailors \times Sailors$ is ill-formed because every attribute name is duplicated. This one-mark question appears in almost every paper.

1.7 Union compatibility, and why it is always tested

$R \cup S$, $R - S$ and $R \cap S$ are defined only if

1. R and S have the **same number** of attributes, and
2. the i-th attribute of R has the **same domain** as the i-th attribute of S, for every i.

Attribute **names** need not agree; the result takes its names from the left operand. This is why the standard solution to "sailors who never reserved a red boat" must project **both** sides to the same attribute list before subtracting:

$$\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \text{red}}(Boats))$$

The left operand has schema ("sid"); the right operand has been reduced to ("sid") as well. Subtracting a two-attribute relation from a one-attribute relation is the single most common error in the subject.

Trap / common mistake. $Sailors - Reserves$ is not union compatible: four attributes against three, and only "sid" in common. Worse, $\pi_{\text{sid, sname}}(Sailors) - \pi_{\text{sid}}(Reserves)$ is **also** illegal (two attributes against one). Always project both sides to the **same list**.

1.8 Cartesian product, theta join, equijoin, natural join

Given R (A, B) and S (C, D), the product $R \times S$ has schema (A, B, C, D) and $|R| \cdot |S|$ tuples. If the two relations share an attribute name, the product is illegal until one side is renamed: the result would have two attributes of the same name and no way to refer to either.

Join form	Meaning and schema

$R \underset{\theta}{\bowtie} S$	$\sigma_\theta(R \times S)$. Schema is the concatenation of both. If θ names an attribute that both relations own, it is ambiguous unless qualified.
$R \underset{\mathrm{R}.A = \mathrm{S}.A}{\bowtie} S$	**Equijoin** on A. Both copies of A remain in the schema; project one away if the question asks for a clean schema.
$R \bowtie S$	**Natural join**. Legal in a well-posed question only when the shared attributes are exactly the ones that should be equated. One copy of each shared attribute survives.
$\rho_{R1}(R) \times \rho_{R2}(R)$	the **self-product**, the workhorse of every "pairs", "same" and "maximum" query in Parts 3 and 5.

In **Database System Concepts** the unadorned $\bowtie$ denotes the natural join, and it degenerates to $\times$ when the operands share no attribute name. This book follows that convention. When a question needs an equijoin on attributes that are **not** identically named — for example `Doctor.SSN = Patient.PrimaryDoctor_SSN` — an explicit theta join is written and the duplicated column is projected away.

1.9 Division: the only operator that answers "for all"

Definition. Let R (X, Y) and S (Y), where Y is the entire schema of S and a subset of the schema of R. Then

$$R \div S = \{t \in \pi_X(R) : \forall s \in S, (t, s) \in R\}$$

so $\mathrm{attr}(R \div S) = X = \mathrm{attr}(R) - \mathrm{attr}(S)$.

The fundamental identity — memorise it, because it **is** the answer to "derive division from the other five operators":

$$R \div S = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$$

Read it as: take every candidate answer t; form all pairs (t, s) that a perfect answer would have to contain; subtract the pairs actually present; every t that still has a missing pair is disqualified; remove those from the candidates.

Three practical rules for division in exams:

1. **The divisor must have exactly the schema you want to quantify over.** "all boats" means $\pi_{\text{bid}}(Boats)$; "all red boats" means $\pi_{\text{bid}}(\sigma_{\text{color} = \textbf{red}}(Boats))$; "all boats reserved by sailor 22" means $\pi_{\text{bid}}(\sigma_{\text{sid} = 22}(Reserves))$.
2. **The dividend must already be projected to exactly (answer attributes + divisor attributes).** If it carries an extra column, division silently answers a different question, because the extra column survives into the result.
3. **Empty divisor edge case.** If S is empty, every tuple of $\pi_{X(R)}$ is an answer, even when R is empty. That is the vacuous truth which makes "sailors who reserved every red boat" return **all sailors** when no red boat exists. Examiners award a mark for noticing it; see Q4.19.

1.10 The derived operators you are allowed to use

Only π, σ, ρ, $\cup$, $-$ and $\times$ are primitive; $\cap$, $\bowtie$ and $\div$ are derived and must be expandable on demand.

Derived form	Primitive definition
$R \cap S$	$R - (R - S)$
$R \underset{\theta}{\bowtie} S$	$\sigma_\theta(R \times S)$
$R \bowtie S$	$\pi_{\mathrm{attr}(R) \cup \mathrm{attr}(S)}\left(R \underset{\mathrm{R}.a_1 = \mathrm{S}.a_1 \wedge \ldots}{\bowtie} S\right)$ over the shared attributes $a_1, \ldots$
$R \div S$	$\pi_X(R) - \pi_X(\pi_X(R) \times S - R)$, with $X = \mathrm{attr}(R) - \mathrm{attr}(S)$
$\sigma_{\theta_1 \vee \theta_2}(R)$	$\sigma_{\theta_1}(R) \cup \sigma_{\theta_2}(R)$, valid because both results are subsets of the same relation

semijoin / antijoin	**Not classical.** Semijoin is $\pi_{\text{attr}(R)}(R \bowtie S)$; antijoin is $R - \pi_{\text{attr}(R)}(R \bowtie S)$. Usable as an annotation, never as the main answer in a classical-RA paper.

1.11 What set semantics costs, and where aggregates are forbidden

Classical relational algebra is defined over **sets**. There is no duplicate count, no order, and therefore:

- "how many", "more than 20 prescriptions", "total units", "average price", "rank by salary" are **not first-order expressible**. They need the extended grouping operator $\gamma_{\text{group, agg(attr)}}(R)$ or SQL's `COUNT/SUM/AVG`.
- "maximum" and "minimum" **are** expressible, but only as a set difference against a self-join, and only as **values**. "The most expensive medicine" is a maximum-of-values question and is expressible; "the book borrowed the most times" is a counting question in disguise and is not.
- "exactly k" for a **fixed numeral** k is expressible without aggregation (Q5.26(a), §10.6), because it is a Boolean combination of existence conditions over **distinct** tuple variables. "Exactly k" where k is a **stored attribute value** is not.

That distinction — **fixed cardinality is expressible, computed cardinality is not** — is worth a full question in most papers and is treated systematically in Q5.22–Q5.27 and §10.6.

1.12 Evaluating an expression: precedence and walking the tree

There is no universal precedence agreement, so every compound expression in this book is fully parenthesised. The **de facto** convention is

$$\text{precedence}: \quad \pi, \sigma, \rho \quad > \quad \times, \bowtie \quad > \quad \cap \quad > \quad \cup, -$$

with binary operators left-associative. When in doubt add brackets: an examiner cannot mark a bracketed expression wrong for being explicit, but a badly bracketed expression that changes the meaning loses the marks for the actual query.

Read an expression bottom-up. At every node ask: what is its schema, and which English condition does it implement? Every worked solution in Parts 6–8 does this node by node using names like R_1, R_2, Ans.

1.13 The recipe: seven steps that work on any question

1. **Underline the quantifier words.** at least one / all / only / none / exactly one / more than / the maximum / the same / both / either.
2. **Write the answer schema.** ("names" → "sname"; "sailors" → "sid"; "pairs" → two copies of an identifier).
3. **Write the source schema**: which relations actually carry those attributes.
4. **Pick the pattern** from §1.14. The quantifier word decides the pattern; everything else is plumbing.
5. **Build inner to outer**, naming each intermediate relation and writing its schema beside it.
6. **Check union compatibility** at every set operation, and **check for ambiguity** at every join.
7. **Check the edge cases**: empty relations, ties, duplicates, "no records at all", and whether the query is expressible at all.

1.14 The 20-pattern catalogue

Every question in Parts 2–9 is an instance of one of these twenty shapes. Learn the shape, not the example.

No.	English trigger	Shape	Where solved
1	at least one; some; who did X	$\pi_{\text{id}}(A \bowtie B \bowtie \sigma_{\theta(C)})$	Q2.6, Q6.4
2	none; never; who did not	$\pi_{\text{id}(A)} - \pi_{\text{id}}(A \bowtie B \bowtie \sigma_{\theta(C)})$	Q3.9, Q6.12
3	both X and Y	pattern 1 twice, then $\cap$	Q6.10, Q7.12

No.	English trigger	Shape	Where solved
4	X but not Y	pattern 1 for X minus pattern 1 for Y	Q6.11, Q7.13
5	only X	all minus the ones related to a non-X	Q6.31, Q7.31
6	all; every; each	division, or pattern 2 with the counter-example set	Q4.1–Q4.22
7	pairs; same; more than each other	ρ twice, $\times$, σ on the two copies	Q3.4, Q6.15
8	maximum; most expensive; oldest	$R - \pi_{\text{attr}(\sigma_{\text{R1}.a < \text{R2}.a}(\rho_{\text{R1}}(R) \times \rho_{\text{R2}}(R)))}$	Q5.1, Q5.8
9	minimum; cheapest; earliest	pattern 8 with the inequality reversed	Q5.2, Q5.11
10	second highest	maximum of the relation after removing the maximum	Q5.5, Q5.6
11	k-th highest, ties preserved	$k-1$ nested "remove the maximum" steps	Q5.7
12	per-group maximum	no better tuple exists **within the same group**	Q5.14–Q5.16
13	exactly one	pattern 1 minus "at least two"	Q5.24
14	at least two	self-join of the relationship with $\neq$ on the identifiers	Q5.23, Q6.21
15	exactly the same set	two divisions: $A \div B$ and $B \div A$	Q7.33, Q8.21
16	all … that satisfy C	divide by $\pi\left(\sigma_{C(\ldots)}\right)$	Q4.12, Q6.35
17	only … that satisfy C	pattern 5 applied to $\sigma_{\neg C}$	Q6.33, Q8.14
18	maximum within a selection	apply σ first, then pattern 8	Q5.18, Q5.19
19	same set of X for two identifiers	pattern 15	Q7.33
20	count / sum / average / "more than n"	**not expressible**; needs γ	Q5.27, Q7.24, Q8.3

1.15 The five words that decide the answer

Word	Logical form	Operator that implements it
at least one	$\exists$	join, then π
none / never	$\neg\exists$	set difference
all / every	$\forall$	division, or — against a counter-example set
only	$\forall$ over the **complement**	— of the "bad" tuples
exactly one	$\exists$ and not (at least two)	difference of joins

These five are not interchangeable, and papers deliberately mix them. "Has all required books" is a $\forall$; "has at least one required book" is an $\exists$; "has only required books" is a $\forall$ over the **complement** of the required set. §10.2 puts the confusable pairs side by side.

1.16 Assumptions declared once, used everywhere

1. **Set semantics.** No duplicates; every operator is a set operator; π eliminates duplicates.
2. **No NULLs in the classical core.** Where a schema has an optional value — only `Borrow.ReturnDate` in this book — we encode "not yet returned" by an explicit constant $\perp$, and say so. A question that stipulates SQL NULLs is answered with the classical expression **plus** an explicit statement of the convention; see Q7.30.
3. **Keys are as declared.** "Same record" is decided by the primary key; "same value" is decided by the attribute. That difference is exactly what separates Q6.15 from Q6.16.
4. **Ties are preserved.** No expression in this book silently returns "one of the maximum"; where a maximum is asked for, all of them are returned. When a question demands exactly one row, we say plainly that classical relational algebra cannot choose deterministically.
5. **Domains are compatible** whenever a comparison is made: numbers with numbers, strings with strings, and dates in ISO form YYYY-MM-DD, whose lexicographic order is chronological.

1.17 Coverage map

This table is the contract for the book; Part 12 repeats it with tick marks and answer references.

Category	Part	Questions	Checklist
Selection, projection, rename	2	Q2.1–Q2.14	C1
Product, theta join, equijoin, natural join	2–3	Q2.15–Q2.20, Q3.1–Q3.3	C2
Union, intersection, difference, union compatibility	3	Q3.5–Q3.14	C3
Self-joins, pairs, "same as"	3, 6	Q3.4, Q6.15–Q6.21	C4
Negative queries: never, none, without, except	3	Q3.9–Q3.14	C5
Division and universal quantification	4	Q4.1–Q4.22	C6
Maximum, minimum, ties, k-th highest	5	Q5.1–Q5.13	C7
Group-wise extremes, ranking, exactly-k	5	Q5.14–Q5.27	C8
Sailors–Boats complete bank	6	Q6.1–Q6.37	C9
Library complete bank	7	Q7.1–Q7.33	C10
Hospital complete bank incl. the five exam questions	8	Q8.1–Q8.27	C11
Mixed multi-technique queries	9	Q9.1–Q9.14	C12
Equivalence, error detection, simplification	10	Q10.1–Q10.18	C13
Not expressible in classical RA	5, 7, 8, 10	Q5.27, Q7.24, Q8.3, Q10.16–Q10.18	C14

Part 2 — Basic university-level questions

What this part fixes. One-relation queries (selection, projection, rename), the two-relation join, the Cartesian product, and the three ways a question can be **almost** right. Schema used: Sailors–Boats–Reserves for Q2.1–Q2.14 and Q2.17–Q2.20, and the employee schema $R(\text{Emp})(\text{eid}, \text{ename}, \text{sal}, \text{deptno})$ / $R(\text{Dept})(\text{deptno}, \text{dname}, \text{loc})$ for Q2.15–Q2.16.

2.1 One relation: selection and projection

Q2.1 **List the complete contents of the relation `Sailors`.**

Answer. $Sailors$
Why. No operator is needed. In an exam, write the relation name and **state its schema**: (sid, sname, rating, age). A bare relation name is a legal relational-algebra expression — the trivial query.

Trap / common mistake. Writing $\pi_{\text{sid, sname, rating, age}}(Sailors)$ is also correct and is **not** wrong, but it is not "all tuples in the stored order": relational algebra has no order, and π over all attributes returns the same set. Do not claim it preserves input order.

Q2.2 Find the names of all sailors.

Answer.

$\pi_{\text{sname}}(Sailors)$
Result schema: (sname). One row per **distinct** name — two sailors called "Jones" produce one row. That is correct set semantics, not a bug.
Why. "Names" fixes the projection list; no condition is given, so no σ.

Trap / common mistake. The classic error is $\pi_{\text{sname}}(Reserves)$ — `Reserves` has no `sname`. Projection can only name attributes that are in the operand's schema.

Q2.3 Find all sailors whose rating is greater than 7.

Answer.

$\sigma_{\text{rating} > 7}(Sailors)$
Schema unchanged: (sid, sname, rating, age).
Why. "Greater than" is a comparison on a numeric attribute, so σ alone suffices. The question asks for **sailors**, not names, so nothing is projected away.

Trap / common mistake. $\sigma_{\text{rating} > 7}(Sailors)$ quotes a number — a type error. Also $\sigma_{\text{rating} > 7}(\pi_{\text{sname}}(Sailors))$ is illegal: `rating` has been projected away before the selection could use it. **Select before you project.**

Q2.4 Find the names and ages of sailors whose age is at least 30 but not more than 40.

Answer.

$\pi_{\text{sname, age}}\left(\sigma_{\text{age} \geq 30 \wedge \text{age} \leq 40}(Sailors)\right)$
Result schema: (sname, age).
Why. Two comparisons joined by $\wedge$ inside one selection; the projection list follows the order the question asked for.

Trap / common mistake. $\sigma_{30 \leq \text{ age} \leq 40}$ is not standard relational-algebra syntax in most textbooks; write $\sigma_{\text{age} \geq 30 \wedge \text{age} \leq 40}$. Also, "at least" is $\geq$ (not $>$) and "not more than" is $\leq$.

Q2.5 Find the sailors whose rating is either 8 or 10.

Answer.

$\sigma_{\text{rating} = 8 \vee \text{rating} = 10}(Sailors)$
Schema unchanged.
Why. One selection with a disjunction. Equivalently $\sigma_{\text{rating} = 8}(Sailors) \cup \sigma_{\text{rating} = 10}(Sailors)$, because the two sets of tuples are both subsets of `Sailors` and are disjoint — the union cannot create a tuple that fails the condition.

Trap / common mistake. $\sigma_{\text{rating} = 8 \wedge \text{rating} = 10}(Sailors)$ is empty, not "either". $\wedge$ inside a single selection applies to the **same** tuple; a tuple cannot have two ratings.

Q2.6 Find the names of sailors who have reserved boat 103.

Answer.

$\pi_{\text{sname}}(\sigma_{\text{bid} = 103}(Reserves) \bowtie Sailors)$

$\sigma_{\text{bid}=103}(Reserves)$ has schema (sid, bid, day). The natural join with `Sailors` equates `sid`, giving (sid, bid, day, sname, rating, age); the projection leaves (sname).

Why. Two relations are involved — the reservation and the sailor's name — so a join on the common attribute `sid` is unavoidable. Selecting on `bid` **before** joining keeps the intermediate relation small, which is also the efficient plan.

Trap / common mistake. Writing $\pi_{\text{sname}}(\sigma_{\text{bid}=103}(Sailors))$ fails: `Sailors` has no `bid`. Writing $\pi_{\text{sname}}(Sailors \times \sigma_{\text{bid}=103}(Reserves))$ is legal but returns the same answer only because the product is filtered by nothing — it pairs **every** sailor with the reservation. The natural join is what enforces "the same sailor".

Q2.7 Find the names of sailors who have reserved a red boat.

Answer.

$$\pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color}='\text{red}'}(Boats))$$

(sid, sname, rating, age, bid, day, bname, color) before projection; after projection, (sname).

Why. "A red boat" is an existential condition: there must **exist** a reservation row linking this sailor to a boat whose colour is red. A chain of natural joins realises the existential quantifier; the projection then removes the witnesses. This is pattern 1 of §1.14.

Trap / common mistake. The join order is free but the **joins are not optional**. Dropping `Sailors` gives boat names, not sailor names; dropping `Reserves` gives the cartesian product of sailors and red boats, i.e. every sailor.

Q2.8 Find the names of sailors who reserved a boat on 2019-10-01.

Answer.

$$\pi_{\text{sname}}\big(Sailors \bowtie \sigma_{\text{day}='2019\text{-}10\text{-}01'}(Reserves)\big)$$

Result schema (sname).

Why. The date is an attribute of `Reserves`, so the selection is pushed onto `Reserves` and only the `sname` lookup needs the join with `Sailors`.

Trap / common mistake. Dates in these notes are ISO strings, so string equality is correct. If your course stores dates as $A(\text{DD})/A(\text{MM})/A(\text{YYYY})$ strings, comparison is still equality, but **range** comparisons would then sort wrongly — see Q7.25.

Q2.9 Find all the colours in which boats exist.

Answer.

$$\pi_{\text{color}}(Boats)$$

Result schema: (color), a set of colours.

Why. Projection with duplicate elimination is exactly "the distinct values of an attribute". This is the one place where a student who is used to SQL must resist adding anything: there is no DISTINCT keyword in relational algebra because duplicate removal is built into π.

Trap / common mistake. Asking for "all colours" of a **specific** boat is different: that is $\pi_{\text{color}}(\sigma_{\text{bid}=103}(Boats))$, which returns at most one row.

Q2.10 Find the sids and names of sailors whose rating is 10, together with the boats they have reserved.

Answer.

$$\pi_{\text{sid, sname, bid}}\big(\sigma_{\text{rating}=10}(Sailors) \bowtie Reserves\big)$$

(sid, sname, bid). Note `rating` is used in the selection but is not in the answer, so it is projected away **after** being used.

Why. The selection is applied to `Sailors` **before** the join so that `rating` is still available. This is the single most frequently examined ordering point in Part 2.

Trap / common mistake. $\pi_{\text{sid, sname, bid}}\big(\sigma_{\text{rating}=10}(Sailors \bowtie Reserves)\big)$ is also correct — selection distributes over the join because `rating` survives the join. But $\sigma_{\text{rating}=10}\big(\pi_{\text{sid, sname, bid}}(Sailors \bowtie Reserves)\big)$ is a **type**

error: `rating` is no longer in the schema after the projection. Rule: **a condition may only mention attributes present in the operand's schema.**

Q2.11 Using only π and σ, express "the sname of every sailor who is older than every sailor with rating 10". Is it possible without a join?

Answer. **No.** A selection can only test attributes of one tuple, so it cannot compare one sailor's age with another sailor's age. A self-join (or a division-style difference) is required:

$$\pi_{\text{sname}}(Sailors) - \pi_{\text{sname}}\big(Sailors \bowtie (\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(\sigma_{\text{age}\,>60}(Sailors)))\big)$$

Why. This is a **conceptual** question, and examiners use it to test whether you know that σ is tuple-local. The second expression is only a sketch: it hard-codes a threshold. The correct general expression needs a self-product — see Q5.20 for the fully general form.

Trap / common mistake. Do not answer "yes, with a nested selection". Nested σs compose conditions on the **same** tuple; they never introduce a second tuple variable.

Q2.12 Explain why $\pi_{\text{sname}}(Sailors) - \pi_{\text{sname}}\big(\sigma_{\text{rating}\,<8}(Sailors)\big)$ does not compute "names of sailors whose rating is at least 8".

Answer. It computes "names that appear on some sailor with rating ≥ 8 **and** do not appear on any sailor with rating < 8". Both are sets of **names**, and names are not keys.

Why. `sname` is not a key of `Sailors`: two different sailors may share a name. If "Jones" has one tuple with rating 9 and another with rating 7, then Jones appears on both sides and is subtracted away, even though a rating-≥ 8 Jones exists.

Both operands have schema (sname), so the expression is **well-typed** — the error is semantic, not syntactic. That is exactly why it is a favourite exam trap.

Trap / common mistake. The safe rule: **do set operations on keys, not on descriptive attributes.** Write $\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}\big(\sigma_{\text{rating}\,<8}(Sailors)\big)$ and join back to `Sailors` for the name.

2.2 Rename and aliasing

Q2.13 Give three different correct ways to write "the names of sailors with rating 8", using ρ at least once.

Answer. (i) $\pi_{\text{sname}}\big(\sigma_{\text{rating}\,=8}(Sailors)\big)$; (ii) $\pi_{\text{sname}}\big(\sigma_{\text{rating}\,=8}(\rho_{\text{S}}(Sailors))\big)$; (iii) $\pi_{\text{S.sname}}\big(\sigma_{\text{S.rating}\,=8}(\rho_{\text{S}}(Sailors))\big)$

All three return (sname). In (iii) the qualified names are needed only if another relation with the same attribute names is in scope.

Why. ρ changes only the **name** used to refer to the relation; it does not change tuples or schema. Renaming is therefore always semantics-preserving when the new name is fresh.

Trap / common mistake. Renaming to a name already in scope, e.g. $\rho_{\text{Sailors}}(Sailors)$ inside an expression that also uses the original `Sailors`, creates an ambiguity rather than an alias. Use fresh names such as $R(\text{S})$, $R(\text{S1})$, $R(\text{S2})$.

Q2.14 Write the expression for "names of sailors who are older than 30" so that the result's attribute is called `name` instead of `sname`.

Answer.

$$\rho_{\text{name} \leftarrow \text{sname}}\big(\pi_{\text{sname}}\big(\sigma_{\text{age}\,>30}(Sailors)\big)\big)$$

or equivalently $\pi_{\text{name}}\big(\rho_{\text{name} \leftarrow \text{sname}}\big(\sigma_{\text{age}\,>30}(Sailors)\big)\big)$

Result schema: (name).

Why. The third form of ρ renames **attributes only**, which is what is wanted when the relation name is irrelevant but the output column must match a specification.

Trap / common mistake. $\rho_{\text{name}}(\pi_{\text{sname}}(\ldots))$ renames the **relation** to "name", not the attribute. The arrow form is the attribute rename.

2.3 Cartesian product, theta join, equijoin, natural join

Q2.15 **Find every (employee, department-name) pair such that the employee works in that department. Then find every possible pair of employees and departments, and explain the difference.**

Answer.

$$\text{Working pairs}: \quad \pi_{\text{ename, dname}}\left(Emp \underset{\text{Emp.deptno} = \text{Dept.deptno}}{\bowtie} Dept\right)$$

$$\text{All possible pairs}: \quad Emp \times Dept$$

The product has schema (eid, ename, sal, deptno, deptno, dname, loc) — `deptno` appears **twice**, which is exactly why the product must be renamed or qualified before it can be used. The theta join keeps both copies too; only the natural join merges them.

Why. The product pairs **every** employee with **every** department, including combinations that do not correspond to employment. The theta join adds the predicate that makes the pair meaningful. Cardinalities: $|Emp| \cdot |Dept|$ versus at most $|Emp|$.

Trap / common mistake. $Emp \times Dept$ is only legal here because the shared attribute `deptno` is **qualified** in any subsequent reference; strictly, the definitions in SKS require the operands to have disjoint attribute sets. If you must write the product, rename first: $\rho_{\text{E}}(Emp) \times \rho_{\text{D}}(Dept)$.

Q2.16 **Find the names of employees who earn more than every employee in department 10.**

Answer. Let $R_1 = \rho_{\text{E}}(Emp) \times \rho_{\text{D10}}\big(\sigma_{\text{deptno}\,=10}(Emp)\big)$ and $R_2 = \pi_{\text{E.eid}}\big(\sigma_{\text{E.sal}\,\leq\,\text{D10.sal}}(R_1)\big)$. Then

$$\pi_{\text{ename}}(Emp \bowtie (\pi_{\text{eid}}(Emp) - R_2))$$

R_1: (E.eid, E.ename, E.sal, E.deptno, D10.eid, D10.ename, D10.sal, D10.deptno). R_2: (E.eid) — employees who are beaten by, or equal to, **someone** in department 10. The final join has schema (eid, ename, sal, deptno), projected to (ename).

Why. "More than every x" is "$\neg$ ($\leq$ some x)". The σ finds the witnesses that disqualify an employee; set difference removes exactly those employees. This is pattern 6 (the counter-example method) applied without division, because the condition is a comparison rather than a membership test.

Trap / common mistake. Two errors are common. (i) Using π_{ename} for the difference: two employees may share a name, so the subtraction removes innocent employees — always subtract on the **key**, `eid`, then join back for the name. (ii) Using $<$ instead of $\leq$ in R_2: the employee who **equals** the department-10 maximum must also be excluded, because the question says **more than every**, not **at least**.

Q2.17 **State the difference between the natural join and the equijoin of `Emp` and `Dept`, and give the schema of each.**

Answer.

$$\text{Equijoin}: \quad Emp \underset{\text{Emp.deptno} = \text{Dept.deptno}}{\bowtie} Dept \qquad \text{Natural join}: \quad Emp \bowtie Dept$$

Nothing: the two expressions return the **same tuples**. They differ in schema: the equijoin keeps two `deptno` columns, the natural join keeps one.

Why. The natural join is defined as an equijoin on **all** same-named attributes followed by removal of the duplicate column. Here the only shared name is `deptno`, so the two coincide on tuples. If the relations also shared an unrelated name — say both had `name` — the natural join would equate that too, which is usually a bug.

Trap / common mistake. In an exam, "natural join is an equijoin" is only half a mark. The full answer is: **equijoin on all common attributes, then project away one copy of each common attribute.** And if the relations share no attribute name, the natural join degenerates to a Cartesian product — a fact examiners like to test.

Q2.18 Find the names of sailors who reserved a red boat, together with the boat name, using a three-relation join. State the schema at each step.
Answer.

$\pi_{\text{sname, bname}}(\sigma_{\text{color} = \textbf{red}}(Boats) \bowtie Reserves \bowtie Sailors)$
After σ: (bid, bname, color). After the first join with `Reserves` on `bid`: (bid, bname, color, sid, day). After the second join with `Sailors` on `sid`: adds (sname, rating, age). After projection: (sname, bname).
Why. Each natural join consumes exactly one shared attribute, so a chain of joins walks the foreign-key path $R(\text{Boats}) \to R(\text{Reserves}) \to R(\text{Sailors})$. Left-deep order (boat first) is the cheapest because the selection shrinks it to red boats only.

Trap / common mistake. The joins must be **natural** on the right attribute. If the schema used different names (say `Reserves.boatid`), the natural join would silently degenerate into a product and the answer would multiply rows. Always check which attributes are shared.

Q2.19 Find the sids of sailors who reserved a boat before 2019-10-05 and whose rating is at least 8.
Answer.

$\pi_{\text{sid}}\big(\sigma_{\text{rating} \geq 8}(Sailors) \bowtie \sigma_{\text{day} < \textbf{2019-10-05}}(Reserves)\big)$
Result schema: (sid).
Why. Both conditions are tuple-local to their own relation, so both selections are pushed down to the leaves. Only then is the join performed. Pushing selections down is always safe (§10.5) and reduces the intermediate size.

Trap / common mistake. Do not put both conditions in one σ over the join result and then forget that the attribute `sid` is ambiguous — after a natural join there is only one `sid`, so $\sigma_{\text{rating} \geq 8 \wedge \text{ day} < \textbf{2019-10-05}}(Sailors \bowtie Reserves)$ is legal too. Either is accepted; the pushed-down version is better style.

Q2.20 Using $\times$ and σ only (no $\bowtie$), express Q2.7.
Answer.

$\pi_{\text{sname}}(\sigma_{\text{Sailors.sid} = \text{Reserves.sid} \wedge \text{Reserves.bid} = \text{Boats.bid} \wedge \text{color} = \textbf{red}}(Sailors \times Reserves \times Boats))$
The product has 4 + 3 + 3 = 10 attributes; the result is (sname).
Why. This is the definition of the join expanded into the primitive operators, and it is what an examiner is checking when the question says "using only the primitive operations". The three conjuncts are: the two foreign-key equalities plus the colour filter.

Trap / common mistake. $Sailors \times Reserves \times Boats$ is **ambiguous** as written — all three relations share no name except through `sid`/`bid`, but `Sailors` and `Reserves` both have `sid`, and `Reserves` and `Boats` both have `bid`. A product is legal only after renaming, e.g. $\rho_{\text{S}}(Sailors) \times \rho_{\text{R}}(Reserves) \times \rho_{\text{B}}(Boats)$, or if you qualify every reference as above. When in doubt, rename.

2.4 The most common Part-2 errors, collected

Wrong expression	Why it is wrong, and the fix
$\pi_{\text{sname}}(Reserves)$	`Reserves` has no `sname`. Project only attributes that exist; join first.
$\sigma_{\text{rating} > \textbf{7}}(Sailors)$	numeric constant quoted as a string. Remove the quotes.
$\sigma_{\text{color} = \textbf{red}}(\pi_{\text{bid}}(Boats))$	type error: `color` was projected away. Select before projecting.

$Sailors - Reserves$	not union compatible (4 vs 3 attributes, only `sid` shared). Project both sides to the same list.
$Sailors \times Sailors$	ambiguous attribute names. Rename one copy with ρ.
$Sailors \bowtie Boats$	natural join on no shared attribute = Cartesian product: 3 × 3 attributes, every pair. Not an error, but almost never what is wanted.
$Sailors \underset{\text{sid} = \text{bid}}{\bowtie} Boats$	compares a sailor id with a boat id. Usually a thinko; check the join attribute.
$\pi_{\text{sname, sname}}(Sailors)$	duplicate attribute in a projection list. Write each attribute once.

Part 3 — Joins, self-joins, set operations and negative queries

What this part fixes. The three set operators and union compatibility in practice; the self-join as a device for comparing two rows; and the "never / without / except" family, which is always a set difference on **keys**. Every question in this part uses the Sailors–Boats–Reserves schema of §1.2A.

3.1 Set operations in practice

Q3.1 Find the names of sailors who have reserved a red boat or a green boat.
Answer.

$$\pi_{\text{sname}}\big(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \textbf{red} \vee \text{color} = \textbf{green}}(Boats)\big)$$

Result schema: (sname), a set.
Why. The condition is on a **single** boat's colour, so it is one selection with a disjunction — not a union of two queries. The corresponding union form, $\pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats)) \cup \pi_{\text{sname}}(\ldots)$, is also correct and returns the same set, but it is longer and does more work.

Trap / common mistake. Do not write $\sigma_{\text{color} = \textbf{red}}(Boats) \cup \sigma_{\text{color} = \textbf{green}}(Boats)$ and then project `sname`: those are two sets of **boat** tuples, and `sname` is not in their schema. Also, "or" over the **same** tuple is `or`; "or" over **different** tuples needs union.

Q3.2 Find the names of sailors who have reserved both a red boat and a green boat.
Answer. Let $R(\text{Red}) = \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats))$ and $R(\text{Green}) = \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} = \textbf{green}}(Boats)\big)$. Then

$$\pi_{\text{sname}}(Sailors \bowtie (R(\text{Red}) \cap R(\text{Green})))$$

$R(\text{Red})$ and $R(\text{Green})$ both have schema (sid), so they are union compatible and the intersection is legal. Result: (sname).
Why. "Both" is two **independent existences**: there must be one reservation witnessing red and another witnessing green. Because each are computed separately and then intersected on `sid`, the two witnesses are allowed to be different reservations. This is the crucial difference from Q6.32's "same reservation" variant.

Trap / common mistake. `sname` is not a key, so intersecting on names loses sailors who share a name — and $R(\text{Red}) \cap R(\text{Green})$ with names also **gains** a false positive when two different sailors with the same name witness the two colours. Intersect on `sid`; join to `Sailors` afterwards.

Q3.3 Find the names of sailors who have reserved a red boat but not a green boat.
Answer.

$$\pi_{\text{sname}}\big(Sailors \bowtie \big(\pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats)) - \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} = \textbf{green}}(Boats)\big)\big)\big)$$

Both operands of the difference are (sid), hence union compatible; the join with `Sailors` restores `sname`.

Why. "A but not B" is the set difference of the **key** sets of A and B (pattern 4). The difference must be taken **before** joining to `Sailors`, because after the join the schema is wider and the subtraction would no longer be a statement about sailors.

Trap / common mistake. $\pi_{\text{sname}}(\text{Red}) - \pi_{\text{sname}}(\text{Green})$ is the name-set error of Q2.12 again. Also, this query is **not** "sailors who reserved only red boats" — a sailor who reserved red and blue qualifies here but not there. That is Q6.31.

Q3.4 Find the names of sailors who have reserved a red boat but not a boat of any other colour. In other words: sailors all of whose reservations are of red boats, and who have at least one reservation.

Answer. Let $R(\text{Bad}) = \pi_{\text{sid}}\big(\mathit{Reserves} \bowtie \sigma_{\text{color} \neq \textbf{red}}(\mathit{Boats})\big)$. Then

$$\pi_{\text{sname}}(\mathit{Sailors} \bowtie (\pi_{\text{sid}}(\mathit{Reserves}) - R(\text{Bad})))$$

$R(\text{Bad})$: (sid). $\pi_{\text{sid}}(\mathit{Reserves})$: (sid). Difference: (sid). Result: (sname).

Why. This is pattern 5 ("only X"): **all** − **any non-X**. Note that $\pi_{\text{sid}}(\mathit{Reserves}) - R(\text{Bad})$ also enforces "at least one reservation" automatically, because a sailor with no reservations is not in $\pi_{\text{sid}}(\mathit{Reserves})$ in the first place.

If the question were "sailors who reserved only red boats" **including** sailors with no reservations at all (a pedantic but real reading), the answer would be $\pi_{\text{sname}}(\mathit{Sailors}) - \pi_{\text{sname}}(\mathit{Sailors} \bowtie R(\text{Bad}))$. Always state which reading you are using — examiners accept either **if** it is stated.

Trap / common mistake. Writing $\pi_{\text{sid}}(\sigma_{\text{color} = \textbf{red}}(\mathit{Boats}) \bowtie \mathit{Reserves})$ alone gives "has a red reservation", not "only red". The word **only** needs the negation of the complement.

Q3.5 Find the names of sailors who have never reserved a red boat.

Answer.

$$\pi_{\text{sname}}(\mathit{Sailors} \bowtie (\pi_{\text{sid}}(\mathit{Sailors}) - \pi_{\text{sid}}(\mathit{Reserves} \bowtie \sigma_{\text{color} = \textbf{red}}(\mathit{Boats}))))$$

Inner difference: (sid) on both sides. Result: (sname).

Why. "Never P" is **all sailors** minus **sailors who P** (pattern 2). The subtraction must be on `sid`, and only afterwards is the name looked up. The join with the difference is equivalent to a set intersection on `sid`, since $\pi_{\text{sid}}(\mathit{Sailors})$ is the key set.

Trap / common mistake. $\pi_{\text{sname}}(\mathit{Sailors}) - \pi_{\text{sname}}(\mathit{Sailors} \bowtie \mathit{Reserves} \bowtie \sigma_{\text{color} = \textbf{red}}(\mathit{Boats}))$ returns the right answer **only if names are unique**. It is the answer most students write and it is worth partial credit, but the key version is the correct one.

Q3.6 Find the boats that have never been reserved.

Answer. $\pi_{\text{bid}}(\mathit{Boats}) - \pi_{\text{bid}}(\mathit{Reserves})$

Both operands (bid) — union compatible. Result: (bid).

Why. The simplest negative query in the subject. Add $\pi_{\text{bname, color}}(\mathit{Boats} \bowtie (\ldots))$ if boat names are wanted; the **difference** itself must stay on the key.

Trap / common mistake. $\mathit{Boats} - \pi_{\text{sid, bid, day}}(\mathit{Reserves})$ is not union compatible (3 attributes versus 3, but `bid`, `bname`, `color` against `sid`, `bid`, `day` — different domains in positions 1 and 2). Project both sides to `bid`.

Q3.7 Find the sailors who have never made a reservation.

Answer.

$$\pi_{\text{sid}}(\mathit{Sailors}) - \pi_{\text{sid}}(\mathit{Reserves})$$

(sid) on both sides. Result: (sid).

Why. Pattern 2 with no join at all: the relationship `Reserves` is consulted directly. To return names instead, join the result back to `Sailors`.

Trap / common mistake. Do not use $\pi_{\text{sid, sname}}(Sailors) - \pi_{\text{sid}}(Reserves)$: two attributes against one. This is the most frequently penalised expression in the whole course.

Q3.8 Find the names of sailors who reserved at least two different boats.
Answer. Let $R(\mathrm{P}) = \pi_{\text{sid, bid}}(Reserves)$. Then

$$\pi_{\text{sname}}\Big(Sailors \bowtie \pi_{\text{P1.sid}}\Big(\sigma_{\text{P1.bid} \neq \text{P2.bid}}(\rho_{\text{P1}}(R(\mathrm{P})) \times \rho_{\text{P2}}(R(\mathrm{P})))\Big)\Big)$$

$R(\mathrm{P})$: (sid, bid). The product: (P1.sid, P1.bid, P2.sid, P2.bid). After σ and π: (sid) — the sailors for whom two **different** boats exist.
Why. "At least two" is a self-join of the relationship with a $\neq$ condition on the second identifier and an equality on the first. The projection to `(sid, bid)` first is essential: otherwise a sailor who reserved one boat on two days would count as two.

Trap / common mistake. Forgetting $\pi_{\text{sid, bid}}$ gives "at least two **reservations**", which is a different question (Q6.21). Also, the inequality must be between the **boat** attributes, not between the sid attributes.

Q3.9 Find the names of sailors who have reserved a boat on at least two different dates.
Answer.

$$\pi_{\text{sname}}\Big(Sailors \bowtie \pi_{\text{R1.sid}}\Big(\sigma_{\text{R1.sid} = \text{R2.sid} \wedge \text{R1.day} \neq \text{R2.day}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves))\Big)\Big)$$

Product schema: 6 attributes, (R1.sid, R1.bid, R1.day, R2.sid, R2.bid, R2.day). After σ and π: (sid).
Why. "At least two different dates" is a self-join, so the **same** relation appears twice under two names. Because $\times$ pairs everything with everything, the equality on `sid` (same sailor) and the inequality on `day` (different dates) must both be written explicitly in the σ.

Trap / common mistake. Do **not** use $\rho_{\text{R1}}(Reserves) \bowtie \rho_{\text{R2}}(Reserves)$: a natural join equates **all three** shared attributes, forcing the two reservations to be the same tuple, and the subsequent σ on `day` then returns nothing. General rule: **for a self-join with an inequality, use $\times$ plus an explicit σ; reserve $\bowtie$ for equality on all shared attributes.**

Q3.10 Find the sids of sailors who reserved boat 101 and also reserved boat 103.
Answer.

$$\pi_{\text{sid}}(\sigma_{\text{bid} = 101}(Reserves)) \cap \pi_{\text{sid}}(\sigma_{\text{bid} = 103}(Reserves))$$

Both operands (sid), union compatible. Alternatively, a self-join:
$\pi_{\text{R1.sid}}(\sigma_{\text{R1.bid} = 101 \wedge \text{R2.bid} = 103 \wedge \text{R1.sid} = \text{R2.sid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves)))$.
Why. Two independent existences on the same key set. Intersection is shorter and shows the intent; the self-join is what you must write if the question forbids $\cap$.

Trap / common mistake. $\pi_{\text{sid}}(\sigma_{\text{bid} = 101 \wedge \text{bid} = 103}(Reserves))$ is empty: one tuple cannot satisfy both. That is the "$\wedge$ on one tuple" trap again.

Q3.11 Find the names of sailors who have reserved a boat that is either red or green, but have never reserved a blue boat.
Answer. $\pi_{\text{sname}}(Sailors \bowtie (R(\mathrm{RG}) - R(\mathrm{Blue})))$ where $R(\mathrm{RG}) = \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} = \textbf{red} \vee \text{color} = \textbf{green}}(Boats)\big)$ and $R(\mathrm{Blue}) = \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{blue}}(Boats))$.
$R(\mathrm{RG})$ and $R(\mathrm{Blue})$ are both (sid). Result: (sname).
Why. The two requirements have different quantifiers: an **existential** one over red-or-green and a **negative** one over blue. Each is computed as a key set, and only then combined.

Trap / common mistake. Do not try to express this with a single σ over a three-way join: the positive and negative conditions range over **different tuples**, so no tuple-local condition can capture it.

Q3.12 Under what conditions is $R(\mathrm{R}) - R(\mathrm{S})$ equal to $R(\mathrm{S}) - R(\mathrm{R})$? Give an example with the sailors schema.

Answer. Only when the two operands contain the same set of tuples, i.e. $R(\mathrm{R}) = R(\mathrm{S})$; then both differences are empty. In every other case set difference is **not** commutative.
Both differences are defined only when the operands are union compatible.
Why. $R(\mathrm{R}) - R(\mathrm{S})$ removes from $R(\mathrm{R})$ those tuples also in $R(\mathrm{S})$. Swapping them changes which side is being reduced. Example: $T(\mathrm{A}) = \{1, 2\}$, $T(\mathrm{B}) = \{2, 3\}$ gives $T(\mathrm{A}) - T(\mathrm{B}) = \{1\}$ but $T(\mathrm{B}) - T(\mathrm{A}) = \{3\}$.

Trap / common mistake. $\cup$ and $\cap$ **are** commutative and associative; $-$ is neither associative nor commutative. This is a standard 2-mark theory question.

Q3.13 Show that $R(\mathrm{R}) \cap R(\mathrm{S})$ can be written with $-$ alone, and verify the identity on small relations.
Answer.

$$R(\mathrm{R}) \cap R(\mathrm{S}) = R(\mathrm{R}) - (R(\mathrm{R}) - R(\mathrm{S}))$$

All three expressions have the schema of $R(\mathrm{R})$.
Why. $R(\mathrm{R}) - R(\mathrm{S})$ is the part of $R(\mathrm{R})$ that is **not** in $R(\mathrm{S})$. Removing that part from $R(\mathrm{R})$ leaves exactly the part that **is** in $R(\mathrm{S})$ — the intersection. Verify with $T(\mathrm{A}) = \{1, 2, 3\}$, $T(\mathrm{B}) = \{2, 3, 4\}$: $T(\mathrm{A}) - T(\mathrm{B}) = \{1\}$, and $T(\mathrm{A}) - \{1\} = \{2, 3\} = T(\mathrm{A}) \cap T(\mathrm{B})$.

Trap / common mistake. The asymmetric-looking form $R(\mathrm{R}) - (R(\mathrm{R}) - R(\mathrm{S}))$ cannot be simplified to $R(\mathrm{S}) - (R(\mathrm{S}) - R(\mathrm{R}))$ unless you also swap the roles; both are correct, but $R(\mathrm{R}) - (R(\mathrm{S}) - R(\mathrm{R}))$ is **wrong** — it returns $R(\mathrm{R})$, because $R(\mathrm{S}) - R(\mathrm{R})$ is disjoint from $R(\mathrm{R})$.

Q3.14 A student writes $\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}\big(\sigma_{\text{rating}\,<8}(Reserves)\big)$. Explain two independent reasons why this is wrong, and repair it.
Answer. Correct version: $\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}\big(\sigma_{\text{rating}\,<8}(Sailors)\big)$.
The student's second operand is (sid) — the **types** match, so the error is not caught by union compatibility. That is what makes it a good exam question.
Why. Reason 1: `Reserves` has no `rating` attribute, so the inner selection is not even well-formed. Reason 2 (the deeper one): rating belongs to `Sailors`, so a condition on rating must be evaluated on `Sailors`, not on the reservation relation.

Trap / common mistake. Even after repairing the schema, resist the urge to project `sname` before the subtraction — see Q2.12.

3.2 Self-joins: comparing two rows

Q3.15 Find the pairs of sailors who have the same rating. Return sids, and do not report a pair twice.
Answer.

$$\pi_{\text{S1.sid},\ \text{S2.sid}}\big(\sigma_{\text{S1.rating} = \text{S2.rating} \wedge \text{S1.sid} < \text{S2.sid}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$$

Product schema: (S1.sid, S1.sname, S1.rating, S1.age, S2.sid, S2.sname, S2.rating, S2.age). Result: (S1.sid, S2.sid).
Why. Two rows of the same relation are needed, so the relation must appear twice under different names. The condition $A(\text{S1.sid}) < A(\text{S2.sid})$ does three jobs: it removes pairs of a sailor with himself, and it removes the mirror image (y, x) of every pair (x, y).

Trap / common mistake. Using $\neq$ instead of $<$ returns each unordered pair twice and includes self-pairs removed — actually $\neq$ removes self-pairs but keeps both orders. Using no inequality at all includes every sailor paired with himself. Read the question: "pairs" almost always means **unordered**, distinct pairs.

Q3.16 Find the sids of sailors who have the same rating as another sailor, but a different sid.
Answer.

$\pi_{\text{S1.sid}}\big(\sigma_{\text{S1.rating} = \text{S2.rating} \wedge \text{S1.sid} \neq \text{S2.sid}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$

Result: (sid) — note that a projection to one attribute is enough, because the question asks **which** sailors, not which pairs.

Why. This is Q3.15 with the pair collapsed to a single sailor. Projecting only $A(\text{S1.sid})$ and letting π remove duplicates is what turns "pairs" into "sailors".

Trap / common mistake. Because π removes duplicates, you do **not** need to add $A(\text{S1.sid}) < A(\text{S2.sid})$ here: each qualifying sailor appears in the result exactly once either way. Adding it is harmless but shows you understand **why** it was needed in Q3.15.

Q3.17 Find the names of sailors who are older than some other sailor with the same rating.

Answer.

$\pi_{\text{S1.sname}}\big(\sigma_{\text{S1.rating} = \text{S2.rating} \wedge \text{S1.age} > \text{S2.age}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$

Result: (sname).

Why. Two comparisons are needed on the **same** pair of tuples: equality on the grouping attribute, inequality on the compared attribute. This is the prototype of every "per-group" query in Part 5.

Trap / common mistake. $\pi_{\text{sname}}\big(\sigma_{\text{age} > \text{age}}(Sailors)\big)$ is always empty — the two `age` references are the **same** attribute of the **same** tuple. You need two tuple variables before you can compare a value with another value.

Q3.18 Find the pairs of sailors who have reserved the same boat.

Answer.

$\pi_{\text{R1.sid, R2.sid}}\big(\sigma_{\text{R1.bid} = \text{R2.bid} \wedge \text{R1.sid} < \text{R2.sid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves))\big)$

Result: (R1.sid, R2.sid).

Why. The relationship is `Reserves`, so the self-join is over `Reserves`, not over `Sailors`. The $<$ condition makes the pairs unordered and distinct.

Trap / common mistake. Note that the two sailors need not have reserved the boat on the same day — the equality is only on `bid`. That distinction is Q3.19.

Q3.19 Find the pairs of sailors who reserved the same boat on the same day.

Answer.

$\pi_{\text{R1.sid, R2.sid}}\big(\sigma_{\text{R1.bid} = \text{R2.bid} \wedge \text{R1.day} = \text{R2.day} \wedge \text{R1.sid} < \text{R2.sid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves))\big)$

Result: (R1.sid, R2.sid).

Why. One more conjunct, one more attribute in the equality. This is the general recipe for "same X as": self-product plus equality on X, plus inequality on the identifier if the pairs must be distinct and unordered.

Trap / common mistake. Do not use $\rho_{\text{R1}}(Reserves) \bowtie \rho_{\text{R2}}(Reserves)$ — the natural join equates all three attributes at once (same sailor, same boat, same day), which defeats the purpose. Use $\times$ and write the equalities you actually want.

Q3.20 Find the names of sailors who have reserved a boat with a higher bid than every boat reserved by sailor 22.

Answer. Let $R(\text{S22}) = \pi_{\text{bid}}(\sigma_{\text{sid} = 22}(Reserves))$ and $R(\text{Low}) = \pi_{\text{R1.sid}}(\sigma_{\text{R1.bid} \leq \text{R2.bid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(R(\text{S22}))))$. Then

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Reserves) - R(\text{Low})))$$

.

$R(\text{S22})$: (bid). The product: (R1.sid, R1.bid, R1.day, bid). $R(\text{Low})$: (sid).

Why. "$>$ every" is "$\neg$ ($\leq$ some)": find the sailors who have at least one reservation that is not higher than **some** boat of sailor 22, and remove them. This is the counter-example method, and it is the only method that works when the comparison is with a **set** rather than a single value.

Trap / common mistake. If sailor 22 has no reservations, $R(\text{S22})$ is empty, the product is empty, and **every** sailor with a reservation is returned. That is vacuously correct ("higher than every boat reserved by 22" is true when there is no such boat) — and examiners like you to say so.

3.3 Negative queries: the complete family

Every "no", "never", "without", "except" question has the same skeleton:

$$\text{Answer} = \pi_{\text{key}}(R(\text{Universe})) - \pi_{\text{key}}(R(\text{Witness}))$$

The only two decisions are (a) what the **universe** is — all sailors, all boats, all patients — and (b) what counts as a **witness** — one join chain that exhibits the forbidden property. Get those two right and the query writes itself.

Q3.21 **Find the names of sailors who have not reserved any boat on 2019-10-01.**
Answer.

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(\sigma_{\text{day} = \text{2019-10-01}}(Reserves))))$$

Inner difference: (sid). Result: (sname).
Why. The witness is "reserved something on that day". The universe is all sailors, so sailors with no reservations at all are correctly included.

Trap / common mistake. If the universe were $\pi_{\text{sid}}(Reserves)$ — sailors who have **some** reservation — the answer would exclude sailors with no reservations, which is the opposite of what "have not reserved any boat on that day" means for them. Choosing the universe is the whole question.

Q3.22 **Find the names of sailors who have reserved every boat that sailor 22 has reserved. (Do it without division, then with division.)**
Answer. **Without division:**

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(\pi_{\text{sid}}(Sailors) \times \pi_{\text{bid}}(\sigma_{\text{sid}\,=22}(Reserves)) - Reserves)))$$

With division:

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{sid}\,=22}(Reserves))))$$

Both inner expressions return (sid). Result: (sname).
Why. The counter-example method: all sailors, minus those for whom **some** boat of sailor 22 is missing from their reservations. The middle expression $\pi_{\text{sid}}(Sailors) \times \pi_{\text{bid}}(\ldots) - Reserves$ is exactly the set of missing (sailor, boat) pairs; projecting its `sid` gives the disqualified sailors.

Trap / common mistake. The dividend must be $\pi_{\text{sid, bid}}(Reserves)$ — **not** `Reserves`, whose schema includes `day`. With `day` present, the divisor's schema is not a subset in the right way and the division silently asks a different (and meaningless) question.

Q3.23 **Find the names of sailors who have reserved all boats, and explain the two equivalent formulations.**
Answer.

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)))$$

$$= \pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(\pi_{\text{sid}}(Sailors) \times \pi_{\text{bid}}(Boats) - \pi_{\text{sid, bid}}(Reserves))))$$

The division returns (sid); the subtraction form also returns (sid).
Why. Division is the direct statement of $\forall$; the second form is its definition, and writing both is the safest exam strategy because it shows the marker you know **why** they are equal. The middle term $\pi_{\text{sid}}(Sailors) \times \pi_{\text{bid}}(Boats) - \pi_{\text{sid, bid}}(Reserves)$ is the set of "missing reservations" that a perfect sailor would have.

Trap / common mistake. Check the sailor with **no** reservations in both forms. **Division:** he is absent from $\pi_{\text{sid}}(Reserves)$, so he is not a candidate and is not returned — correct, since "reserved all boats" is false for him. **Subtraction form:** his every (sailor, boat) pair is missing, so he lands in the disqualified set and is removed — also correct. The two forms agree, which is exactly why they are worth writing side by side.

Q3.24 Find the sailors who have reserved at least one boat but not all boats.
Answer.

$$\pi_{\text{sid}}(Reserves) - \left(\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)\right)$$

Both operands (sid). Result: (sid).
Why. "At least one" gives the universe; "all" is subtracted. This is the **complement within the reservers**, not within all sailors — the question says "at least one", so a sailor with no reservations must not appear.

Trap / common mistake. $\pi_{\text{sid}}(Sailors) - \left(\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)\right)$ answers "did not reserve all boats", which includes sailors with no reservations. Both readings appear in papers; the wording "at least one but not all" forces the first.

Q3.25 Find the boats that have been reserved by at least one sailor but not by all sailors.
Answer.

$$\pi_{\text{bid}}(Reserves) - \left(\pi_{\text{bid, sid}}(Reserves) \div \pi_{\text{sid}}(Sailors)\right)$$

Both operands (bid).
Why. The roles are swapped: now the "answer attribute" is `bid` and the quantified attribute is `sid`. Note the dividend has schema (bid, sid); attribute order in the projection is irrelevant to division, only the **set** matters.

Trap / common mistake. Using $\pi_{\text{bid, day}}(Reserves)$ as the dividend would quantify over days, not sailors, and answer a completely different question.

Q3.26 Find the names of sailors who have reserved only boats that are red.
Answer.

$$\pi_{\text{sname}}\Big(Sailors \bowtie \Big(\pi_{\text{sid}}(Reserves) - \pi_{\text{sid}}\Big(Reserves \bowtie \sigma_{\text{color} \neq \textbf{red}}(Boats)\Big)\Big)\Big)$$

The subtracted term is (sid); the difference is (sid).
Why. "Only red" = "has reservations" minus "has a non-red reservation". This is pattern 5 and it is the same expression as Q3.4.

Trap / common mistake. "$\neq$ red" versus "$=$ not-red": in the presence of NULL colours these differ. In this book colours are non-null, so $\sigma_{\text{color} \neq \textbf{red}}$ is the complement of $\sigma_{\text{color} = \textbf{red}}$ and both are fine. State the assumption if the paper mentions NULLs.

Part 4 — Division, "all", "every", "only" and universal conditions

What this part fixes. Division is the only classical operator that expresses $\forall$. This part gives the definition, the derivation from $\pi, \times, -$, the three schema discipline rules, the empty-divisor edge case, and twenty-two questions that separate "all", "only", "every", "at least one" and "no". Schema: Sailors–Boats–Reserves unless a question says otherwise.

4.1 The definition, once and properly

Let $R(\text{R})(X, Y)$ and $R(\text{S})(Y)$, with $\text{attr}(R(\text{S})) = Y \subseteq \text{attr}(R(\text{R}))$. Then

$$R(\text{R}) \div R(\text{S}) = \{t \in \pi_X(R(\text{R})) : \forall s \in R(\text{S})\ [(t, s) \in R(\text{R})]\}$$

Three consequences that decide most exam answers:

1. The **result schema** is $X = \text{attr}(R(\text{R})) - \text{attr}(R(\text{S}))$. Nothing else survives.
2. The divisor must be **exactly** the set you are quantifying over. "All boats" is $\pi_{\text{bid}}(Boats)$; "all boats that are red" is $\pi_{\text{bid}}(\sigma_{\text{color} = \textbf{red}}(Boats))$; "all boats reserved by sailor 22" is $\pi_{\text{bid}}(\sigma_{\text{sid} =22}(Reserves))$.
3. The dividend must be **projected** to $X \cup Y$ before dividing. If it carries an extra attribute — the classic mistake of dividing `Reserves` itself — the extra attribute joins X and the answer has the wrong schema, even if the tuples look plausible.

4.2 Deriving division from the five primitive operators

Claim. $R(\text{R}) \div R(\text{S}) = \pi_X(R(\text{R})) - \pi_X((\pi_X(R(\text{R})) \times R(\text{S})) - R(\text{R}))$, where $X = \text{attr}(R(\text{R})) - \text{attr}(R(\text{S}))$.

Proof.
1. $\pi_X(R(\text{R}))$ is the set of **candidate** answers — every t that is attached to at least one s in $R(\text{R})$.
2. $\pi_X(R(\text{R})) \times R(\text{S})$ is the set of **all pairs** (t, s) that a perfect answer would have to contain.
3. Subtracting $R(\text{R})$ leaves precisely the pairs (t, s) that are **missing** from $R(\text{R})$.
4. Projecting to X gives the set of candidates t that are missing at least one s, i.e. the candidates that fail the $\forall$.
5. Subtracting that from the candidates leaves exactly the t for which no s is missing. That is the definition of division. □

Q4.1 Find the sids of sailors who have reserved all boats.
Answer.

$$\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)$$

Dividend: (sid, bid). Divisor: (bid). Result: (sid).
Why. The dividend has been projected to $(\text{answer}, \text{quantified}) = (\text{sid}, \text{bid})$. The divisor is the full set of boats. Any extra attribute in the dividend — `day` in particular — would end up in the result schema and change the meaning.

Trap / common mistake. $Reserves \div \pi_{\text{bid}}(Boats)$ is **not** equivalent: `Reserves` has schema (sid, bid, day), so $\text{attr}(R) - \text{attr}(S) = (\text{sid}, \text{day})$ and the "answer" is a set of (sailor, day) pairs — a sailor who reserved **all** boats on one single day. Silently different, and a classic lost mark.

Q4.2 Find the names of sailors who have reserved all boats.
Answer.

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)))$$

Division result (sid); join with `Sailors` on `sid`; project (sname).
Why. Compute the key set first — division on keys only — then look up descriptive attributes. This "divide, then join" habit prevents almost every name-related error.

Trap / common mistake. Do not attempt to divide a relation that contains `sname`. If you need names, join **after** the division.

Q4.3 Find the sids of sailors who have reserved all red boats.
Answer.

$$\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{color} = \textbf{red}}(Boats))$$

Result (sid).
Why. The quantified set is "red boats", so the colour filter belongs inside the **divisor**.

Trap / common mistake. $\pi_{\text{sid, bid}}(\sigma_{\text{color} = \textbf{red}}(Reserves \bowtie Boats)) \div \pi_{\text{bid}}(Boats)$ asks a completely different question — it removes non-red reservations from the dividend and then requires all boats, which can never succeed unless every boat is red. Filters belong on the divisor when they define the quantified set, and on the dividend only when they restrict the witness set.

Q4.4 Find the sids of sailors who have reserved every boat that sailor 22 has reserved.

Answer.

$\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{sid}=22}(Reserves))$

Result (sid).

Why. The quantified set is **whatever 22 reserved**, which is computed as a projection of a selection. Sailor 22 appears in his own answer (he did reserve all of them) — decide from the wording whether to exclude him with a further difference.

Trap / common mistake. If 22 has reserved nothing, the divisor is empty and division returns **every** sid in $\pi_{\text{sid}}(Reserves)$. That is the vacuous-truth edge case; see Q4.19.

Q4.5 Find the sids of sailors who have reserved every boat on 2019-10-01, and explain why the schema makes this question ambiguous.

Answer. Two readings, and the schema decides which is expressible: **(a) "for each boat, there is a reservation by this sailor on that date"** —

$$\pi_{\text{sid, bid}}\big(\sigma_{\text{day}=\texttt{2019-10-01}}(Reserves)\big) \div \pi_{\text{bid}}(Boats)$$

; **(b) "this sailor's reservations on that date cover all boats"** — exactly the same expression, because the dividend is already restricted to that date. The **diagonal** reading — "there is one day on which he reserved all boats" — is $\pi_{\text{sid, day}}(Reserves) - \pi_{\text{sid, day}}\big((\pi_{\text{sid, day}}(Reserves) \times \pi_{\text{bid}}(Boats)) - \pi_{\text{sid, bid, day}}(Reserves)\big)$, i.e. a division whose result keeps `day`.

(a)/(b) return (sid); the diagonal reading returns (sid, day).

Why. The word "on" attaches either to the reservation (readings a and b) or to the quantification (the diagonal reading). Classical relational algebra can express all three, but **which schema the answer must have** tells you which one is wanted. Say which reading you are using.

Trap / common mistake. Reading (b) is **not** the same as (a) when the schema allowed a sailor to reserve the same boat twice on the same day — it does not, by the key — but they **are** different from the diagonal reading, which is the one most students accidentally write by dividing $(\pi_{\text{sid, day, bid}})$.

Q4.6 Find the names of sailors who have reserved all boats of colour red or all boats of colour green.

Answer.

$$\pi_{\text{sname}}(Sailors \bowtie (R(\text{AllRed}) \cup R(\text{AllGreen})))$$

where $R(\text{AllRed}) = \pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{color}=\textbf{red}}(Boats))$ and $R(\text{AllGreen}) = \pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}\big(\sigma_{\text{color}=\textbf{green}}(Boats)\big)$.

Both division results are (sid), so the union is legal. Result (sname).

Why. "All red or all green" is a disjunction of two **complete** universal conditions. Each universal condition is one division; the disjunction is union. Compare with Q4.7, where the disjunction is **inside** the quantifier and the answer is different.

Trap / common mistake. Do **not** distribute union into division: $R \div (S_1 \cup S_2) \neq (R \div S_1) \cup (R \div S_2)$ in general. Dividing by a larger set is **harder**; the right-hand side is easier for each term. This non-distributivity is a standard 2-mark theory question.

Q4.7 Find the sids of sailors who have reserved all boats that are red or green.

Answer.

$\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}\big(\sigma_{\text{color}=\textbf{red} \vee \text{color}=\textbf{green}}(Boats)\big)$

Result (sid).

Why. Here the disjunction defines the **quantified set**: every boat whose colour is red or green. It is one division, not a union of two divisions. The distinction from Q4.6 is the whole point of the

pair: compare “all red or all green” (union of divisions) with “all red-or-green” (division by a union).

Trap / common mistake. Neither expression implies the other. A sailor who reserved all red boats and no green boats satisfies Q4.6 but not Q4.7.

Q4.8 Find the sids of sailors who have reserved all boats, and whose rating is at least 8.
Answer.

$$\left(\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)\right) \cap \pi_{\text{sid}}\left(\sigma_{\text{rating} \geq 8}(Sailors)\right)$$

Both operands (sid). Result (sid).
Why. Two independent conditions on a sailor, each computed on the key, then intersected. Mixing a universal condition (division) with a tuple-local one (selection) is always done this way — never by trying to push the selection into the division.

Trap / common mistake. Restricting the dividend instead — $\pi_{\text{sid, bid}}\left(\sigma_{\text{rating} \geq 8}(Sailors) \bowtie Reserves\right) \div \ldots$ — also works here because the join preserves the pair, but it is fragile: it silently changes the meaning if the restriction is on an attribute that does not survive the join. The intersection form is safer and easier to mark.

Q4.9 Find the sids of sailors who have reserved at least one boat but not all boats.
Answer.

$$\pi_{\text{sid}}(Reserves) - \left(\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)\right)$$

Both (sid).
Why. “At least one” is the universe, “all” is subtracted. The order matters: subtracting in the other direction is meaningless but still well-typed, so no error will warn you.

Trap / common mistake. See Q3.24 for the difference between this and its complement over **all** sailors.

Q4.10 Find the boats that have been reserved by all sailors with rating 10.
Answer.

$$\pi_{\text{bid, sid}}(Reserves) \div \pi_{\text{sid}}\left(\sigma_{\text{rating} = 10}(Sailors)\right)$$

Result (bid). Note the dividend’s attribute order is irrelevant to division; what matters is that its schema is exactly (answer attributes ∪ divisor attributes).
Why. The direction of the question is reversed: the answer attribute is `bid` and the quantified attribute is `sid`. Division does not care which attribute is which.

Trap / common mistake. Using $\pi_{\text{bid, sid}}(\sigma_{\text{bid} = 101}(Reserves))$ as the dividend restricts the boats considered, which is a different (usually easier) question.

Q4.11 Find the names of sailors who have reserved every boat that has been reserved by every sailor with rating 10.
Answer. Let $R(\text{B10}) = \pi_{\text{bid, sid}}(Reserves) \div \pi_{\text{sid}}\left(\sigma_{\text{rating} = 10}(Sailors)\right)$ — the boats reserved by all rating-10 sailors. Then

$$\pi_{\text{sname}}\left(Sailors \bowtie \left(\pi_{\text{sid, bid}}(Reserves) \div R(\text{B10})\right)\right)$$

.
$R(\text{B10})$: (bid). The second division returns (sid). Result (sname).
Why. Nested universals compose naturally when the inner one produces exactly the set the outer one quantifies over. Read inside-out: first find the set of boats, then divide by it.

If $R(\text{B10})$ is empty — no boat was reserved by every rating-10 sailor, which happens whenever a rating-10 sailor reserved nothing — the outer division returns **every** sailor. Vacuous truth again; state it.

Trap / common mistake. Do not try to flatten the two divisions into one. $(R \div S) \div T$ is not a meaningful expression here because the schemas do not line up.

Q4.12 Find the sids of sailors who have reserved every boat that is not red.
Answer.

$$\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}\left(Boats - \pi_{\text{bid, color}}(Boats) \underset{\text{color} \neq \text{red}}{\bowtie} \ldots\right)$$

— written correctly:

$$\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats - \sigma_{\text{color} = \textbf{red}}(Boats))$$

The divisor is $\pi_{\text{bid}}(Boats - \sigma_{\text{color} = \textbf{red}}(Boats))$, a set of bids. Result (sid).
Why. The quantified set is defined by a **difference** on `Boats`, and only then projected to `bid`. The order — filter first, project second — is what keeps the domain of the divisor right.

Trap / common mistake. $\pi_{\text{bid}}(Boats) - \pi_{\text{bid}}(\sigma_{\text{color} = \textbf{red}}(Boats))$ is equivalent and also correct. But $\pi_{\text{bid}}(Boats) - \pi_{\text{bid, color}}(Boats)$ would be a type error.

4.3 The schema discipline of division: three worked diagnostics

The three checks below catch essentially every division error in an exam script. Do them in this order before writing anything.

Check	How to verify
1. Divisor schema	List the attributes of the divisor. Say aloud what set they describe ("the set of all bids", "the set of bids of red boats"). If you cannot name the set in one phrase, the divisor is wrong.
2. Dividend schema	It must be exactly $X \cup Y$ — answer attributes plus divisor attributes, nothing else. If `day` or `sname` is in there and you did not intend it, the division answers a different question.
3. Result schema	It must be exactly X. Say aloud what a tuple of the result means ("a sailor such that..."). If you cannot finish the sentence with "for all", you did not need division.

Q4.13 A student writes $Reserves \div \pi_{\text{bid}}(Boats)$ for "sailors who reserved all boats". Diagnose the error, state the schema the student's expression actually has, and repair it.
Answer. Correct: $\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)$.
Student's result: $\text{attr}(Reserves) - \text{attr}(\pi_{\text{bid}}(Boats)) = (\text{sid}, \text{bid}, \text{day}) - (\text{bid}) = (\text{sid}, \text{day})$. The repaired expression has result (sid).
Why. The student's expression is **well-typed and non-empty**: it returns (sailor, day) pairs such that, on that single day, the sailor reserved every boat. That is a real query — just not the one asked. This is the most instructive error in the whole subject, because nothing about the syntax is wrong.

Trap / common mistake. The habit that prevents it: always write the dividend as $\pi_{\text{sid, bid}}$ — i.e. **project to exactly the two attributes you are relating** — before dividing.

Q4.14 What does $\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid, day}}(Reserves)$ mean? Is it the same as Q4.1?
Answer. It is **not** Q4.1. The divisor is a set of (bid, day) pairs; the dividend's schema is (sid, bid), which does not contain `day`, so the divisor's schema is not a subset of the dividend's — the expression is **ill-formed**.
Undefined: division requires $\text{attr}(R(\text{S})) \subseteq \text{attr}(R(\text{R}))$.
Why. Division quantifies over **tuples of the divisor**, and those tuples must be able to combine with candidate tuples to form tuples of the dividend. If the divisor mentions an attribute the dividend does not have, the concatenation is impossible.

To ask "sailors who reserved every (boat, day) pair that occurs", the dividend must include `day`: $\pi_{\text{sid, bid, day}}(Reserves) \div \pi_{\text{bid, day}}(Reserves)$, whose result is (sid) and whose meaning is exactly that. This is a genuinely different query from Q4.1 and much harder to satisfy.

Q4.15 Prove that $R \div (S_1 \cup S_2) \subseteq (R \div S_1) \cap (R \div S_2)$, and give an example showing the inclusion is strict.

Careful: the natural expectation is the opposite direction. Work it out. If t has all pairs with every element of $S_1 \cup S_2$, it certainly has all pairs with every element of S_1 — so $t \in R \div S_1$, and likewise for S_2. Hence $R \div (S_1 \cup S_2) \subseteq (R \div S_1) \cap (R \div S_2)$.

Answer. The inclusion is in that direction and is generally strict. Example: $R(\mathrm{R})$ relates sailor 1 to boats 101 and 102, and sailor 2 to boat 101 only. $R(\mathrm{S_1}) = \{101\}$, $R(\mathrm{S_2}) = \{102\}$. Then $R(\mathrm{R}) \div R(\mathrm{S_1}) = \{1, 2\}$ and $R(\mathrm{R}) \div R(\mathrm{S_2}) = \{1\}$, whose intersection is $\{1\}$; and $R(\mathrm{R}) \div (R(\mathrm{S_1}) \cup R(\mathrm{S_2})) = \{1\}$. Equality here.
Why. To see strictness, add a third sailor who has all of $R(\mathrm{S_1})$ and all of $R(\mathrm{S_2})$ but not their union — impossible, because the union is just the two elements. Strictness arises only when the elements of $R(\mathrm{S_1})$ and $R(\mathrm{S_2})$ are **pairs** whose combinations are missing: let $R(\mathrm{S_1}) = \{(101, \mathrm{a})\}$, $R(\mathrm{S_2}) = \{(102, \mathrm{b})\}$, and let t have exactly those two pairs but not $(101, \mathrm{b})$ or $(102, \mathrm{a})$. Then t is in both quotients but not in $R(\mathrm{R}) \div (R(\mathrm{S_1}) \cup R(\mathrm{S_2}))$.

Trap / common mistake. Many textbooks state $R \div (S_1 \cup S_2) = (R \div S_1) \cap (R \div S_2)$ in exercises that **assume** one-attribute divisors, where the identity does hold. Always check the arity of the divisor before quoting an identity.

Q4.16 Express "sailors who have reserved all boats" without using division, then expand your expression into the five primitive operators only.
Answer.

$$\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}\big(\pi_{\text{sid}}(Sailors) \times \pi_{\text{bid}}(Boats) - \pi_{\text{sid, bid}}(Reserves)\big)$$

This uses only π, $\times$, $-$ — already primitive.
Inner: (sid, bid) on both sides of the difference; after π: (sid).
Why. The middle term is the **missing-pairs** relation. A sailor is disqualified exactly when he is attached to at least one missing pair. This is the definition of division written out, and it is the answer an examiner wants when division is disallowed.

Trap / common mistake. Note the universe: $\pi_{\text{sid}}(Sailors)$, **not** $\pi_{\text{sid}}(Reserves)$. With the latter, sailors with no reservations would be excluded from consideration and would **wrongly** be counted as having reserved all boats. Subtle, and worth a mark.

Q4.17 Use the definition to evaluate $R(\mathrm{R}) \div R(\mathrm{S})$ for the small relations $R(\mathrm{R}) = \{(1, \mathrm{a}), (1, \mathrm{b}), (2, \mathrm{a})\}$ and $R(\mathrm{S}) = \{(\mathrm{a}), (\mathrm{b})\}$.
Answer. $R(\mathrm{R}) \div R(\mathrm{S}) = \{(1)\}$.
Result schema is the single attribute of $R(\mathrm{R})$ that is not in $R(\mathrm{S})$, i.e. the id.
Why. Candidate answers are $\{1, 2\}$. Pairs required for a perfect answer: $(1, \mathrm{a}), (1, \mathrm{b}), (2, \mathrm{a}), (2, \mathrm{b})$. Present pairs: all but $(2, \mathrm{b})$. Missing pairs: $\{(2, \mathrm{b})\}$, whose projection is $\{2\}$. Candidates minus $\{2\}$ is $\{1\}$.

Trap / common mistake. If $R(\mathrm{S})$ were empty, the answer would be all candidates $\{1, 2\}$; if $R(\mathrm{R})$ were empty, then candidates $\{\}$ is empty and so is the answer. Both are consistent with the definition, and both look wrong to a first-time reader.

Q4.18 State the correct dividend and divisor schema for each of the following, without writing the full expression. (a) sailors who reserved all boats; (b) boats reserved by all

sailors; (c) sailors who reserved all red boats; (d) sailors who reserved all boats reserved by 22; (e) pairs (sailor, day) such that the sailor reserved all boats on that day.
Answer.

(a)	dividend (sid, bid)	divisor (bid)	result (sid)
(b)	dividend (bid, sid)	divisor (sid)	result (bid)
(c)	dividend (sid, bid)	divisor (bid) of red boats only	result (sid)
(d)	dividend (sid, bid)	divisor (bid) of 22's boats	result (sid)
(e)	dividend (sid, day, bid)	divisor (bid)	result (sid, day)

Why. In (e) the answer schema itself contains `day`, so `day` must be part of the dividend's non-quantified attributes. This is exactly the expression a student writes by accident in Q4.13 — here it is the **correct** answer to a different question.

Trap / common mistake. In (b) the dividend is $\pi_{\text{bid, sid}}$, not $\pi_{\text{sid, bid}}$: the **set** of attributes is what matters, but writing them in answer-first order prevents mistakes when you read the expression back.

4.4 Edge cases: empty divisor, empty dividend, ties

Q4.19 Explain the vacuous-truth case: what does $\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}\big(\sigma_{\text{color} = \text{yellow}}(Boats)\big)$ return when there are no yellow boats?
Answer. It returns **every** sailor who appears in $\pi_{\text{sid, bid}}(Reserves)$ — i.e. every sailor with at least one reservation.
Result (sid).
Why. $\forall s \in \varnothing : P(s)$ is true for every t; there is no counter-example to find. So every candidate passes.

Trap / common mistake. The candidate set is π_{sid} of the **dividend**, so sailors with no reservations are still excluded. Students often claim "all sailors"; the precise answer is "all sailors who have made at least one reservation". Also state whether the question intends the vacuous reading — the mathematically correct one — or the colloquial "must have reserved something yellow".

Q4.20 Give an expression for "sailors who have reserved at least one yellow boat and all red boats" and explain why the two conditions do not interact.
Answer.

$$\big(\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{color} = \text{red}}(Boats))\big) \cap \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} = \text{yellow}}(Boats)\big)$$

Both operands (sid).
Why. The universal condition and the existential condition are evaluated independently on `sid` sets and combined by intersection. The intersections' operands need not be disjoint or related — set operators do not care about the provenance of tuples.

Trap / common mistake. The existential part must use a join with σ; the **division** alone would not require any yellow boat to exist — see Q4.19.

Q4.21 Can classical relational algebra express "sailors who have reserved all boats reserved by more than three sailors"? Explain.

Not in the classical algebra: "more than three sailors" is a **cardinality** condition on a group, which needs counting. With the extended operator γ one writes $\gamma_{\text{bid; count(sid)} \leftarrow n}\big(\pi_{\text{bid, sid}}(Reserves)\big)$ and then selects $\sigma_{n>3}$, but γ is not part of the classical algebra of $\pi, \sigma, \rho, \cup, -, \times$.

Answer. The query **as stated** is not expressible in classical relational algebra. It becomes expressible if the threshold is a **fixed numeral** and the schema makes the count first-order definable — for "reserved by more than three sailors" it is not, because "more than three" compares a numeral to the size of a set of arbitrary tuples.

Q4.22 **Using only division and set operations, decide for each pair whether it is expressible: (a) sailors who reserved all boats; (b) sailors who reserved exactly 3 boats; (c) sailors who reserved all boats of the same colour; (d) sailors who reserved all boats that are red or blue but not both.**

Answer. **(a)** Expressible — Q4.1. **(b)** Not expressible ("exactly 3" is a cardinality condition when 3 is a numeral only in the fixed-arity sense; see §10.6 — for a **fixed numeral** such as 3, a finite disjunction of "at least k" conditions is expressible, but it requires the identity of the boats and becomes impractically large; the clean answer is that it needs counting). **(c)** Expressible with one division per colour value, but not uniformly in the schema: with a fixed, finite set of colours $c_1, c_2, \ldots$ it is a finite union of divisions. **(d)** Expressible — the inner condition is a plain set expression inside the divisor.

Why. The dividing line is **fixed versus computed** cardinality (§1.11). Anything that can be written as a finite Boolean combination of existentials and universals over the stored relations is expressible; anything that must **count an unbounded number of tuples** is not.

Trap / common mistake. (b) catches students who answer "expressible" because 3 is a constant. For a fixed numeral k the statement "there exist k distinct boats reserved by s" is expressible, and for $k = 3$ that is a finite expression (§10.6 shows the pattern). What is **not** expressible is "exactly k" where k is read from an attribute, or "more than 3" as a comparative count. Examiners usually want "not expressible" for (b) with the reason "requires counting"; if you add the fixed-numeral caveat you will get the benefit of the doubt.

Part 5 — Maximum, minimum, ranking, ties and group-wise comparisons

What this part fixes. Extremes are the classic "cannot use MAX()" question. The trick is always the same: **a value is the maximum exactly when no larger value exists.** Writing that as a set difference over a self-product gives every maximum query in the syllabus — global, per-group, after a selection, with ties, and for the k-th value. The part ends with the three things that are genuinely **not** expressible.

5.1 The master pattern

Maximum of an attribute. Let $R(\mathrm{R})$ have attribute a.

$$\mathrm{MaxVal}(a, R(\mathrm{R})) = \pi_a(R(\mathrm{R})) - \pi_{\mathrm{R1.a}}(\sigma_{\mathrm{R1.a} < \mathrm{R2.a}}(\rho_{\mathrm{R1}}(R(\mathrm{R})) \times \rho_{\mathrm{R2}}(R(\mathrm{R}))))$$

Read the second term as "the values that are smaller than some other value" and subtract them. What remains are the values that are smaller than **nothing** — the maxima.

Minimum is the same with $>$ in place of $<$.

Records with the maximum value are then $R(\mathrm{R}) \bowtie \mathrm{MaxVal}(a, R(\mathrm{R}))$ — a natural join on a, which preserves **all** ties.

The one thing to be careful about: the subtraction must be on **values**, so the projection to a is essential. If you subtract whole tuples you get "records that are beaten", which is a different (and usually wrong) answer, because a record can be beaten on a yet still be tied for the maximum **value** after projection. See Q5.4.

5.2 Global maximum and minimum

Q5.1 **Find the highest rating among sailors, without using MAX.**

Answer.

$$\pi_{\mathrm{rating}}(Sailors) - \pi_{\mathrm{S1.rating}}\big(\sigma_{\mathrm{S1.rating} < \mathrm{S2.rating}}(\rho_{\mathrm{S1}}(Sailors) \times \rho_{\mathrm{S2}}(Sailors))\big)$$

$\pi_{\text{rating}}(Sailors)$: (rating). The σ result has schema (S1.sid, S1.sname, S1.rating, S1.age, S2.sid, S2.sname, S2.rating, S2.age); projecting A(S1.rating) gives (rating), which is union compatible with the left operand.

Why. All values minus values that are less than some other value. The survivors are precisely the maximum values. If several sailors share rating 10, the result is still a single tuple (10) — the question asked for the **value**, not the sailors.

Trap / common mistake. If the relation is empty, both operands are empty and the result is empty — there is no maximum of an empty set, and returning the empty relation is the correct answer. Say so rather than inventing a value.

Q5.2 Find the minimum age among sailors.

Answer.

$$\pi_{\text{age}}(Sailors) - \pi_{\text{S1.age}}\big(\sigma_{\text{S1.age} > \text{S2.age}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$$

(age).

Why. Identical to Q5.1 with the inequality reversed.

Trap / common mistake. Do not "simplify" by negating the comparison inside a single product: $\sigma_{\neg(\text{S1.age} > \text{S2.age})}$ is $\leq$, which includes the equality case and therefore removes **everything**, giving the empty relation for the minimum. Negating the **whole** expression is not the same as negating the **condition inside** it — the set difference does the negation, not the comparison operator.

Q5.3 Find the sailor(s) with the highest rating: return the complete sailor tuples, and preserve ties.

Answer.

$$Sailors \bowtie \big(\pi_{\text{rating}}(Sailors) - \pi_{\text{S1.rating}}\big(\sigma_{\text{S1.rating} < \text{S2.rating}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)\big)$$

The natural join is on `rating`; result schema (sid, sname, rating, age) — the same as `Sailors`.

Why. Compute the maximum **value** first (Q5.1), then join back to get the records carrying that value. Every sailor whose rating equals the maximum survives the join, so ties are preserved automatically. This is why the natural join is the right tool here: it **is** the "records with this value" operator.

Trap / common mistake. A common wrong answer is $Sailors - \pi_{\text{sid, sname, rating, age}}\big(\sigma_{\text{S1.rating} < \text{S2.rating}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$. It **also** works when ratings are distinct, and it is the pattern to use when the "maximum" is defined over the **whole tuple** rather than one attribute (see Q5.4). But it silently differs from the join form if two sailors tie on `rating` and one is subtracted for a different reason. Prefer "value, then join".

Q5.4 Find the sailor with the maximum rating using only set difference on whole tuples. Compare with Q5.3 and explain when the two answers differ.

Answer.

$$Sailors - \pi_{\text{sid, sname, rating, age}}\big(\sigma_{\text{S1.rating} < \text{S2.rating}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$$

The projection must list **all four** attributes of the R(S1) side so that the difference is union compatible with `Sailors`. This is the "preserve full schema when subtracting records" rule.

Why. A whole tuple is removed when it is **dominated** by another tuple. If rating is the only criterion, a tied tuple is not dominated (it is not $<$ any other), so ties survive. The two answers coincide whenever the only comparison is on the maximum attribute.

They differ as soon as the criterion is composite, e.g. "the sailor with the highest rating, and among those the oldest". The whole-tuple method can express that with a compound σ; the "value then join" method needs two nested extremes.

Trap / common mistake. Forgetting one attribute in the projection produces a **well-typed** expression if the remaining list happens to match — for instance projecting only (sid, rating) would be union compatible with

$\pi_{\text{sid, rating}}(Sailors)$, not with `Sailors`, so it is caught. But projecting the wrong **four** attributes in a different order is not caught, and changes the meaning. Always write the projection in the schema order of the target relation.

5.3 Second, third and k-th highest

Q5.5 **Find the second-highest distinct rating.**
Answer. Let $R(\text{M}) = \pi_{\text{rating}}(Sailors) - \pi_{\text{S1.rating}}\big(\sigma_{\text{S1.rating} < \text{S2.rating}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$. Then

$$\text{econd}) = \pi_{\text{rating}}\big(\sigma_{\text{rating} < \text{M.rating}}(Sailors)\big) - \pi_{\text{S1.rating}}\big(\sigma_{\text{S1.rating} < \text{S2.rating}}\big(\rho_{\text{S1}}\big(\sigma_{\text{rating} < \text{M.rating}}(Sailors)\big) \times \rho_{\text{S2}}\big(\sigma_{\text{rating} < \text{M.rating}}(Sailo$$

Both $R(\text{M})$ and $R(\text{Second})$ have schema (rating).
Why. **Remove the maximum, then take the maximum of what is left.** The inner selection $\sigma_{\text{rating} < \text{M.rating}}(Sailors)$ restricts to strictly smaller ratings — note that it must be applied to `Sailors`, joined with $R(\text{M})$, or equivalently written as $\sigma_{\text{rating} <10}(Sailors)$ once the maximum is known. Then Q5.1 is applied to that restricted relation.

An equivalent and shorter formulation: let $R(\text{Below}) = \pi_{\text{rating}}(Sailors) - R(\text{M})$. Then the second-highest is $R(\text{Below}) - \pi_{\text{S1.rating}}\big(\sigma_{\text{S1.rating} < \text{S2.rating}}(\rho_{\text{S1}}(R(\text{Below})) \times \rho_{\text{S2}}(R(\text{Below})))\big)$. Since $R(\text{Below})$ is a **relation of values**, it can appear directly as a product operand — this is legal because ρ renames a relation, and a relation of ratings is a relation.

Trap / common mistake. "Second-highest rating" vs "rating of the second-highest sailor": with ties, these are different. If three sailors share rating 10 and one has 9, the second-highest **distinct** rating is 9, but "the sailor in second place by rating" is ambiguous. State which you compute — the distinct-value reading is the standard one.

Q5.6 **Find the sailor(s) with the second-highest rating, preserving ties.**
Answer. Let $R(\text{S2})$ be the expression of Q5.5 (the second-highest distinct rating). Then

$$Sailors \bowtie R(\text{S2})$$

.
(sid, sname, rating, age).
Why. The same "value, then join" step as Q5.3. All sailors tied at the second-highest value are returned.

Trap / common mistake. If there is only one distinct rating, the second-highest is **undefined**. The expression returns the empty relation, which is the honest answer; a student who forces a value by using $\leq$ instead of $<$ returns the maximum itself. Do not.

Q5.7 **Find the third-highest distinct rating, and then the k-th highest.**
Answer. **Third:** remove the top two values, then take the maximum of what remains.

$$R(\text{Top2}) = R(\text{M}) \cup R(\text{S2}) \quad R(\text{Third}) = \text{Max}\big(\pi_{\text{rating}}(Sailors) - R(\text{Top2})\big)$$

***k*-th:** define $R(\text{Top})_0 = \varnothing$ and $R(\text{Top})_{i+1} = R(\text{Top})_i \cup \text{Max}\big(\pi_{\text{rating}}(Sailors) - R(\text{Top})_i\big)$. Then the k-th highest is $\text{Max}\big(\pi_{\text{rating}}(Sailors) - R(\text{Top})_{k-1}\big)$.
Every intermediate is a relation of schema (rating), so the unions are legal.
Why. "Remove the largest, repeat" is the definition of the k-th largest in a set. Because each step is expressible in classical RA, every **fixed** k is expressible: the expression is a finite composition of the same five operators.

Trap / common mistake. This is expressible for a **fixed numeral** k written into the query. It is **not** expressible for "k-th highest where k is a column value" — that needs arithmetic and counting. See Q5.27 and §10.6. Examiners often ask the fixed case and expect you to note the distinction.

5.4 Global versus group-wise extremes

Q5.8 **Using $R(\text{Emp})(\text{eid}, \text{ename}, \text{sal}, \text{deptno})$ and $R(\text{Dept})(\text{deptno}, \text{dname}, \text{loc})$, find the highest salary in each department, returning the employees.**

Answer.

$$Emp - \pi_{\text{E1.eid, E1.ename, E1.sal, E1.deptno}}\left(\sigma_{\text{E1.deptno} = \text{E2.deptno} \wedge \text{E1.sal} < \text{E2.sal}}(\rho_{\text{E1}}(Emp) \times \rho_{\text{E2}}(Emp))\right)$$

The product has eight attributes; the projection keeps the four of the $R(\text{E1})$ side in the **schema order of** `Emp`, so the difference is union compatible. Result schema (eid, ename, sal, deptno).

Why. An employee is **not** a group maximum iff there exists another employee **in the same department** with a strictly larger salary. The equality on `deptno` is what makes the comparison group-local; without it you would get the global maximum. Ties survive because the comparison is strict.

Trap / common mistake. Three separate errors are common. (i) Omitting `E1.deptno = E2.deptno` — global instead of per-department. (ii) Omitting the projection or shortening it — the subtraction then either fails union compatibility or, worse, produces a different relation and no error. (iii) Using $\leq$ — every employee has someone with an equal salary (himself), so **every** row is removed and the answer is empty.

Q5.9 **Find the department(s) whose average salary is the highest — without γ and without averages. Is it expressible?**

Not expressible in classical relational algebra. "Highest average salary" is a **computed group statistic**. Classical relational algebra has no aggregation, so averages cannot even be formed, let alone compared. The honest answer is: not expressible.

Answer. Not expressible in classical relational algebra. It becomes expressible with the extended grouping operator:

$$\gamma_{\text{deptno; avg(sal)} \leftarrow \text{avgsal}}(Emp)$$

, then the maximum-of-values pattern on `avgsal`, then a join back.

Why. Every expressible query in classical RA is a first-order formula over the stored tuples; "average" requires summing an unbounded number of values, which first-order logic over a finite structure cannot do without aggregation.

Trap / common mistake. Students often try to compare salaries pairwise and conclude "the department whose every member earns more than every member of any other department". That is a **different** query (dominance), and it is expressible — but it is not "highest average". If you write it, label it as a different interpretation, not as an answer to this question.

Q5.10 **Find the highest salary in the department with the most employees. Discuss.**

Not expressible in classical relational algebra. "The department with the most employees" is a cardinality comparison between groups — again an aggregate.

Answer. Not expressible in classical relational algebra; needs $\gamma_{\text{deptno; count(eid)} \leftarrow n}(Emp)$ followed by a maximum and a join.

Why. Contrast with Q5.8, which **is** expressible. The difference is that Q5.8 compares salaries **within** a group using a stored value, while this question compares the **sizes** of groups.

Trap / common mistake. A common wrong instinct is "find the department that is not smaller than any other", i.e. $\neg\exists$ a department with more employees. That condition **is** expressible — it is a finite formula — but it computes **maximal** departments, and if two departments tie for the largest size both are returned, which is right, while a department that is smaller than another is excluded. Actually this works! See the note below.

Careful: "the department with the most employees" **is** expressible when read as "no other department has strictly more employees", because "more employees" can be phrased as the existence of an injective mapping — which is

not first-order over the relation alone. In practice, examiners treat it as **not expressible** and expect the γ answer. If you want to argue expressibility, you must exhibit a first-order formula; the natural attempt ("there is no department d' with $|d'| > |d|$") quantifies over **sets**, not tuples, and is therefore not a formula of the classical algebra. State the limitation explicitly.

Q5.11 **Find the cheapest medicine in the `Medicine(TradeName, UnitPrice, GenericFlag)` relation, and the most expensive one.**
Answer.

$= \pi_{\text{TradeName, UnitPrice, GenericFlag}}(Medicine) - \pi_{\text{M1.TradeName, M1.UnitPrice, M1.GenericFlag}}(\sigma_{\text{M1.UnitPrice} > \text{M2.UnitPrice}}(\rho_{\text{M1}}(Medicine) \times \rho_{\text{M2}}($

Most expensive $= Medicine - \pi_{\text{M1.TradeName, M1.UnitPrice, M1.GenericFlag}}(\sigma_{\text{M1.UnitPrice} < \text{M2.UnitPrice}}(\rho_{\text{M1}}(Medicine) \times \rho_{\text{M2}}(Medicine))$
Both results have the schema of `Medicine`, (TradeName, UnitPrice, GenericFlag).
Why. Here the extreme is required on **records**, not on a single value, because the question asks for "the cheapest medicine" — the whole row. The whole-tuple pattern (Q5.4) is therefore the natural one; the projection must list all three attributes.

Trap / common mistake. If the question asked only for the **price**, project (UnitPrice) on both sides and compare only that attribute. Mixing the two modes — projecting three attributes on the left and one on the right — breaks union compatibility.

5.5 Extremes after a selection, and extremes over dates

Q5.12 **Find the highest rating among sailors who are older than 30.**
Answer.

$$\pi_{\text{rating}}(\sigma_{\text{age} > 30}(Sailors)) - \pi_{\text{S1.rating}}(\sigma_{\text{S1.rating} < \text{S2.rating}}(\rho_{\text{S1}}(\sigma_{\text{age} > 30}(Sailors)) \times \rho_{\text{S2}}(\sigma_{\text{age} > 30}(Sailors))))$$
(rating).
Why. **Apply the selection first, to both copies.** The extreme is taken over the restricted relation, not over all sailors and then filtered — those two give different answers whenever the global maximum is held by a young sailor.

Trap / common mistake. The shortcut $\sigma_{\text{age} > 30}(\text{HighestRated})$ is wrong: it answers "is the globally highest-rated sailor also over 30?", which is usually empty. Selection and extremum do not commute. This is the "maximum after applying a selection condition" trap, and it is worth a full question in most papers.

Q5.13 **Find the most recent prescription date for each doctor, using `Prescription(Id, Date, Doctor_SSN, Patient_SSN)`.**
Answer.

$Presc - \pi_{\text{P1.Id, P1.Date, P1.Doctor_SSN, P1.Patient_SSN}}(\sigma_{\text{P1.Doctor_SSN} = \text{P2.Doctor_SSN} \wedge \text{P1.Date} < \text{P2.Date}}(\rho_{\text{P1}}(Presc) \times \rho_{\text{P2}}(Presc)))$
Result schema equals that of `Prescription`.
Why. Exactly the Q5.8 pattern with `Date` in place of `sal` and `Doctor_SSN` in place of `deptno`. ISO date strings compare chronologically as strings, so $<$ is the right operator.

Trap / common mistake. If dates were stored as DD/MM/YYYY strings, $<$ would compare day-of-month first and give nonsense. Either convert to ISO, or compare the three components as a lexicographic tuple — and say which you are doing.

5.6 Ranking: four different questions that share one word

Reading	What is being ordered	Expression
Ranking distinct values	the set of values, not the rows	$\text{Max}(\pi_a R)$, $\text{Max}(\pi_a R - \text{Max}(\pi_a R))$, ... (Q5.5, Q5.7)

Reading	What is being ordered	Expression
Ranking individual records	rows, broken by a tie-breaker on the key	whole-tuple pattern with a compound σ (Q5.14)
Ranking with ties	rows, all ties of a value reported together	value-then-join (Q5.3, Q5.6) — ties are preserved by construction
Ranking within each group	rows, ordered inside their own partition	group-wise pattern: equality on the group attribute in the σ (Q5.8, Q5.15)

Classical relational algebra has **no order** and no row number, so "rank" can only mean one of the four things above. A question that says "give the rank of each sailor" with no tie-breaking rule is under-specified; say so, then answer the closest expressible reading.

Q5.14 Find the sailors who are the oldest in their rating group, breaking ties by sid (i.e. the single oldest sailor per rating, and if two share the maximum age, the one with the smaller sid). Is this deterministically expressible?

Answer. The **first** part — "oldest in each rating group", ties preserved — is:

$$Sailors - \pi_{\text{S1.sid, S1.sname, S1.rating, S1.age}}\big(\sigma_{\text{S1.rating} = \text{S2.rating} \wedge \text{S1.age} < \text{S2.age}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$$

The **tie-break by sid** is also expressible, because it is a stored attribute:

$$Sailors - \pi_{\text{S1.sid, S1.sname, S1.rating, S1.age}}\big(\sigma_{\text{S1.rating} = \text{S2.rating} \wedge (\text{S1.age} < \text{S2.age} \vee (\text{S1.age} = \text{S2.age} \wedge \text{S1.sid} > \text{S2.sid}))}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$$

Both results have schema (sid, sname, rating, age).

Why. The compound σ says: "there exists another sailor in my rating group who is either older than me, or the same age with a smaller sid". Anyone satisfying that is not the winner. This is a **total order** on the group because `sid` is unique, so exactly one winner per group survives.

Trap / common mistake. A tie-break on something that is **not** unique (say `sname`) does not give a total order, and the expression then returns several sailors per group. That is not an error in the expression — it is a guarantee the data does not support. Say so.

Q5.15 For each boat, find the sailor(s) who reserved it most recently.

Answer.

$$\pi_{\text{R1.sid, R1.bid}}(Reserves)\ldots$$

— written fully, with the group-wise pattern on `Reserves`:

$$Reserves - \pi_{\text{R1.sid, R1.bid, R1.day}}\big(\sigma_{\text{R1.bid} = \text{R2.bid} \wedge \text{R1.day} < \text{R2.day}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves))\big)$$

Result schema equals `Reserves`: (sid, bid, day). If the question asks for sailor names, join with `Sailors` afterwards.

Why. The group attribute is `bid`; the compared attribute is `day`. A reservation is not the latest for its boat exactly when another reservation of the **same** boat has a later date.

Trap / common mistake. The natural join $\rho_{\text{R1}}(Reserves) \bowtie \rho_{\text{R2}}(Reserves)$ equates all three attributes and returns only identical rows, so the σ on `day` is unsatisfiable. Use $\times$.

Q5.16 Find the patient(s) with the highest number of prescriptions. Explain the difficulty.

Not expressible in classical relational algebra. "Highest number of prescriptions" compares **counts**, so it is not expressible in classical relational algebra.

Answer. Not expressible. With the extended operator:

$$\gamma_{\text{Patient_SSN};\ \text{count(Id)} \leftarrow n}(Presc)$$

, then the maximum-of-values pattern on n, then a join back to `Patient` for names.
Why. This is the single most important "not expressible" example for the hospital schema. It looks exactly like Q5.14 but is fundamentally different: Q5.14 compares **stored values**, this compares **cardinalities of sets**. Classical RA can say "there exist three prescriptions"; it cannot say "there exist more prescriptions than for any other patient".

5.7 Fixed-cardinality conditions without aggregates

Q5.17 **Find the sailors who have made exactly one reservation.**
Answer.

$$R(\text{Once}) = \pi_{\text{sid}}(Reserves) - \pi_{\text{R1.sid}}\Big(\sigma_{\text{R1.sid} = \text{R2.sid} \wedge \text{R1.bid} \neq \text{R2.bid}}\big(\rho_{\text{R1}}(\pi_{\text{sid, bid}}(Reserves)) \times \rho_{\text{R2}}(\pi_{\text{sid, bid}}(Reserves))\big)\Big)$$

$$\text{Answer} = \pi_{\text{sname}}(Sailors \bowtie R(\text{Once}))$$

$R(\text{Once})$: (sid). Result: (sname).
Why. "Exactly one" is "at least one" minus "at least two". "At least two" is expressed by existence of two **distinct** boats, which is a self-join with $\neq$ — no counting involved. Projecting to (sid, bid) first is essential so that "two distinct boats" is what is measured.

Trap / common mistake. If the question means "exactly one **reservation record**" (one day, one boat), then measure distinct (sid, bid, day) triples instead. On this schema a sailor cannot reserve the same boat twice on the same day, so the two readings differ only when the same boat is reserved on two days. State which one you mean.

Q5.18 **Find the sailors who have made exactly two distinct reservations. First show why a counting attempt cannot be written, then give the classical expression.**
Answer. The counting attempt "$|\sigma_{\text{sid} = s}(Reserves)| = 2$" cannot be written at all: cardinality of a set is not a term of classical relational algebra. The classical expression uses **three distinct witnesses**:

$$R(\text{Two+}) = \pi_{\text{R1.sid}}\Big(\sigma_{\text{R1.sid} = \text{R2.sid} \wedge \text{R1.day} \neq \text{R2.day}}\big(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves)\big)\Big)$$

$$\text{e+}) = \pi_{\text{R1.sid}}\Big(\sigma_{\text{R1.sid} = \text{R2.sid} \wedge \text{R2.sid} = \text{R3.sid} \wedge \text{R1.day} \neq \text{R2.day} \wedge \text{R2.day} \neq \text{R3.day} \wedge \text{R1.day} \neq \text{R3.day}}\big(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves) \times \rho_{\text{R3}}(\text{Re}$$

$$\text{Answer} = \pi_{\text{sname}}(Sailors \bowtie (R(\text{Two+}) - R(\text{Three+})))$$

Each intermediate has schema (sid); the final result is (sname).
Why. "Exactly two" is "at least two" minus "at least three". Each of those is a pure existence statement over **distinct** tuple variables, so both are first-order and therefore expressible. This is the general recipe of §10.6: a fixed numeral can always be eliminated in favour of that many distinct witnesses.

Trap / common mistake. Reusing two variables cannot express "three distinct days": a formula with only $R(\text{R1})$ and $R(\text{R2})$ can say "there are two different days", never "there are three". The number of variables must be at least the numeral. Also, the pairwise $\neq$ conjuncts are all needed — with three witnesses, excluding only $A(\text{R1.day}) \neq A(\text{R2.day})$ and $A(\text{R2.day}) \neq A(\text{R3.day})$ still allows $A(\text{R1})$ and $A(\text{R3})$ to share a day.

Q5.19 **Which of these is expressible in classical relational algebra? (a) sailors with at least two reservations; (b) sailors with more reservations than sailor 22; (c) sailors with exactly 5 reservations; (d) sailors with more reservations than any other sailor.**
Answer.

(a)	**Yes**	self-join with $\neq$ on the day or boat (Q5.17)
(b)	**No**	compares two cardinalities; needs γ

(c)	**Yes in principle, No in practice**	a fixed numeral 5 needs five distinct tuple variables; expressible but unwieldy, and most syllabi mark it "not expressible"
(d)	**No**	"more than any other" is a maximum of a count; needs γ

Why. The dividing line is whether the condition mentions a **number that must be computed**. (a) and (c) use fixed numerals; (b) and (d) require computing a count for every sailor and comparing.

Trap / common mistake. Examiners who set (c) usually want "not expressible" with the explanation that counting is required. If you can write the five-variable expression, do, and then explain that the syllabus convention is to call it inexpressible because the query's **structure** must change with the numeral. That answer gets full marks either way.

Q5.20 **Find the sailors who are older than every sailor with rating 10, using the counter-example method — and explain why the naive "maximum" approach is wrong.**

Answer. Let $R(\mathrm{T10}) = \sigma_{\text{rating }=10}(Sailors)$ and $R(\mathrm{Beaten}) = \pi_{\mathrm{S.sid}}\big(\sigma_{\mathrm{S.age}\ \le\ \mathrm{T.age}}(\rho_{\mathrm{S}}(Sailors) \times \rho_{\mathrm{T}}(R(\mathrm{T10})))\big)$. Then

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Sailors) - R(\mathrm{Beaten})))$$

.

$R(\mathrm{Beaten})$: (sid). Result: (sname).

Why. "$>$ every" is "$\neg$ ($\le$ some)": a sailor fails iff he is at most as old as **some** rating-10 sailor. Computing the maximum age among rating-10 sailors and comparing would give the same answer **only if** $R(\mathrm{T10})$ is non-empty and the maximum exists; it also requires an extra extreme query. The counter-example method is shorter and correct even for the empty case.

When $R(\mathrm{T10})$ is empty, $R(\mathrm{Beaten})$ is empty and every sailor is returned — vacuously true, and the mathematically correct reading of "older than every member of the empty set".

Trap / common mistake. Using $<$ instead of $\le$ in $R(\mathrm{Beaten})$ excludes sailors who are **equal** to a rating-10 sailor, but "older than every" requires strict superiority, so equality must disqualify. This off-by-one in the comparison is the most common error in the whole part.

Q5.21 **Find the sailors with the maximum rating among those who are older than 25. Then find the maximum rating among those who are not older than 25, and explain why the two answers can be equal even when the two groups are disjoint.**

Answer.

$$\text{MaxYoung} = \pi_{\text{rating}}\big(\sigma_{\text{age }\le 25}(Sailors)\big) - \pi_{\mathrm{S1.rating}}\big(\sigma_{\mathrm{S1.rating\ <\ S2.rating}}\big(\rho_{\mathrm{S1}}\big(\sigma_{\text{age }\le 25}(Sailors)\big) \times \rho_{\mathrm{S2}}\big(\sigma_{\text{age }\le 25}(Sailors)\big)\big)\big)$$

$$\text{MaxOld} = \pi_{\text{rating}}\big(\sigma_{\text{age }> 25}(Sailors)\big) - \pi_{\mathrm{S1.rating}}\big(\sigma_{\mathrm{S1.rating\ <\ S2.rating}}\big(\rho_{\mathrm{S1}}\big(\sigma_{\text{age }> 25}(Sailors)\big) \times \rho_{\mathrm{S2}}\big(\sigma_{\text{age }> 25}(Sailors)\big)\big)\big)$$

Both (rating).

Why. Both groups can contain sailors with rating 10 — rating values are not partitioned by age. The two maxima are **values**, and values can coincide across disjoint groups.

Trap / common mistake. Concluding "the groups are disjoint, so the maxima must differ" is a non-sequitur. Also note that if one group is empty its maximum is the empty relation, not zero.

Q5.22 **Give the general recipe for "k-th highest" when ties are present, and show why "k-th value" and "k-th record" diverge.**

1. ***k*-th distinct value.** Repeatedly remove the maximum, $k-1$ times, then take the maximum (Q5.7). Ties do not affect the result, because values are a set.

2. ***k*-th record.** Impose a total order using a unique tie-breaker (a key), then repeatedly remove the maximum tuple. If no total order exists in the data, the question is under-specified.

Answer. With ratings 10, 10, 9: the 2nd **distinct** value is 9; the 2nd **record** by rating is also 9 if we order 10, 10, 9 and count rows; the 2nd record under "ties share a rank" does not exist as a single row. Three different answers to one English sentence.
Why. Set semantics makes values canonical and records non-canonical: a set has no second element unless you supply an order, and relations have no order. Hence only the distinct-value reading is stable across all of classical RA.

Trap / common mistake. Never write "$R(\mathrm{R})[2]$" or "the second tuple". Indexing does not exist in relational algebra; any answer that depends on position is wrong by definition.

Q5.23 For each department, find the employee(s) with the lowest salary, but only for departments that have at least two employees.
Answer. Let $R(\mathrm{Min1}) = Emp - \pi_{\mathrm{E1.eid},\ \mathrm{E1.ename},\ \mathrm{E1.sal},\ \mathrm{E1.deptno}}\big(\sigma_{\mathrm{E1.deptno}\,=\,\mathrm{E2.deptno}\,\wedge\,\mathrm{E1.sal}\,>\,\mathrm{E2.sal}}(\rho_{\mathrm{E1}}(Emp) \times \rho_{\mathrm{E2}}(Emp))\big)$ and $R(\mathrm{TwoPlus}) = \pi_{\mathrm{E1.deptno}}\Big(\sigma_{\mathrm{E1.deptno}\,=\,\mathrm{E2.deptno}\,\wedge\,\mathrm{E1.eid}\,\neq\,\mathrm{E2.eid}}(\rho_{\mathrm{E1}}(Emp) \times \rho_{\mathrm{E2}}(Emp))\Big)$. Then

$$R(\mathrm{Min1}) \bowtie R(\mathrm{TwoPlus})$$

.
$R(\mathrm{Min1})$: schema of `Emp`. $R(\mathrm{TwoPlus})$: (deptno). The natural join is on `deptno`. Result: schema of `Emp`.
Why. Two independent conditions — a group-wise extreme, and a group-level existence condition. Each is computed separately and combined by a join on the group attribute. The existential condition uses the **key** `eid` for the inequality so that two distinct rows are counted, not two distinct salaries.

Trap / common mistake. Using $\sigma_{\mathrm{E1.sal}\,\neq\,\mathrm{E2.sal}}$ for "at least two employees" counts two employees with different salaries — a department with two employees on the same salary would be wrongly excluded. Cardinality statements must compare **keys**.

Q5.24 Find the sailors who have made exactly one reservation, and explain why this is expressible while "exactly one more than sailor 22" is not.
Answer. Let $R(\mathrm{Res}) = \pi_{\mathrm{sid},\ \mathrm{bid},\ \mathrm{day}}(Reserves)$ be the set of reservation records.

$$R(\mathrm{Two+}) = \pi_{\mathrm{R1.sid}}\Big(\sigma_{\mathrm{R1.sid}\,=\,\mathrm{R2.sid}\,\wedge\,\mathrm{R1.day}\,\neq\,\mathrm{R2.day}}(\rho_{\mathrm{R1}}(R(\mathrm{Res})) \times \rho_{\mathrm{R2}}(R(\mathrm{Res})))\Big)$$

$$R(\mathrm{Once}) = \pi_{\mathrm{sid}}(R(\mathrm{Res})) - R(\mathrm{Two+})$$

and the names come from

$$\pi_{\mathrm{sname}}(Sailors \bowtie R(\mathrm{Once}))$$

.
$R(\mathrm{Res})$: (sid, bid, day). $R(\mathrm{Two+})$ and $R(\mathrm{Once})$: (sid). Result: (sname).
Why. "Exactly one" is "at least one" minus "at least two", and "at least two" is two **distinct** witnesses — a fixed numeral, so the formula needs only two tuple variables. Nothing is counted; the two variables simply witness that a second record exists.

Trap / common mistake. "Exactly one more than sailor 22" fails at the first step: the bound is a **computed count**, so no fixed number of tuple variables can express it. The distinction is not the word "exactly" but whether the bound is a literal in the query text. See §10.6.

Q5.25 **Find the sailor(s) with the highest rating after removing all sailors younger than 25 from consideration — and explain why "removing records first" changes the answer in a way that "removing values first" does not.**

Answer. Let $R(\text{Elig}) = \sigma_{\text{age} \geq 25}(Sailors)$. Then

$$R(\text{Elig}) \bowtie \left(\pi_{\text{rating}}(R(\text{Elig})) - \pi_{\text{E1.rating}}\left(\sigma_{\text{E1.rating} < \text{E2.rating}}(\rho_{\text{E1}}(R(\text{Elig})) \times \rho_{\text{E2}}(R(\text{Elig})))\right)\right)$$

.

$R(\text{Elig})$: schema of `Sailors`. Result: schema of `Sailors`.

Why. The extreme must be taken over the **reduced relation** (Q5.12). Removing records can remove the holder of the global maximum, in which case the eligible maximum is a different value; removing **values** — that is, projecting first — would leave the maximum intact but lose the sailors who hold it.

This is the cleanest illustration of "maximum after removing certain records". On the sample instance of Part 6, Zorba (16) is excluded, so the highest eligible rating is 10 held by Rusty — the exclusion changed nothing; but if Rusty were 24, the answer would change to 9.

Trap / common mistake. Applying the age filter **after** the extremum — $\sigma_{\text{age} \geq 25}(\text{HighestRated})$ — answers "is the globally highest-rated sailor at least 25?", which is a different and usually empty question.

Q5.26 **State precisely which of the following are expressible in classical relational algebra, and give the expression or the reason: (a) sailors with exactly three reservations; (b) sailors with exactly n reservations where n is an attribute of `Sailors`; (c) the third-highest distinct rating; (d) the average rating of each rating group.**

Answer. **(a) Expressible.** Three distinct tuple variables:

$$R(\text{Three+}) = \pi_{\text{R1.sid}}(\sigma_{\theta}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves) \times \rho_{\text{R3}}(Reserves)))$$

with θ the conjunction of the three sid-equalities and the three pairwise day-inequalities; then "exactly three" $= R(\text{Three+}) - R(\text{Four+})$, where $R(\text{Four+})$ uses four variables. **(b) Not expressible.** The number of tuple variables would have to depend on a stored value. **(c) Expressible** — three nested "remove the maximum" steps (Q5.7). **(d) Not expressible** — averaging is an aggregate.

(a) and (c): (sid) and (rating) respectively.

Why. The dividing line is again **fixed versus computed** cardinality (§1.11, §10.6), and the presence or absence of an arithmetic aggregate for (d).

Trap / common mistake. Part (a) is often marked "not expressible" by students who confuse it with part (b). Write out why three variables suffice and the marker will see the distinction.

Q5.27 **Give the complete list of the questions in this part that are not expressible, with the operator each one needs.**

Question	What must be computed	Missing operator
Q5.9	average salary per department	γ with `avg`
Q5.10	number of employees per department	γ with `count`
Q5.16	number of prescriptions per patient	γ with `count`
Q5.19(b), (d)	comparison of two counts	γ with `count`
Q5.24	"exactly one more than sailor 22"	γ with `count`
Q5.26(b)	"exactly n reservations" where n is an attribute	γ with `count`
Q5.26(d)	average rating per group	γ with `avg`

Why. Every entry has the same shape: a **number** has to be produced from a set of tuples and then compared. Classical relational algebra has no operator that turns a relation into a number; γ is that operator, and it belongs to the extended algebra.

Trap / common mistake. Note what is **not** in the list: Q5.8 (highest salary per department), Q5.13 (latest prescription per doctor), Q5.14 (oldest per rating group), Q5.22 (**k**-th highest for fixed k). Those compare **stored values**, which needs no aggregation — only a self-join and a set difference. Confusing the two families is the single most costly mistake in this part.

Part 6 — Sailors and Boats — complete solved question bank

What this part fixes. The complete classical bank on the **Database System Concepts** running example, in the order in which it is normally taught: basic queries, set operations, self-joins, division, then the advanced queries (extremes, "only", "all", "exactly one"). Every question states the schema it uses. Questions marked **(SKS pattern)** follow the standard shape of Chapter 4 exercises; they are not verbatim reproductions.

6.0 Sample instance

All answers below can be checked against this small instance. It is used only to **illustrate**; the expressions are correct for every instance.

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

sid	bid	day
22	101	2019-10-01
22	102	2019-10-01
22	103	2019-10-01
22	104	2019-10-01
31	102	2019-10-03
31	103	2019-10-05
64	101	2019-10-07
64	102	2019-10-07
74	103	2019-10-09

Note the deliberate features: two sailors share the name "Horatio" (sids 64 and 74), two share rating 8 (31 and 32), and two share rating 10 (58 and 71). These are the ties that make the difference between "on keys" and "on names" visible.

6.1 Basic queries

Q6.1 Find the names of all sailors.
Answer.

$\pi_{\text{sname}}(Sailors)$
(sname); one row per distinct name, so "Horatio" appears once even though two sailors bear it.
Why. Pure projection: no condition, and the requested attribute is `sname`.

Trap / common mistake. If the question wants one row per sailor, project the key as well ($\pi_{\text{sid, sname}}(Sailors)$) and say why.

Q6.2 Find the sailors whose rating is 8.
Answer.

$\sigma_{\text{rating}=8}(Sailors)$
Schema of `Sailors`.
Why. Tuple-local condition on a stored attribute.

Trap / common mistake. "Rating 8" is one condition, not a join. Do not add `Reserves` unless a reservation is mentioned.

Q6.3 Find the names of sailors older than 35.

Answer.

$\pi_{\text{sname}}(\sigma_{\text{age}\,>35}(Sailors))$

(sname).

Why. Select then project — the order matters because `age` must survive until the selection.

Trap / common mistake. $\sigma_{\text{age}\,>35}(\pi_{\text{sname}}(Sailors))$ is a type error. With the sample instance the answer is empty (no one is strictly older than 35), which is the right answer, not a mistake.

Q6.4 Find the names of sailors who have reserved a red boat. (classic)

Answer.

$\pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats))$

Join chain schema is the union of the three; the result is (sname).

Why. Existential: there is a reservation linking the sailor to a red boat. With the sample instance, Dustin (22), Lubber (31), Horatio (64) are returned — Horatio appears once although the name matches two sids.

Trap / common mistake. Projecting after the joins gives names; projecting before would lose the link.

Q6.5 Find the sids of sailors who have reserved boat 103.

Answer.

$\pi_{\text{sid}}(\sigma_{\text{bid}\,=103}(Reserves))$

(sid).

Why. `Reserves` already contains both attributes needed; no join is required.

Trap / common mistake. Do not join with `Sailors` — that only adds attributes which are then projected away.

Q6.6 Find the bid and bname of boats reserved on 2019-10-03.

Answer.

$\pi_{\text{bid, bname}}(\sigma_{\text{day} = \textbf{2019-10-03}}(Reserves) \bowtie Boats)$

(bid, bname).

Why. The date is in `Reserves`; the name is in `Boats`; the shared attribute is `bid`.

Trap / common mistake. Using $\pi_{\text{bid, bname}}(Boats)$ restricted by the date is impossible — `Boats` has no `day`.

Q6.7 Find the names of sailors who have reserved a boat of colour blue or green.

Answer.

$\pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \textbf{blue} \vee \text{color} = \textbf{green}}(Boats))$

(sname).

Why. One selection with a disjunction over a single boat tuple.

Trap / common mistake. The **union** form is also correct, but it must be a union of two **complete queries**: $\pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \textbf{blue}}(Boats)) \cup \pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \textbf{green}}(Boats))$, never a union of two `Boats` relations followed by a projection — the projection would have no `sname` to project.

Q6.8 Find the names of sailors above rating 7 who have reserved a red boat.

Answer.

$\pi_{\text{sname}}(\sigma_{\text{rating}\,>7}(Sailors) \bowtie Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats))$

(sname).

Why. Two independent restrictions, both pushed to their leaves.

Trap / common mistake. $\sigma_{\text{rating}\,>7}(\pi_{\text{sname}}(Sailors) \bowtie Reserves \bowtie \sigma_{\text{color}\,=\,\text{red}}(Boats))$ is **legal** here because `rating` survives the join — but only by accident of schema. Pushing selections down is always safe; pulling them up is safe only when the attribute survives.

6.2 Union, intersection, difference

Q6.9 Find the names of sailors who have reserved a red boat or a green boat.
Answer.

$$\pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color}\,=\,\text{red}\,\vee\,\text{color}\,=\,\text{green}}(Boats))$$

(sname), a set.
Why. "Or" over the same boat tuple. With the sample instance: Dustin, Lubber, Horatio.

Trap / common mistake. Compare Q6.10: adding "both" changes the expression from $\vee$ to an intersection of two $\exists$ conditions. The English word "or" is inclusive here; exclusive-or is a different (and rarely asked) question.

Q6.10 Find the names of sailors who have reserved both a red boat and a green boat. (classic)
Answer.

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color}\,=\,\text{red}}(Boats)) \cap \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color}\,=\,\text{green}}(Boats))))$$

Both intersected operands are (sid).
Why. Two independent existences, intersected on the key. With the sample instance: Dustin (reserved 102 red and 103 green), Lubber (102 red, 103 green), Horatio 64 (102 red only) — no; Horatio 64 is excluded, Horatio 74 excluded. So {Dustin, Lubber}.

Trap / common mistake. The whole-tuple intersection $\sigma_{\text{color}\,=\,\text{red}}(Boats) \cap \sigma_{\text{color}\,=\,\text{green}}(Boats)$ is **empty**: a boat cannot be two colours. Intersections of existential conditions must be taken **after** projecting to the key that the existence is about.

Q6.11 Find the names of sailors who have reserved a red boat but never a green boat.
Answer.

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color}\,=\,\text{red}}(Boats)) - \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color}\,=\,\text{green}}(Boats))))$$

(sid) on both sides of the difference.
Why. Difference of key sets, then look up names. Same shape as Q3.3.

Trap / common mistake. $\pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color}\,=\,\text{red}}(Boats)) - \pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color}\,=\,\text{green}}(Boats))$ is the (incorrect) name-set version — two different sailors called Horatio would cancel each other out.

Q6.12 Find the names of sailors who have never reserved a red boat.
Answer.

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color}\,=\,\text{red}}(Boats))))$$

(sid).
Why. Universe = all sailors; witness = red reservation.

Trap / common mistake. Omitting the $\pi_{\text{sid}}(Sailors)$ universe and using $\pi_{\text{sid}}(Reserves)$ instead silently excludes sailors with no reservations — who certainly never reserved a red boat.

Q6.13 Find the boats that have never been reserved. (classic)
Answer.

$$\pi_{\text{bid}}(Boats) - \pi_{\text{bid}}(Reserves)$$

(bid).
Why. Pattern 2 with the universe `Boats`.

Trap / common mistake. To print names, join afterwards: $\pi_{\text{bname}}(Boats \bowtie (\pi_{\text{bid}}(Boats) - \pi_{\text{bid}}(Reserves)))$.

Q6.14 Find the sids and names of sailors who have never made a reservation.
Answer.

$\pi_{\text{sid, sname}}(Sailors \bowtie (\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(Reserves)))$
(sid, sname, rating, age) if the whole tuple is returned; project (sid, sname) to match the question.
Why. The difference is on `sid`; the join restores the descriptive attributes.

Trap / common mistake. $\pi_{\text{sid, sname}}(Sailors) - \pi_{\text{sid}}(Reserves)$ is not union compatible: two attributes against one. This exact error is worth a mark in almost every paper.

6.3 Self-joins and comparisons

Q6.15 Find the pairs of sailors with the same rating. Report each unordered pair once.
Answer.

$\pi_{\text{S1.sid, S2.sid}}\big(\sigma_{\text{S1.rating} = \text{S2.rating} \wedge \text{S1.sid} < \text{S2.sid}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$
(S1.sid, S2.sid).
Why. "Same rating" is an equality between two tuple variables; $<$ on the keys makes each pair appear once and removes self-pairs.

Trap / common mistake. With the sample instance, ratings 8 (31,32), 7 (22,64) and 10 (58,71) each produce one pair; rating 9 and rating 1 produce none. If you forget the $<$ condition you get 6 ordered pairs plus 8 self-pairs.

Q6.16 Find the sids of sailors who share their rating with at least one other sailor with a different sid.
Answer.

$\pi_{\text{S1.sid}}\big(\sigma_{\text{S1.rating} = \text{S2.rating} \wedge \text{S1.sid} \neq \text{S2.sid}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$
(sid).
Why. Same join, but the answer is a set of sailors, and π removes the duplicate witnesses.

Trap / common mistake. Using $<$ here returns only the smaller sid of each pair (31, not 32). Because the question asks **which sailors**, both members are wanted, so $\neq$ is correct and $<$ is not.

Q6.17 Find the pairs (older, younger) of sailors with the same rating.
Answer.

$\pi_{\text{S1.sid, S2.sid}}\big(\sigma_{\text{S1.rating} = \text{S2.rating} \wedge \text{S1.age} > \text{S2.age}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\big)$
(S1.sid, S2.sid), ordered by the roles older/younger.
Why. Here the inequality is on the **compared** attribute, not on the key, because the pair is ordered by age. This is the difference from Q6.15.

Trap / common mistake. If two sailors have the same rating **and** the same age, the pair appears in neither direction — that is correct for an "older/younger" question but would be wrong for a "same age" question, which needs $=$.

Q6.18 Find the pairs of sailors who have reserved the same boat.
Answer.

$\pi_{\text{R1.sid, R2.sid}}(\sigma_{\text{R1.bid} = \text{R2.bid} \wedge \text{R1.sid} < \text{R2.sid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves)))$
(R1.sid, R2.sid).
Why. The self-join is on `Reserves`, because that is where "reserved" lives.

Trap / common mistake. Do not put the self-join on `Sailors`: sailors do not carry boats.

Q6.19 Find the pairs of sailors who reserved the same boat on the same day.
Answer.

$\pi_{\text{R1.sid, R2.sid}}\big(\sigma_{\text{R1.bid} = \text{R2.bid} \wedge \text{R1.day} = \text{R2.day} \wedge \text{R1.sid} < \text{R2.sid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves))\big)$
(R1.sid, R2.sid).
Why. Two equalities on the relationship, one inequality on the key.

Trap / common mistake. The natural join $\rho_{\text{R1}}(Reserves) \bowtie \rho_{\text{R2}}(Reserves)$ equates all three attributes, so it can only find a sailor paired with himself — use $\times$.

Q6.20 Find the names of sailors who have reserved a boat with a higher bid than every boat reserved by sailor 22.
Answer. Let $R(\text{B22}) = \pi_{\text{bid}}(\sigma_{\text{sid} = 22}(Reserves))$ and $R(\text{Low}) = \pi_{\text{R1.sid}}(\sigma_{\text{R1.bid} \leq \text{R2.bid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(R(\text{B22}))))$. Then

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Reserves) - R(\text{Low})))$$

.
$R(\text{B22})$: (bid). $R(\text{Low})$: (sid).
Why. The counter-example method: a sailor fails if **some** of his boats is not higher than **some** boat of 22. With the sample: 22 has 101–104, so a higher boat does not exist and the answer is empty.

Trap / common mistake. The naive answer "bid > max(bid of 22)" would require an extra extreme query and breaks on the empty case.

Q6.21 Find the names of sailors who have made at least two reservations — and contrast with "at least two different boats".
Answer. At least two reservations:

$$\pi_{\text{sname}}\Big(Sailors \bowtie \pi_{\text{R1.sid}}\big(\sigma_{\text{R1.sid} = \text{R2.sid} \wedge \text{R1.bid} \neq \text{R2.bid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves))\big)\Big)$$

At least two different boats: same expression with the projection $\pi_{\text{sid, bid}}$ applied to `Reserves` before the self-join.
Both results: (sid) before the join with `Sailors`.
Why. Because the key of `Reserves` is (sid, bid, day), two reservations of the **same** boat on different days are two reservations but one boat. The sample instance has no such case, so invent one to test: adding (64, 101, 2019-10-08) makes 64 a two-reservation sailor without changing the boat count.

Trap / common mistake. Questions that say "at least two" usually mean two **distinct entities**; read the noun after "two". This pair of queries is the standard test of that reading.

Q6.22 Find the sailors who share the same name but have different sids.
Answer.

$\pi_{\text{S1.sid, S2.sid}}(\sigma_{\text{S1.sname} = \text{S2.sname} \wedge \text{S1.sid} < \text{S2.sid}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors)))$
(S1.sid, S2.sid).
Why. The comparison attribute is `sname`, which is **not** a key — that is the whole point.

Trap / common mistake. A common wrong answer is $\pi_{\text{sname}}(Sailors) - \pi_{\text{sname}}(\sigma_{\text{sname} = \text{X}}(Sailors))$ style set arithmetic on names; it cannot detect duplicates because projection has already removed them. Duplicate detection needs a self-join.

6.4 Division and universal quantification

Q6.23 Find the sids of sailors who have reserved every boat. (classic)
Answer.

$\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)$
(sid).
Why. Dividend restricted to the two attributes of interest; divisor is the full boat set.

Trap / common mistake. Dividing `Reserves` directly returns (sid, day) pairs — see Q4.13.

Q6.24 Find the names of sailors who have reserved every red boat.
Answer.

$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{color} = \textbf{red}}(Boats))))$
(sname).
Why. The quantified set is **red** boats, so the colour condition is inside the divisor.

Trap / common mistake. With the sample instance there are two red boats (102, 104); no sailor reserved both, so the answer is empty. Empty answers are common in division queries — do not "fix" them.

Q6.25 Find the names of sailors who have reserved every boat that sailor 22 has reserved.
Answer.

$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{sid} = 22}(Reserves))))$
(sname).
Why. The divisor is whatever 22 reserved; Dustin himself is included, as he reserved all of them.

Trap / common mistake. If 22 had no reservations the divisor would be empty and **every** sailor returned — vacuous truth. Say so.

Q6.26 Find the sids of sailors who have reserved all boats of colour red.
Answer.

$\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{color} = \textbf{red}}(Boats))$
(sid).
Why. This is Q4.3, restated as part of the complete bank.

Trap / common mistake. "All red boats" and "all boats, which happen to be red" are different queries when non-red boats exist. Only the first is asked here.

Q6.27 Explain how division solves "for all" questions, and derive the equivalent expression that uses only π, $\times$ and $-$.
Answer. See §4.1–4.2 for the full derivation. The identity is $R(\mathrm{R}) \div R(\mathrm{S}) = \pi_X(R(\mathrm{R})) - \pi_X((\pi_X(R(\mathrm{R})) \times R(\mathrm{S})) - R(\mathrm{R}))$ with $X = \mathrm{attr}(R(\mathrm{R})) - \mathrm{attr}(R(\mathrm{S}))$.
Why. The English sentence "for every boat, the sailor reserved it" has the form $\forall b \in R(\mathrm{Boats}), \exists r \in R(\mathrm{Reserves})$ with matching attributes. Division is exactly the operator whose definition contains that $\forall$. The primitive form of the same statement is "there is no boat that is missing", and "missing boats" is a set difference against a product.

Trap / common mistake. The double quantifier "$\forall b \exists r$" is **not** the same as "$\exists r \forall b$". The second is division when r ranges over a single sailor; the first allows a different reservation per boat, which is precisely what division computes. Mixing them up produces the "same day" variant of Q4.5.

6.5 Advanced queries

Q6.28 Find the sailor(s) with the highest rating.
Answer.

$Sailors \bowtie (\pi_{\text{rating}}(Sailors) - \pi_{\text{S1.rating}}(\sigma_{\text{S1.rating} < \text{S2.rating}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))))$
Schema of `Sailors`.
Why. Value, then join (Q5.3). With the sample instance: Zorba (71) and Rusty (58), both rating 10.

Trap / common mistake. Returning only one of them loses the tie mark. The natural join preserves both automatically — that is why the value-then-join form is preferred over tuple subtraction here.

Q6.29 Find the oldest sailor(s).

Answer.

$$Sailors \bowtie \left(\pi_{\text{age}}(Sailors) - \pi_{\text{S1.age}}\left(\sigma_{\text{S1.age} < \text{S2.age}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\right)\right)$$

Schema of `Sailors`.

Why. Same pattern with `age`. With the sample instance: Lubber (31) at 55.5.

Trap / common mistake. If ages were stored as strings (e.g. "45.0"), comparison would still work for these values but not in general for unequal digit counts; assume numeric.

Q6.30 Find the second-highest distinct rating.

Answer. Let $R(\text{MaxR}) = \pi_{\text{rating}}(Sailors) - \pi_{\text{S1.rating}}\left(\sigma_{\text{S1.rating} < \text{S2.rating}}(\rho_{\text{S1}}(Sailors) \times \rho_{\text{S2}}(Sailors))\right)$ and $R(\text{Rest}) = \pi_{\text{rating}}(Sailors) - R(\text{MaxR})$. Then

$$R(\text{Rest}) - \pi_{\text{R1.rating}}\left(\sigma_{\text{R1.rating} < \text{R2.rating}}(\rho_{\text{R1}}(R(\text{Rest})) \times \rho_{\text{R2}}(R(\text{Rest})))\right)$$

(rating) throughout.

Why. Remove the maximum, then take the maximum of the remainder — Q5.5. With the sample instance: ratings are {1, 7, 8, 9, 10}; removing 10 leaves {1,7,8,9} whose maximum is 9.

Trap / common mistake. "Second-highest rating" is 9 (Horatio 74), even though two sailors hold rating 10. Ties are collapsed because we are ranking **values**.

Q6.31 Find the names of sailors who have reserved only red boats.

Answer.

$$\pi_{\text{sname}}\left(Sailors \bowtie \left(\pi_{\text{sid}}(Reserves) - \pi_{\text{sid}}\left(Reserves \bowtie \sigma_{\text{color} \neq \textbf{red}}(Boats)\right)\right)\right)$$

(sid).

Why. Pattern 5: at least one reservation, minus any non-red reservation.

Trap / common mistake. A sailor with no reservations is not returned, because he is not in $\pi_{\text{sid}}(Reserves)$. If the question intends "vacuously only red", use $\pi_{\text{sname}}(Sailors) - \pi_{\text{sname}}\left(Sailors \bowtie \pi_{\text{sid}}\left(Reserves \bowtie \sigma_{\text{color} \neq \textbf{red}}(Boats)\right)\right)$ and say so.

Q6.32 Find the names of sailors who reserved a red boat and a green boat in the same reservation.

Not expressible in classical relational algebra. Under this schema the query is **not meaningful**, and the mark is for saying so.

Answer. A row of `Reserves` records **one** boat on **one** day: the schema is (sid, bid, day), so "one reservation" can never contain two boats. The question becomes answerable only under a different schema — a `Reservation(ResId, sid, day)` relation plus `Reservation_Item(ResId, bid)`. It is then a self-join on `ResId`:

$$\pi_{\text{sid}}\left(Reservation \bowtie \pi_{\text{ResId}}\left(\sigma_{\text{b1.color} = \textbf{red} \wedge \text{b2.color} = \textbf{green}}(\rho_{\text{b1}}(Reservation_Item \bowtie Boats) \times \rho_{\text{b2}}(Reservation_Item \bowtie Boats))\right)\right)$$

Under the standard schema: not expressible / not meaningful. Under the multi-item schema: (sid).

Why. This is the **reason** the standard schema is called "Sailors, Boats, Reserves" rather than "reservations with items": it cannot model a multi-boat reservation, and therefore the "same reservation" variant of the red-and-green question has no answer in it.

> If the examiner's intended reading is "on the same **day**", that **is** expressible:
>
> $$\pi_{\text{sname}}\left(Sailors \bowtie \pi_{\text{R1.sid}}\left(\sigma_{\text{R1.sid} = \text{R2.sid} \wedge \text{R1.day} = \text{R2.day}}\left(\rho_{\text{R1}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats)) \times \rho_{\text{R2}}\left(Reserves \bowtie \sigma_{\text{color} = \textbf{green}}(Boats)\right)\right)\right)\right)$$
>
> .

Trap / common mistake. A student who writes the "same day" expression without noticing the schema limitation usually still gets most of the marks, but not the mark awarded for identifying that a single reservation row holds a single boat. Say both.

Q6.33 Find the names of sailors who have reserved red boats but no other colours.
Answer.

$$\pi_{\text{sname}}\Big(Sailors \bowtie \big(\pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats)) - \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} \neq \textbf{red}}(Boats)\big)\big)\Big)$$

(sid) on both sides.
Why. "At least one red" gives the universe; "any non-red" is removed. This differs from Q6.31 only in how the universe is written: Q6.31 uses **all** reservations, which forces the sailor to have at least one reservation of any kind; Q6.33 forces at least one **red** one. For a sailor with no reservations the two agree (both exclude him); for a sailor with a blue boat only, both exclude him. The expressions differ if the table could contain a reservation with a NULL colour.

Trap / common mistake. $\sigma_{\text{color} \neq \textbf{red}}$ is the complement of $\sigma_{\text{color} = \textbf{red}}$ only when `color` is never NULL. State the assumption.

Q6.34 Find the boats that have been reserved by every sailor in the set {22, 31}.
Answer.

$$\pi_{\text{bid, sid}}(Reserves) \div \pi_{\text{sid}}(\sigma_{\text{sid} = 22 \vee \text{sid} = 31}(Sailors))$$

Divisor: (sid). Result: (bid).
Why. The divisor is a **constant relation** built by selection over `Sailors`. This is the standard way to quantify over a named finite set of stored entities: encode the set as a selection, then divide.

Trap / common mistake. If the set is supplied as a literal relation that is not in the database — say $R(\mathrm{S}) = \{(22), (31)\}$ — division still works, because the divisor need not be a stored relation. Write the constant relation explicitly and say that you are doing so.

Q6.35 Find the names of sailors whose every reservation satisfies the condition "the boat is red".
Answer.

$$\pi_{\text{sname}}\Big(Sailors \bowtie \big(\pi_{\text{sid}}(Reserves) - \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} \neq \textbf{red}}(Boats)\big)\big)\Big)$$

(sid).
Why. A universally quantified condition over a sailor's reservations is pattern 5: all rows minus the violating rows. This is the same shape as Q6.31, which shows that "only X" and "every reservation is X" are the same query in disguise.

Trap / common mistake. Note that this expression **also** requires at least one reservation. The purely universal reading would include sailors with no reservations (vacuous truth). Both readings must be labelled.

Q6.36 Find the ordered pairs of sailors (x, y) such that every boat reserved by x has a higher bid than every boat reserved by y.
Answer. Let $R(\mathrm{X}) = \pi_{\text{sid, bid}}(Reserves)$ and $R(\mathrm{Bad}) = \pi_{\text{x.sid, y.sid}}\big(\sigma_{\text{x.bid} \leq \text{y.bid}}\big(\rho_{\text{x}}(R(\mathrm{X})) \times \rho_{\text{y}}(R(\mathrm{X}))\big)\big)$. Then

$$(\pi_{\text{sid}}(Reserves) \times \pi_{\text{sid}}(Reserves)) - R(\mathrm{Bad})$$

$R(\mathrm{Bad})$: (x.sid, y.sid). The product of the two sid sets is union compatible with it, so the difference is legal. Result: (sid, sid) — ordered pairs.
Why. "$>$ every boat of y" is "$\neg$ ($\leq$ some boat of y)". The product of **all** sailors pairs with itself gives every candidate pair; the σ finds the pairs with at least one violating comparison; the difference leaves the good pairs. This is the counter-example method applied to a **pair** of tuple variables.

Trap / common mistake. The pairs where x has no reservations or y has no reservations are **vacuously** satisfying and are returned. If the question means non-vacuous, intersect with $\pi_{\text{sid}}(Reserves) \times \pi_{\text{sid}}(Reserves)$ restricted to sailors who reserved something — which is exactly what the universe above already does. Say which reading you use.

Q6.37 Find the sids of sailors who have made a reservation on every date that appears in a given relation $R(\text{Dates})(\text{day})$.
Answer.

$\pi_{\text{sid, day}}(Reserves) \div R(\text{Dates})$
Dividend: (sid, day). Divisor: (day). Result: (sid).
Why. The answer attribute is `sid`; the quantified attribute is `day`. Note that the dividend must be projected to exactly (sid, day) — if `bid` is kept, the result becomes (sid, bid) and asks "for every date, this sailor reserved **this particular boat** on it".

Trap / common mistake. If $R(\text{Dates})$ is the projection $\pi_{\text{day}}(Reserves)$ of the database itself, the query becomes "reserved on every date on which anybody reserved", which is a real and interesting query — and it is the one students often write by accident. Check which set the question names.

6.6 Answers on the sample instance — a quick self-check

Q	Answer on the instance	Rows
6.4	Dustin, Lubber, Horatio	3 names
6.10	Dustin, Lubber	2
6.12	Brutus, Andy, Rusty, Zorba	5 — note Horatio appears once as a name
6.13	none; all four boats were reserved	0
6.15	(22,64), (31,32), (58,71)	3 pairs
6.23	none; nobody reserved all four boats	0
6.28	Rusty, Zorba	2
6.29	Lubber	1
6.30	9	1

These counts are for the instance above only. They are a sanity check on your reading of the expression, never a substitute for it.

Part 7 — Books, Authors and Borrowing — complete solved question bank

What this part fixes. The library schema exercises **many-to-many relationship** reasoning: a book has several authors, a member borrows many books, and the same book may be borrowed many times on different dates. Every question states whether it uses the **full** schema or the **simplified** one (§1.2B), because several queries are meaningful only in one of them.

7.0 The two schemas, side by side

Full schema

$R(\text{Book})(\text{BookID}, \text{Title}, \text{Publisher}, \text{Category})$
$R(\text{Author})(\text{AuthorID}, \text{AuthorName})$
$R(\text{WrittenBy})(\text{BookID}, \text{AuthorID})$
$R(\text{Member})(\text{MemberID}, \text{MemberName})$
$R(\text{Borrow})(\text{MemberID}, \text{BookID}, \text{BorrowDate}, \text{ReturnDate})$

Simplified schema

$R(\text{Books})(\text{BookID}, \text{Title})$
$R(\text{BookAuthors})(\text{BookID}, \text{Author})$
$R(\text{Borrow})(\text{MemberID}, \text{BookID})$

No dates, no publisher, no separate author entity. A member borrows a given book at most once, so "borrowed twice" is undefined.,

Where the two schemas differ in expressiveness. Anything mentioning a **date** — "latest", "at least two dates", "not yet returned" — is answerable only in the full schema. Anything mentioning an author as an **entity** with an id ("authors who wrote every category") needs the full schema. The simplified schema is used for the many "who borrowed what" questions in §7.1–7.2 because it keeps the expressions short and the idea visible.

7.1 Basic and join queries

Q7.1 Find all book titles. (full schema)

Answer.

$\pi_{\text{Title}}(Book)$
(Title), duplicates removed — two books with the same title appear once.
Why. Pure projection.

Trap / common mistake. If the question wants one row per book, project (BookID, Title) instead, and say why. "List all titles" with set semantics cannot distinguish two copies of a title.

Q7.2 Find the books published by "Pearson". (full schema)

Answer.

$\sigma_{\text{Publisher} = \textbf{Pearson}}(Book)$
Schema of `Book`.
Why. Tuple-local selection.

Trap / common mistake. Selecting $\pi_{\text{Title}}(Book)$ first and then filtering is a type error, because `Publisher` is gone.

Q7.3 Find the titles of books written by the author with AuthorID 7. (full schema)

Answer.

$\pi_{\text{Title}}(Book \bowtie \sigma_{\text{AuthorID} = 7}(WrittenBy))$
(Title).
Why. `WrittenBy` carries the link; `Book` carries the title; the natural join is on `BookID`.

Trap / common mistake. Joining `WrittenBy` with `Author` is unnecessary for this question — the author is identified by id, not by name. Adding it is harmless but wastes a join.

Q7.4 Find the names of the authors of the book titled "DBMS". (full schema)

Answer.

$\pi_{\text{AuthorName}}(Author \bowtie WrittenBy \bowtie \sigma_{\text{Title} = \textbf{DBMS}}(Book))$
(AuthorName).
Why. Two joins walk `Author` ⟶ `WrittenBy` ⟶ `Book`. This is the standard "many-to-many" chain, and the filter is pushed to the `Book` leaf.

Trap / common mistake. If the title is not unique, all authors of all books with that title are returned — which is what the question literally asks. Say so if the schema does not make `Title` a key.

Q7.5 Find the members who borrowed the book with BookID 12. (full schema)

Answer.

$\pi_{\text{MemberID}}(\sigma_{\text{BookID} = 12}(Borrow))$
(MemberID).
Why. `Borrow` already contains both attributes; no join is needed.

Trap / common mistake. Projecting to `MemberID` with π removes duplicates, so a member who borrowed book 12 three times appears once. If the question asks for borrowing **events**, keep the date.

Q7.6 Find the titles of books borrowed by the member with MemberID 5. (full schema)

Answer.

$\pi_{\text{Title}}(Book \bowtie \sigma_{\text{MemberID}\,=5}(Borrow))$

(Title).

Why. Filter on `Borrow`, join to `Book`, project the title.

Trap / common mistake. If the member borrowed the same title twice, the projection collapses the two rows. That is correct under set semantics and is usually what "titles" means.

Q7.7 Find the member names together with the titles of the books they borrowed. (full schema)

Answer.

$\pi_{\text{MemberName, Title}}(Member \bowtie Borrow \bowtie Book)$

(MemberName, Title).

Why. The natural join equates `MemberID` between `Member` and `Borrow`, and `BookID` between `Borrow` and `Book`; the projection keeps only the two descriptive attributes.

Trap / common mistake. The result is a set of **pairs**. If a member borrowed two books with the same title, or two members share a name, the pairs collapse — and that is correct for a pair-set. If the question wants ids as well, include them.

Q7.8 Find the titles of books written by any author in the set {AuthorID 3, AuthorID 5}. (full schema)

Answer.

$\pi_{\text{Title}}(Book \bowtie \sigma_{\text{AuthorID}\,=3 \vee \text{AuthorID}\,=5}(WrittenBy))$

(Title).

Why. A finite set of identifiers is a disjunction inside one selection.

Trap / common mistake. If the set has twenty members, the selection becomes unwieldy; the alternative is to store the set as a relation and use a natural join, which is exactly the "constant relation" technique of Q6.34.

Q7.9 Find the books that have never been borrowed. (full schema)

Answer.

$\pi_{\text{BookID}}(Book) - \pi_{\text{BookID}}(Borrow)$

(BookID).

Why. Universe = all books; witness = any borrow row.

Trap / common mistake. $\pi_{\text{Title}}(Book) - \pi_{\text{Title}}(Book \bowtie Borrow)$ is the name-set version and is wrong when two books share a title. Subtract on the key.

Q7.10 Find the members who have never borrowed a book. (full schema)

Answer.

$\pi_{\text{MemberID}}(Member) - \pi_{\text{MemberID}}(Borrow)$

(MemberID).

Why. Same negative pattern with the member as the universe.

Trap / common mistake. Do not subtract on `MemberName`; two members may share a name, and the difference would delete the wrong one.

7.2 Set operations

Q7.11 Find the members who have borrowed book A (BookID 12) or book B (BookID 15). (full schema)
Answer.

$\pi_{\text{MemberID}}(\sigma_{\text{BookID}=12 \vee \text{BookID}=15}(Borrow))$
(MemberID), a set.
Why. One selection with a disjunction. The union form $\pi_{\text{MemberID}}(\sigma_{\text{BookID}=12}(Borrow)) \cup \pi_{\text{MemberID}}(\sigma_{\text{BookID}=15}(Borrow))$ is equally correct.

Trap / common mistake. Union is **not** needed and, if written carelessly as a union of `Borrow` relations, is still fine here (they have the same schema) — but the projection must come after the union, not inside it, or the two sides may not line up.

Q7.12 Find the members who have borrowed both book A and book B. (full schema)
Answer.

$\pi_{\text{MemberID}}(\sigma_{\text{BookID}=12}(Borrow)) \cap \pi_{\text{MemberID}}(\sigma_{\text{BookID}=15}(Borrow))$
(MemberID).
Why. Two independent existences; intersection on the member key.

Trap / common mistake. The intersection cannot be taken on `Borrow` tuples: a single borrow row has one book, so intersecting the two filtered `Borrow` relations always yields the empty set.

Q7.13 Find the members who have borrowed book A but not book B. (full schema)
Answer.

$\pi_{\text{MemberID}}(\sigma_{\text{BookID}=12}(Borrow)) - \pi_{\text{MemberID}}(\sigma_{\text{BookID}=15}(Borrow))$
(MemberID).
Why. Difference of key sets.

Trap / common mistake. Note that this does **not** say "borrowed only A"; a member who borrowed A and C is included. That is Q7.31's territory.

Q7.14 Find the books written by author A but not by author B. (full schema)
Answer.

$\pi_{\text{BookID}}(\sigma_{\text{AuthorID}=3}(WrittenBy)) - \pi_{\text{BookID}}(\sigma_{\text{AuthorID}=5}(WrittenBy))$
(BookID).
Why. Both operands are sets of book identifiers.

Trap / common mistake. A book with **two** authors, 3 and 5, appears on both sides and is correctly excluded — "written by A but not by B" means B is not among its authors. Check the wording: "written by A but not by B" is not "written by A alone".

Q7.15 Find the books that have more than one author. (full schema)
Answer.

$\pi_{\text{W1.BookID}}\left(\sigma_{\text{W1.BookID} = \text{W2.BookID} \wedge \text{W1.AuthorID} \neq \text{W2.AuthorID}}(\rho_{\text{W1}}(WrittenBy) \times \rho_{\text{W2}}(WrittenBy))\right)$
(BookID).
Why. "More than one" is "at least two distinct", which is a self-join with $\neq$ on the author. No counting is needed, because **two** is a fixed numeral: two distinct witnesses suffice.

Trap / common mistake. $\pi_{\text{BookID}}(WrittenBy)$ cannot answer this. A two-author book produces two different tuples (b, a_1) and (b, a_2), and projecting to `BookID` collapses them to one row — so the projection **destroys** exactly the information you need. Duplicate rows are not duplicates of the key.

Q7.16 Find the authors who have not written any book. (full schema)

Answer.

$\pi_{\text{AuthorID}}(Author) - \pi_{\text{AuthorID}}(WrittenBy)$
(AuthorID).
Why. Universe = all authors; witness = any `WrittenBy` row.

Trap / common mistake. To print names, join back afterwards: $\pi_{\text{AuthorName}}(Author \bowtie (\pi_{\text{AuthorID}}(Author) - \pi_{\text{AuthorID}}(WrittenBy)))$.

7.3 Division and universal queries

Q7.17 Find the members who have borrowed every book. (full schema)
Answer.

$\pi_{\text{MemberID, BookID}}(Borrow) \div \pi_{\text{BookID}}(Book)$
(MemberID).
Why. The dividend is projected to (answer attribute, quantified attribute); the divisor is the whole book set.

Trap / common mistake. Dividing `Borrow` itself is **legal** in the full schema, because `Borrow` has attributes (MemberID, BookID, BorrowDate, ReturnDate) and the divisor's schema is a subset — but the result is (MemberID, BorrowDate, ReturnDate), a different and much stronger query. Project first.

Q7.18 Find the members who have borrowed every book in the category "DB". (full schema)
Answer.

$\pi_{\text{MemberID, BookID}}(Borrow) \div \pi_{\text{BookID}}(\sigma_{\text{Category} = \text{DB}}(Book))$
(MemberID).
Why. The category restriction belongs inside the divisor, because it defines the quantified set.

Trap / common mistake. If no book has category "DB", the divisor is empty and every member who has borrowed at least one book is returned. State the vacuous-truth reading.

Q7.19 Find the members who have borrowed all books written by author 7. (full schema)
Answer.

$\pi_{\text{MemberID, BookID}}(Borrow) \div \pi_{\text{BookID}}(\sigma_{\text{AuthorID} = 7}(WrittenBy))$
(MemberID).
Why. The quantified set is computed through the relationship `WrittenBy`.

Trap / common mistake. A student may try to use `Author` here; only `WrittenBy` carries the link. Including `Author` adds nothing and can hide the mistake.

Q7.20 "Find the authors who wrote every book in a given set." Under what reading is this expressible? (full schema)
Answer. **Not** as stated for a general set of books: "every book" is a fixed set only if the set is named in the query. With a named set $R(\text{S})(\text{BookID})$ it **is** expressible:

$$\pi_{\text{AuthorID, BookID}}(WrittenBy) \div R(\text{S})$$

(AuthorID).
Why. Division quantifies over a **given** set. If the set is "all books in the library" it is $\pi_{\text{AuthorID, BookID}}(WrittenBy) \div \pi_{\text{BookID}}(Book)$, which is a real (and usually empty) query. If the set is "all books in some category" it is again division with a filtered divisor.

Trap / common mistake. "Every book" without a qualifier means $\pi_{\text{BookID}}(Book)$, and in practice no author writes every book — expect the empty relation and do not "repair" it.

Q7.21 Find the publishers that publish books in every required category, where the required categories are given by $R(\text{ReqCat})(\text{Category})$. (full schema)
Answer.

$\pi_{\text{Publisher, Category}}(Book) \div R(\text{ReqCat})$
(Publisher).
Why. The relation of interest is (Publisher, Category) pairs, obtained by projecting `Book`. The divisor is the requirement relation. This is the cleanest example of "publisher covers all categories".

Trap / common mistake. If the requirement is "every category that occurs in the library", use $\pi_{\text{Publisher, Category}}(Book) \div \pi_{\text{Category}}(Book)$. Both are sensible; read which one the question specifies.

Q7.22 Find the members who have borrowed every book borrowed by member 5. (full schema)
Answer.

$\pi_{\text{MemberID, BookID}}(Borrow) \div \pi_{\text{BookID}}(\sigma_{\text{MemberID}=5}(Borrow))$
(MemberID).
Why. Exactly the pattern of Q4.4 with the library schema.

Trap / common mistake. Member 5 appears in his own answer; whether to exclude him is a matter of the question's wording, not of the algebra. If asked to exclude, subtract $\pi_{\text{MemberID}}(\sigma_{\text{MemberID}=5}(Borrow))$.

Q7.23 Explain the correct dividend and divisor schemas for the four division queries above.

Query	Dividend	Divisor	Result
7.17	(MemberID, BookID)	(BookID)	(MemberID)
7.18	(MemberID, BookID)	(BookID) from DB books	(MemberID)
7.19	(MemberID, BookID)	(BookID) of author 7	(MemberID)
7.21	(Publisher, Category)	(Category) required	(Publisher)

In every case the dividend is exactly "answer attributes ∪ divisor attributes", and the divisor is exactly the set being quantified over. Nothing else is allowed.

7.4 Advanced queries

Q7.24 Find the most-borrowed book (the book borrowed the largest number of times), without using counting. Is it expressible? (full schema)

Not expressible in classical relational algebra. "Most borrowed" compares **counts** of borrow rows. Classical relational algebra cannot count, so the query is **not expressible**.

Answer. Not expressible. It becomes expressible with the extended operator γ:

$$\gamma_{\text{BookID};\ \text{count}(\text{MemberID})\leftarrow n}(Borrow)$$

, then the maximum-of-values pattern on n, then a join back to `Book` for the title.
Why. This is **the** example examiners use to test whether you understand the limits of the algebra. Note the contrast with "the most expensive book", which **is** expressible because price is a stored value; "most borrowed" is a stored **count** that must be computed.

Trap / common mistake. A very common wrong answer is "the book that is not dominated by any other book", where domination is defined by a self-join on `BookID` — but counting has no tuple-level counterpart, so the natural expressions either return all books or none. If you write one, explain why it does not compute the maximum count.

Q7.25 Find the latest borrowing date. (full schema)
Answer.

$\pi_{\text{BorrowDate}}(Borrow) - \pi_{\text{B1.BorrowDate}}(\sigma_{\text{B1.BorrowDate} < \text{B2.BorrowDate}}(\rho_{\text{B1}}(Borrow) \times \rho_{\text{B2}}(Borrow)))$
(BorrowDate).
Why. The standard maximum-of-values pattern (Q5.1) applied to a date attribute. With ISO dates, the string comparison $<$ coincides with the chronological one.

Trap / common mistake. If dates are stored as DD/MM/YYYY, string comparison is chronological only within a year and only if zero-padded. State the assumption or compare components.

Q7.26 Find the members who borrowed books on at least two distinct dates. (full schema)
Answer.

$\pi_{\text{MemberID}}\left(\sigma_{\text{B1.MemberID} = \text{B2.MemberID} \wedge \text{B1.BorrowDate} \neq \text{B2.BorrowDate}}(\rho_{\text{B1}}(Borrow) \times \rho_{\text{B2}}(Borrow))\right)$
(MemberID).
Why. Self-join with equality on the member and inequality on the date.

Trap / common mistake. Do not use the natural join: `Borrow` has three shared attributes with its renamed copy and the join would require the rows to be identical. Use $\times$ plus explicit conditions.

Q7.27 Find the books borrowed by every member in the set {MemberID 1, MemberID 2}. (full schema)
Answer.

$\pi_{\text{BookID, MemberID}}(Borrow) \div \pi_{\text{MemberID}}(\sigma_{\text{MemberID} =1 \vee \text{MemberID} =2}(Member))$
(BookID).
Why. The divisor is a constant relation built by selection — Q6.34's technique.

Trap / common mistake. If one of the two members does not exist in `Member`, the divisor has one row instead of two and the answer silently changes. Build the constant relation explicitly if the question supplies it as data.

Q7.28 Find the authors who have written books in every category in $R(\text{ReqCat})(\text{Category})$. (full schema)
Answer. We need an author–category relation. Build it by joining:

$$R(\text{AC}) = \pi_{\text{AuthorID, Category}}(WrittenBy \bowtie Book)$$

and then divide:

$$R(\text{AC}) \div R(\text{ReqCat})$$

.
$R(\text{AC})$: (AuthorID, Category). Result: (AuthorID).
Why. Division needs a binary relation pairing the answer attribute with the quantified attribute. That relation does not exist in the schema; you must **derive** it by joining `WrittenBy` with `Book`. This "derive the relation, then divide" step is what the question is really testing.

Trap / common mistake. Trying to divide `WrittenBy` directly fails because its schema is (BookID, AuthorID) and `Category` never appears. The join is not optional.

Q7.29 Find the members who have borrowed books by every author in the set {3, 5}. (full schema)
Answer. Let $R(\text{MB}) = \pi_{\text{MemberID, AuthorID}}(Borrow \bowtie WrittenBy)$. Then

$$R(\text{MB}) \div \pi_{\text{AuthorID}}(\sigma_{\text{AuthorID} =3 \vee \text{AuthorID} =5}(Author))$$

$R(\text{MB})$: (MemberID, AuthorID). Result: (MemberID).

Why. Again the binary relation must be derived. Note the join path: `Borrow` ⟶ `WrittenBy` on `BookID`.

Trap / common mistake. A member who borrowed a book with **both** authors 3 and 5 satisfies the condition with a single borrow. That is correct for the "has borrowed a book by every author" reading, and it is the reading division implements.

Q7.30 Find the books that were borrowed but never returned. State the NULL convention. (full schema)

Classical relational algebra has no NULL. Two conventions are possible and the answer must say which one is in force. **(a) Sentinel convention.** $A(\text{ReturnDate}) = \perp$ marks "not yet returned". Then the answer is a straightforward selection. **(b) Null convention.** $A(\text{ReturnDate})$ may be NULL for "unknown", and comparison with NULL is never true. Then "not returned" is **not** the complement of "returned": an unknown return date does not prove the book is outstanding. The honest classical answer is "not expressible", and the honest SQL answer needs `IS NULL`.

Answer. Under convention **(a)**:

$$\pi_{\text{BookID},\ \text{MemberID}}\left(\sigma_{\text{ReturnDate} = \perp}(Borrow)\right)$$

Under convention **(b)**: not expressible in classical relational algebra without extending the schema; with SQL NULL semantics it is `WHERE ReturnDate IS NULL`.
(BookID, MemberID) — or add `BorrowDate` if individual loans are wanted.

Why. The trap is the word "never": a book returned once and borrowed again and not yet returned **is** currently outstanding, but it was returned at least once. The expression above answers the **current** state; the historical question needs a division over loan events.

Trap / common mistake. $\pi_{\text{BookID}}(Borrow) - \pi_{\text{BookID}}\left(\sigma_{\text{ReturnDate} \neq \perp}(Borrow)\right)$ looks right and is right under convention (a) — it is the same set. Under convention (b) it is wrong because `!= NULL` is not true. State the convention before you write the expression.

Q7.31 Find the members who have borrowed only books from the category "DB". (full schema)

Answer.

$$\pi_{\text{MemberID}}(Borrow) - \pi_{\text{MemberID}}\left(\sigma_{\text{Category} \neq \text{DB}}(Borrow \bowtie Book)\right)$$

(MemberID).

Why. Pattern 5: members with at least one borrow, minus members with a non-DB borrow. The join with `Book` is performed once, inside the subtracted term.

Trap / common mistake. "Only DB books" excludes members who borrowed nothing, because they are not in $\pi_{\text{MemberID}}(Borrow)$. If the intended reading is "vacuously only DB", use $\pi_{\text{MemberID}}(Member) - \pi_{\text{MemberID}}\left(\sigma_{\text{Category} \neq \text{DB}}(Borrow \bowtie Book)\right)$ and say so. Also note that $\sigma_{\text{Category} \neq \text{DB}}$ is the complement of $\sigma_{\text{Category} = \text{DB}}$ only when `Category` is never NULL.

Q7.32 Find the members who have borrowed books from category A but none from category B. (full schema)

Answer.

$$\pi_{\text{MemberID}}\left(\sigma_{\text{Category} = \text{A}}(Borrow \bowtie Book)\right) - \pi_{\text{MemberID}}\left(\sigma_{\text{Category} = \text{B}}(Borrow \bowtie Book)\right)$$

(MemberID).

Why. Two independent existential conditions, one positive and one negative, combined by difference.

Trap / common mistake. A member who borrowed a book that is **both** A and B (impossible here, since category is a single attribute — but possible if categories were multi-valued) would be excluded. With a multi-valued category, the second term should select on the same book. Say which model you use.

Q7.33 Find the pairs of members who have borrowed exactly the same set of books. (full schema)

Answer. Let $R(\mathrm{MB}) = \pi_{\mathrm{MemberID},\ \mathrm{BookID}}(Borrow)$ and let $R(\mathrm{Pairs}) = \pi_{\mathrm{MemberID}}(Member) \times \pi_{\mathrm{MemberID}}(Member)$ be every ordered pair of members. Define the **violation** relation

$$R(\mathrm{V}) = \pi_{\mathrm{A.mid},\ \mathrm{A.bid},\ \mathrm{B.mid}}(\rho_{\mathrm{A}}(R(\mathrm{MB})) \times \rho_{\mathrm{Mem}}(\pi_{\mathrm{MemberID}}(Member))) - \pi_{\mathrm{A.mid},\ \mathrm{A.bid},\ \mathrm{B.mid}}(\sigma_{\mathrm{A.bid}\ =\ \mathrm{B.bid}}(\rho_{\mathrm{A}}(R(\mathrm{MB})) \times \rho_{\mathrm{B}}(R(\mathrm{MB}))$$

Then $R(\mathrm{Bad}) = \pi_{\mathrm{A.mid},\ \mathrm{B.mid}}(R(\mathrm{V}))$ holds the ordered pairs (a, b) such that a has a book that b lacks, and the answer is

$$R(\mathrm{Pairs}) - R(\mathrm{Bad}) - \rho_{\mathrm{A.mid}\ \leftarrow\ \mathrm{B.mid},\ \mathrm{B.mid}\ \leftarrow\ \mathrm{A.mid}}(R(\mathrm{Bad}))$$

$R(\mathrm{MB})$: (MemberID, BookID). $R(\mathrm{Pairs})$: (MemberID, MemberID). $R(\mathrm{V})$: three attributes — a member, a book, and a member who lacks that book. $R(\mathrm{Bad})$: two member ids. The final difference has the same schema as $R(\mathrm{Pairs})$.

Why. "Exactly the same set" is **two** coverage conditions: a covers b and b covers a. The violation relation $R(\mathrm{V})$ computes, for every ordered pair and every book, whether the first member has the book while the second does not — a three-place statement that needs three tuple variables. Subtracting the projection of $R(\mathrm{V})$ removes every pair where coverage fails in one direction; subtracting the **swapped** version removes failures in the other.

The compact textbook statement is "(a, b) such that each has borrowed every book the other has", i.e. two divisions. The expression above is that statement written out with the five primitive operators, and it is what to produce if the question forbids division. Note that the diagonal pair (a, a) survives, which is correct: every member has borrowed exactly the same books as himself. Remove it with $\sigma_{\mathrm{A.mid}\ \neq\ \mathrm{B.mid}}$ if unordered distinct pairs are wanted.

Trap / common mistake. Testing only one direction returns all pairs (a, b) where b borrowed everything a did — a **superset** condition, not equality. Both directions are required, and it is the second subtraction that makes the query symmetric.

Part 8 — Hospital and Prescription — complete solved question bank

What this part fixes. The uploaded university exam pattern, in full. The five original questions are Q8.1–Q8.5 and are solved with the same care as the rest; Q8.6–Q8.27 extend the schema across joins, self-joins, division, difference, extremes and multi-condition queries. The two attributes named `SSN`, and the two named `FirstName`/`LastName`, make renaming mandatory in almost every query — that is the single most examined point of this schema.

8.0 Schema, keys and the two name clashes

Relation	Attributes	Keys and foreign keys
Doctor	("SSN", "FirstName", "LastName", "Specialty", "YearsOfExperience", "PhoneNum")	PK: "SSN"
Patient	("SSN", "FirstName", "LastName", "Address", "DOB", "PrimaryDoctor_SSN")	PK: "SSN"; "PrimaryDoctor_SSN" → Doctor.SSN
Medicine	("TradeName", "UnitPrice", "GenericFlag")	PK: "TradeName"
Prescription	("Id", "Date", "Doctor_SSN", "Patient_SSN")	PK: "Id"; "Doctor_SSN" → Doctor.SSN "Patient_SSN" → Patient.SSN
Prescription_Medicine	("Prescription_Id", "TradeName", "NumOfUnits")	PK: ("Prescription_Id", "TradeName"); "Prescription_Id" →

Relation	Attributes	Keys and foreign keys
		Prescription.Id “TradeName” → Medicine

Name clashes to check before writing any expression.

1. `Doctor.SSN` and `Patient.SSN` — same name, different entities. Always rename one: $\rho_{\mathrm{D}}(Doctor)$ or $\rho_{\mathrm{P}}(Patient)$, or qualify as $A(\text{Doctor.SSN})$.
2. `Doctor.FirstName`, `Doctor.LastName` and `Patient.FirstName`, `Patient.LastName` — same problem for names.
3. `Prescription.Patient_SSN` and `Patient.SSN` — **different** names, so an explicit theta join $\overset{}{\bowtie}_{\text{Patient_SSN} = \text{SSN}}$ is needed; a natural join would silently produce nothing (no shared attribute name) and the result would be a Cartesian product.

8.1 The five original exam questions

Q8.1 **List the first and last names of patients whose primary doctor is named John Smith.**
Answer.

$$\pi_{\text{Patient.FirstName},\ \text{Patient.LastName}}\Big(Patient \underset{\text{Patient.PrimaryDoctor_SSN} = \text{Doctor.SSN}}{\bowtie} \sigma_{\text{FirstName} = \textbf{John} \wedge \text{LastName} = \textbf{Smith}}(\rho_{\text{Doctor}}(Doctor))\Big)$$

The theta join's result carries all six `Patient` attributes plus all six `Doctor` attributes — the join predicate uses attributes with **different** names, so nothing is merged. The projection leaves the patient's two name attributes, and they must be **qualified**, because `Doctor` also has `FirstName` and `LastName`.

Why. Two relations, one join, one selection. The join predicate is `Patient.PrimaryDoctor_SSN = Doctor.SSN` — an equijoin between attributes with **different** names, which is exactly why a natural join cannot be used here.

Two equally acceptable spellings. Renaming the patient's attributes at the source avoids all ambiguity:

$$\pi_{\text{pfn},\ \text{pln}}\Big(\rho_{\text{pfn} \leftarrow \text{FirstName},\ \text{pln} \leftarrow \text{LastName}}(Patient) \underset{\text{PrimaryDoctor_SSN} = \text{D.SSN}}{\bowtie} \sigma_{\text{D.FirstName} = \textbf{John} \wedge \text{D.LastName} = \textbf{Smith}}(\rho_{\mathrm{D}}(Doctor))\Big)$$

. Or qualify the projection as above. What is **not** acceptable is an unqualified $\pi_{\text{FirstName},\ \text{LastName}}$ over a relation that owns both attributes twice.

Trap / common mistake. Three errors appear again and again. (i) **Using a natural join**: `Patient` and `Doctor` share `SSN`, `FirstName` and `LastName`, so the natural join equates the patient's own SSN with the doctor's — it returns patients who **are** John Smith, not patients **of** John Smith. (ii) Treating “John Smith” as one value instead of a conjunction of two attributes. (iii) Matching the **patient's** name against “John” instead of the doctor's. Read the sentence twice: **whose primary doctor is named** John Smith.

Q8.2 **List the first and last names of doctors who are not primary doctors to any patient.**
Answer. The simple form, subtracting on the key, is

$$\pi_{\text{FirstName},\ \text{LastName}}\big(Doctor \bowtie (\pi_{\text{SSN}}(Doctor) - \pi_{\text{PrimaryDoctor_SSN}}(Patient))\big)$$

The fully qualified form, which shows the join explicitly, is

$$\pi_{\text{D.FirstName},\ \text{D.LastName}}\big(\rho_{\mathrm{D}}(Doctor) \bowtie \rho_{\mathrm{M}}(\pi_{\text{SSN}}(Doctor) - \pi_{\text{PrimaryDoctor_SSN}}(Patient))\big)$$

Here ρ_{M} names the intermediate set of “doctors who are somebody's primary doctor”, and the natural join on `SSN` keeps the doctors outside it.

The inner difference is (SSN) on both sides — union compatible, because $\pi_{\text{PrimaryDoctor_SSN}}(Patient)$ is a set of doctor identifiers even though the attribute has a different **name**. The natural join in the simple form equates `SSN`, and the projection gives the doctor's names.

Why. Pattern 2: universe = all doctors; witness = “appears in the `PrimaryDoctor_SSN` column”. A doctor who has **prescribed** many times but is nobody's primary doctor is correctly returned — see Q8.3's distinction list.

Trap / common mistake. The name-based version $\pi_{\text{FirstName, LastName}}(Doctor) - \pi_{\text{FirstName, LastName}}\big(Doctor \bowtie Patient \bowtie \dots\big)$ is wrong whenever two doctors share a name. It is also wrong in a subtler way: the natural join `Doctor` $\bowtie$ `Patient` matches on `SSN`, `FirstName` and `LastName` simultaneously, which is not the join you want. Subtract on the key, always.

Q8.3 For medicines written in more than 20 prescriptions, report the trade name and the total number of units prescribed. Do not use COUNT() or SUM(). Explain whether the requested total can be computed in classical relational algebra, and distinguish the count of distinct prescriptions from the sum of units.

Not expressible in classical relational algebra. Both parts of the request — "more than 20 prescriptions" and "total number of units" — require aggregation. Neither is expressible in classical relational algebra.

Answer. Not expressible. With the extended operator the intended query is

$$\gamma_{\text{TradeName; count(Prescription_Id)}\leftarrow \text{ n, sum(NumOfUnits)}\leftarrow \text{ units}}(Prescription_Medicine)$$

followed by $\sigma_{\text{n} > 20}$ and a join with `Medicine` if `UnitPrice` is also wanted. In classical relational algebra you may only write the **fixed-cardinality** fragments: "appears in at least two prescriptions", "appears in at least three prescriptions", and so on.

The γ result has schema (TradeName, n, units); the classical fragments are all subsets of $\pi_{\text{TradeName}}(Prescription_Medicine)$, i.e. schema (TradeName).

Why. Three separate arguments are needed for full marks.

1. **"More than 20"** compares a numeral with the **cardinality** of a set of tuples. Cardinality is not a term of the algebra; there is no operator that turns a relation into a number.
2. **"Total number of units"** is a **sum** over a set of values, which again is not a term. Even with a fixed bound on the number of prescriptions, the sum of an unbounded multiset of values cannot be expressed.
3. **The two conditions are different.** "Written in more than 20 prescriptions" counts **rows** of `Prescription_Medicine`; "total units exceeds 20" sums the `NumOfUnits` column. A medicine prescribed once with 500 units fails the first test and passes the second. Contrast, on the same instance: Aspirin in 3 prescriptions of 100 units each has $n = 3$ and 300 units; Vitamin in 1 prescription of 300 units has $n = 1$ and 300 units. Same total, different counts — which is exactly why a student who answers one is not answering the other.

What you can **write.** For a fixed numeral k, "appears in at least k prescriptions" is expressible with k distinct tuple variables:

$$R(\text{AtLeast2}) = \pi_{\text{P1.TradeName}}\Big(\sigma_{\text{P1.TradeName} = \text{P2.TradeName} \wedge \text{P1.Prescription_Id} \neq \text{P2.Prescription_Id}}(\rho_{\text{P1}}(Prescription_Medicine) \times \rho_{\text{P2}}(Prescription_Medicine))\Big)$$

and similarly for 3, 4, … . "More than 20" would need twenty-one variables, and — crucially — the resulting formula changes **shape** with the numeral, so it does not express the query "more than 20" as a function of a parameter. The examinable statement is: **fixed cardinality is expressible; parameterised cardinality is not.**

Trap / common mistake. Do not write a γ expression and call it relational algebra. Do not invent a $\div$ or a $-$ expression and hope. The mark for this question is awarded for (i) saying "not expressible", (ii) naming the missing operator, (iii) explaining **why** it is missing, and (iv) distinguishing the row count from the unit sum. A script that says all four gets full marks even without writing the γ formula.

Q8.4 List the first and last names of patients who have no prescriptions written by doctors other than their primary doctors.

Answer. Define the set of **violators** — patients who have at least one prescription written by a doctor other than their own primary doctor:

$$R(\text{Bad}) = \pi_{\text{SSN}}\Big(Patient \underset{\text{Patient.SSN} = \text{Prescription.Patient_SSN}}{\bowtie} \sigma_{\text{PrimaryDoctor_SSN} \neq \text{Doctor_SSN}}(Prescription)\Big)$$

Then the answer is

$$\pi_{\text{FirstName, LastName}}(Patient \bowtie (\pi_{\text{SSN}}(Patient) - R(\text{Bad})))$$

.

$R(\text{Bad})$ has schema (SSN) — patient identifiers. Both operands of the difference are (SSN), so it is union compatible. The final natural join on `SSN` restores the two name attributes.

Why. This is pattern 2. The universe is **all patients**; the witness is a prescription whose `Doctor_SSN` differs from that patient's own `PrimaryDoctor_SSN`. The correlation is essential: the two attributes must be read from the **same** patient's row, which is why `Patient` is joined into the witness relation before the comparison.

The three cases the question is really testing.

Case	Description	Returned?
A	patient with no prescriptions at all	**Yes** — vacuously: he has no prescription by another doctor
B	patient whose every prescription was written by his primary doctor	**Yes**
C	patient with at least one prescription by another doctor	**No**

So "no prescriptions by other doctors" is A ∪ B. It is **not** "prescriptions, and all of them by my primary doctor" (B only). The wording of the question asks for A ∪ B; if your examiner wants B only, intersect with $\pi_{\text{Patient_SSN}}(Prescription)$ and say so.

Trap / common mistake. The most common error is to forget the correlation: writing $\pi_{\text{Patient_SSN}}\left(\sigma_{\text{Doctor_SSN} \neq \text{PrimaryDoctor_SSN}}(Prescription \times Patient)\right)$ compares the prescribing doctor with the primary doctor of **every** patient in the cross product. Without the equality $A(\text{Patient_SSN}) = A(\text{SSN})$ the expression is meaningless, yet it returns a plausible-looking set and is rarely caught.

Q8.5 List the SSNs of patients who have Aspirin and Vitamin trade names in one prescription.

Answer.

$$\text{atient_SSN}\left(Prescription \underset{\text{Pr.Id} = \text{P1.Prescription_Id}}{\bowtie} \rho_{\text{P1}}(Prescription_Medicine) \underset{\text{P1.Prescription_Id} = \text{P2.Prescription_Id}}{\bowtie} \rho_{\text{P2}}(Prescription_Medici\right.$$

with the conditions $A(\text{P1.TradeName}) = \textbf{Aspirin}$ and $A(\text{P2.TradeName}) = \textbf{Vitamin}$ added inside a σ, giving

$$\left.\text{= Aspirin} \wedge \text{P2.TradeName} = \text{Vitamin}\left(Prescription \underset{\text{Id} = \text{P1.Prescription_Id}}{\bowtie} \rho_{\text{P1}}(Prescription_Medicine) \underset{\text{P1.Prescription_Id} = \text{P2.Prescription_Id}}{\bowtie} \rho_{\text{P2}}(P\right.\right.$$

The two copies of `Prescription_Medicine` are renamed so that the **same** `Prescription_Id` can appear twice, once for Aspirin and once for Vitamin. Result: (Patient_SSN).

Why. "In one prescription" is an equality between the two medicine rows' prescription identifiers. Two copies of the relation are needed because the two medicines are two different rows; the equality on `Prescription_Id` is what forces them into the **same** prescription, and the join with `Prescription` supplies the patient. This is pattern 7 (self-join) combined with pattern 1 (existence).

Why the equality is the whole question. If you drop $A(\text{P1.Prescription_Id}) = A(\text{P2.Prescription_Id})$ and keep only "the same patient", you answer "has Aspirin in some prescription and Vitamin in some (possibly different) prescription" — which is a **different** and much more common situation. The exam is testing exactly this. Use two tuple variables and one equality.

Trap / common mistake. Three variants, all wrong for this question:

1. $\sigma_{\text{TradeName} = \textbf{Aspirin} \wedge \text{TradeName} = \textbf{Vitamin}}(Prescription_Medicine)$ — one row cannot have two trade names; the result is empty.

2. $\pi_{\text{Patient_SSN}}(Prescription \bowtie \sigma_{\text{TradeName} = \textbf{Aspirin}}(Prescription_Medicine)) \cap \pi_{\text{Patient_SSN}}(Prescription \bowtie \sigma_{\text{TradeName} = \textbf{Vitamin}}(Prescription_Medicine))$ — different prescriptions, possibly different dates.
3. Comparing `P1.TradeName != P2.TradeName` but forgetting the prescription equality — returns patients with any two different medicines in any two prescriptions.

8.2 Twenty-two further questions on the same schema

Q8.6 **List the full names of all doctors along with their specialty.**
Answer.

$\pi_{\text{FirstName, LastName, Specialty}}(Doctor)$
(FirstName, LastName, Specialty).
Why. One relation, one projection; `SSN` is not requested, so it is dropped.

Trap / common mistake. Two doctors may share a name and specialty, in which case the projection collapses them. If the question wants one row per doctor, include `SSN` in the projection.

Q8.7 **List the trade names of all generic medicines.**
Answer.

$\pi_{\text{TradeName}}(\sigma_{\text{GenericFlag} = \textbf{Y}}(Medicine))$
(TradeName).
Why. `GenericFlag` is a stored attribute; the condition is tuple-local.

Trap / common mistake. If `GenericFlag` is stored as a Boolean rather than a character, the constant must match the domain: $\sigma_{\text{GenericFlag} = \text{true}}$. State the domain assumption.

Q8.8 **Find the names of patients who are older than 60.**
Answer.

$\pi_{\text{FirstName, LastName}}(\sigma_{\text{DOB} < \textbf{1966-10-22}}(Patient))$
(FirstName, LastName).
Why. "Older than 60" is a comparison between the stored `DOB` and a **constant date** computed from today's date. Relational algebra does not do arithmetic on dates, so the constant must be written out; say how you computed it.

Trap / common mistake. The expression $\sigma_{\text{DOB} < \text{today} - 60}(Patient)$ is not relational algebra: it contains date arithmetic. If the paper wants that, write it as a comment and give the constant form as the answer.

Q8.9 **Find the names of doctors who have prescribed at least one medicine.**
Answer.

$$\pi_{\text{D.FirstName, D.LastName}}\left(\rho_{\text{D}}(Doctor) \underset{\text{D.SSN} = \text{Doctor_SSN}}{\bowtie} Prescription\right)$$

(FirstName, LastName) of the doctor.
Why. Existential: a prescription row linking the doctor to any patient. Note the theta join on differently-named attributes.

Trap / common mistake. A doctor who has **seen** a patient as his primary doctor but never written a prescription must not appear. The witness relation is `Prescription`, not `Patient`.

Q8.10 **Find the names of patients who have never received a prescription.**
Answer.

$\pi_{\text{FirstName, LastName}}(Patient \bowtie (\pi_{\text{SSN}}(Patient) - \pi_{\text{Patient_SSN}}(Prescription)))$
Inner difference: (SSN); result: the patient's two name attributes.
Why. Pattern 2 with `Patient` as the universe.

Trap / common mistake. If the join is a natural join, the shared attributes are `SSN`, `FirstName` and `LastName`; here that is harmless because the right operand has only `SSN` — but in general, prefer the explicit key join.

Q8.11 Find the names of doctors who have prescribed for every patient.
Answer.

$$\pi_{\text{FirstName, LastName}}\left(\rho_{\text{D}}(Doctor) \underset{\text{D.SSN} = \text{Doctor_SSN}}{\bowtie} \left(\pi_{\text{Doctor_SSN, Patient_SSN}}(Prescription) \div \pi_{\text{SSN}}(Patient)\right)\right)$$

The division returns (Doctor_SSN); the join restores names.
Why. The dividend is $\pi_{\text{Doctor_SSN, Patient_SSN}}(Prescription)$ — the relation pairing doctors with the patients they have prescribed for. The divisor is every patient.

Trap / common mistake. Dividing `Prescription` itself gives (Doctor_SSN, Id, Date) as the retained attributes, which asks for a doctor who has a prescription with **every** patient **and** a fixed prescription id — impossible. Project first.

Q8.12 Find the trade names of medicines that have never been prescribed.
Answer.

$$\pi_{\text{TradeName}}(Medicine) - \pi_{\text{TradeName}}(Prescription_Medicine)$$

(TradeName).
Why. Universe = all medicines; witness = any `Prescription_Medicine` row.

Trap / common mistake. A medicine can exist in `Medicine` and never appear in any prescription — that is the point of the question; do not conclude the answer is always empty.

Q8.13 Find the names of patients whose primary doctor has specialty "Cardiology".
Answer.

$$\pi_{\text{FirstName, LastName}}\left(Patient \underset{\text{PrimaryDoctor_SSN} = \text{D.SSN}}{\bowtie} \sigma_{\text{D.Specialty} = \textbf{Cardiology}}(\rho_{\text{D}}(Doctor))\right)$$

The patient's (FirstName, LastName) — qualify the projection because `Doctor` also has them.

Trap / common mistake. Do not confound "primary doctor is a cardiologist" with "was prescribed by a cardiologist": the second uses `Prescription.Doctor_SSN`, the first uses `Patient.PrimaryDoctor_SSN`. Read which attribute is named.

Q8.14 Find the names of patients all of whose prescriptions were written by doctors with at least 10 years of experience.
Answer.

$$\text{stName}\left(Patient \bowtie \left(\pi_{\text{SSN}}(Patient) - \pi_{\text{P.SSN}}\left(Prescription \underset{\text{Patient_SSN} = \text{P.SSN}}{\bowtie} \rho_{\text{P}}(Patient) \underset{\text{Doctor_SSN} = \text{D.SSN}}{\bowtie} \sigma_{\text{D.YearsOfExperience} < 10}(\rho_{\text{D}}\right.\right.\right.$$

The subtracted term is the set of patient SSNs that have a **violating** prescription — written by a doctor with fewer than 10 years. The difference is (SSN), and the projection gives the names.
Why. "All of whose prescriptions satisfy C" is pattern 5 with the complement of C: patients having at least one prescription that fails C are removed. Patients with no prescriptions at all survive vacuously — say whether that is intended.

Trap / common mistake. Writing $\sigma_{\text{YearsOfExperience} \geq 10}$ and joining positively gives "has at least one prescription by an experienced doctor" — the opposite quantifier. The negation must be inside the subtracted term: **violators are removed**, not **satisfiers are kept**.

Q8.15 Find the names of doctors who have prescribed more than one medicine in a single prescription.
Answer. Let $R(\text{Multi})$ be the set of prescriptions that contain at least two distinct medicines:

$= \pi_{\text{P1.Prescription_Id}}\left(\sigma_{\text{P1.Prescription_Id} = \text{P2.Prescription_Id} \wedge \text{P1.TradeName} \neq \text{P2.TradeName}}(\rho_{\text{P1}}(Prescription_Medicine) \times \rho_{\text{P2}}(Prescription_M\right.$

Then

$$\pi_{\text{FirstName, LastName}}\left(\rho_{\text{D}}(Doctor) \underset{\text{D.SSN} = \text{Doctor_SSN}}{\bowtie} Prescription \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} R(\text{Multi})\right)$$

.
$R(\text{Multi})$: (Prescription_Id). The join chain returns the doctor's two name attributes.
Why. "More than one medicine in one prescription" is **at least two distinct trade names sharing a prescription id** — a self-join with equality on the prescription and inequality on the medicine. No counting is needed, because **two** is a fixed numeral.

Trap / common mistake. Dropping the equality on `Prescription_Id` answers "has prescribed two different medicines at some time", a much weaker condition. Dropping the inequality on `TradeName` answers nothing at all: every prescription trivially pairs with itself.

Q8.16 **Find the names of patients who have received the most expensive medicine (by UnitPrice).**
Answer. Let $R(\text{MaxP}) = \pi_{\text{UnitPrice}}(Medicine) - \pi_{\text{M1.UnitPrice}}(\sigma_{\text{M1.UnitPrice} < \text{M2.UnitPrice}}(\rho_{\text{M1}}(Medicine) \times \rho_{\text{M2}}(Medicine)))$ be the maximum unit price, and $R(\text{Top}) = \pi_{\text{TradeName}}(Medicine \bowtie R(\text{MaxP}))$ the medicines at that price. Then

$$\pi_{\text{FirstName, LastName}}\left(Patient \underset{\text{SSN} = \text{Patient_SSN}}{\bowtie} Prescription \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} Prescription_Medicine \bowtie R(\text{Top})\right)$$

.
$R(\text{MaxP})$: (UnitPrice). $R(\text{Top})$: (TradeName). Result: the patient's names.
Why. Maximum-of-values (Q5.1), then join through the prescription chain to the patient. Ties are preserved: every medicine at the maximum price is in $R(\text{Top})$, and every patient who received any of them is returned.

Trap / common mistake. The tempting "most expensive medicine **prescribed**" is the same query — the price is attached to the medicine, not to the line item. If prices could differ per prescription, the schema would need a price column on `Prescription_Medicine`. Say which model you are using.

Q8.17 **Find the names of patients who have been prescribed both Aspirin and Vitamin (not necessarily in one prescription), and contrast with Q8.5.**
Answer.

$$\text{atient_SSN}\left(Prescription \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} \sigma_{\text{TradeName} = \textbf{Aspirin}}(Prescription_Medicine)\right) \cap \pi_{\text{Patient_SSN}}\left(Prescription \underset{\text{Id} = \text{Prescription_Id}}{\bowtie}\right.$$

Both intersected operands are (Patient_SSN).
Why. Two independent existences, intersected on the patient key. This is **weaker** than Q8.5: any pair of prescriptions suffices.

Trap / common mistake. If you intersect on whole `Prescription` tuples instead of on the patient, you get the empty relation, because a single prescription cannot contain both medicines unless the self-join of Q8.5 is used. The projection before the intersection is what makes the two existences independent.

Q8.18 **Find the SSNs of patients whose every prescription contains at least one generic medicine.**
Answer.

$$\text{scription} - \pi_{\text{Pr.Id, Pr.Date, Pr.Doctor_SSN, Pr.Patient_SSN}}\left(\rho_{\text{Pr}}(Prescription) \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} Prescription_Medicine \underset{\text{TradeName} = \text{M.TradeNar}}{\bowtie}\right.$$

The inner subtraction is between two relations of schema (Id, Date, Doctor_SSN, Patient_SSN); the result is (Patient_SSN); the outer difference is (SSN).

Why. "Every prescription of this patient has a generic item" = all patients minus patients having a prescription with **no** generic item. The prescription with no generic item is found by subtracting from `Prescription` the prescriptions that do have one. The projection in the middle is essential to restore the exact schema of `Prescription` before the subtraction.

Trap / common mistake. This is the "preserve the full schema when subtracting records" rule of §1.7 in action. If you project only `Id`, the subtraction is on ids, and the outer difference then subtracts `Ids` from `SSNs` — a type error that a careless script will not notice because both are single-attribute relations of the same domain type.

Q8.19 Find the names of doctors who have never prescribed for patient "Ravi Kumar".

Answer.

$$\text{me, LastName}\Big(Doctor \bowtie \Big(\pi_{\text{SSN}}(Doctor) - \pi_{\text{Doctor_SSN}}\Big(Prescription \underset{\text{Patient_SSN} = \text{P.SSN}}{\bowtie} \sigma_{\text{P.FirstName} = \textbf{Ravi} \wedge \text{P.LastName} = \textbf{Kumar}}(\rho_{\text{P}}(Patie$$

Inner witness: (Doctor_SSN). Difference: (SSN). Result: the doctor's two name attributes.

Why. Pattern 2. The universe is all doctors; the witness is "has a prescription whose patient is Ravi Kumar". Only the prescribing relation counts — a doctor who merely has Ravi as his **primary** patient has not prescribed for him.

Trap / common mistake. If several patients are called Ravi Kumar, all of them count, because name is not a key. If the question expects a single patient, it should give the SSN; say so in your answer.

Q8.20 Find the trade names of medicines whose unit price is higher than the price of every generic medicine.

Answer. Let $R(\text{GMax}) = \pi_{\text{UnitPrice}}\big(\sigma_{\text{GenericFlag} = \textbf{Y}}(Medicine)\big) - \pi_{\text{M1.UnitPrice}}\big(\sigma_{\text{M1.UnitPrice} < \text{M2.UnitPrice}}\big(\rho_{\text{M1}}\big(\sigma_{\text{GenericFlag} = \textbf{Y}}(Medicine)\big) \times \rho_{\text{M2}}\big(\sigma_{\text{GenericFlag} = \textbf{Y}}(Medicine)\big)\big)\big)$. Then

$$\pi_{\text{TradeName}}(Medicine \bowtie \sigma_{\text{UnitPrice} > \text{GMax.UnitPrice}}(Medicine \times \rho_{\text{GMax}}(R(\text{GMax}))))$$

.

R(GMax): (UnitPrice). Result: (TradeName).

Why. "Higher than every generic" is a comparison against a **single computed value** — the maximum generic price — because "higher than all of them" is equivalent to "higher than the largest of them". Computing the maximum first is simpler than the counter-example method.

When the comparison set is defined by a condition rather than named, the **maximum** is the right reduction: $\forall g \in G : x > g \quad \text{iff} \quad x > \max(G)$, valid whenever G is non-empty. For the empty case, "higher than every generic" is vacuously true and the counter-example form handles it correctly, while the maximum form would return the empty relation. Prefer the counter-example form if the empty case matters.

Trap / common mistake. Using $\sigma_{\text{GenericFlag} \neq \textbf{Y}}$ to find non-generic medicines and comparing pairwise answers "higher than some non-generic", not "higher than every generic".

Q8.21 Find the pairs of doctors who have prescribed exactly the same set of medicines.

Answer. Let $R(\text{DM}) = \pi_{\text{Doctor_SSN, TradeName}}\Big(Prescription \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} Prescription_Medicine\Big)$. Define $R(\text{Pairs}) = \pi_{\text{SSN}}(Doctor) \times \pi_{\text{SSN}}(Doctor)$ and $R(\text{Missing}) = \pi_{\text{A.sid, B.sid}}\big(\sigma_{\text{X.sid} = \text{A.sid} \wedge \text{X.sid} \neq \text{B.sid} \wedge \text{X.TradeName} = \text{Y.TradeName}}(\ldots)\big)$ — see the construction below. The clean statement is the pair of divisions: (d_1, d_2) such that d_1 covers d_2 and d_2 covers d_1.

Answer. Construction. Let $R(\text{Pairs}) = \rho_{\text{A}}(Doctor) \times_{\ldots} \ldots$ — written with the three-variable device of Q7.33:

$$\text{V}) = \pi_{\text{X.d, X.m, Y.d}}(\rho_{\text{X}}(R(\text{DM})) \times \rho_{\text{Yd}}(\pi_{\text{SSN}}(Doctor))) - \pi_{\text{X.d, X.m, Yd.SSN}}(\sigma_{\text{X.m} = \text{Z.m}}(\rho_{\text{X}}(R(\text{DM})) \times \rho_{\text{Z}}(R(\text{DM})) \times \rho_{\text{Yd}}(\pi_{\text{SSN}}(Docto$$

,

$$R(\text{Bad}) = \pi_{\text{X.d, Y.d}}(R(\text{V}))$$

, and the answer is

$$R(\text{Pairs}) - R(\text{Bad}) - \rho_{\text{X.d} \leftarrow \text{Y.d, Y.d} \leftarrow \text{X.d}}(R(\text{Bad}))$$

.

Every intermediate is a pair of doctor identifiers, (SSN, SSN).

Why. "Same set of medicines" is the two-directional coverage condition — identical in structure to Q7.33 with members replaced by doctors and books by medicines.

Trap / common mistake. Do not attempt to compare `(Doctor_SSN, TradeName)` pairs directly with set equality; relations are not first-class values that can be compared. The two-directional difference is the only classical formulation.

Q8.22 **Find the names of patients who have prescriptions from at least three different doctors.**

Answer.

nt_SSN = B.Patient_SSN ∧ B.Patient_SSN = C.Patient_SSN ∧ A.Doctor_SSN ≠ B.Doctor_SSN ∧ B.Doctor_SSN ≠ C.Doctor_SSN ∧ A.Doctor_SSN ≠ C.Doctor_SSN$(\rho_{\text{A}}(Pre$

Three-way self-product: 12 attributes; after σ and π: (Patient_SSN).

Why. "At least three distinct" needs three witnesses — three tuple variables with pairwise inequalities. This is the fixed-cardinality pattern (§10.6).

Trap / common mistake. Two variables cannot express "three distinct doctors": with only A and B you can force $A \neq B$, but a third doctor is invisible. The number of variables must be at least the numeral.

Q8.23 **Find the names of doctors whose every prescription had at least two medicines in it.**

Answer.

$\pi_{\text{W.Doctor_SSN}}\big(Prescription - \pi_{\text{Id, Date, Doctor_SSN, Patient_SSN}}\big(\sigma_{\text{P1.Prescription_Id = P2.Prescription_Id} \wedge \text{P1.TradeName} \neq \text{P2.TradeName}}(Prescriptio$

The subtracted term is a set of prescriptions with at least two items, projected to the prescription schema; the outer subtraction gives doctors with a **single-item** prescription; the final difference gives doctors with none.

Why. Two nested instances of pattern 5: "every prescription of this doctor" and "at least two items per prescription".

Trap / common mistake. The projection inside must restore the exact `Prescription` schema, not just `Id` — if you subtract on `Id` you get a set of ids, and subtracting that from `Prescription` (4 attributes) is a union-compatibility error.

Q8.24 **Find the most recent prescription date for each doctor.**

Answer.

$scription - \pi_{\text{P1.Id, P1.Date, P1.Doctor_SSN, P1.Patient_SSN}}(\sigma_{\text{P1.Doctor_SSN = P2.Doctor_SSN} \wedge \text{P1.Date} < \text{P2.Date}}(\rho_{\text{P1}}(Prescription) \times \rho_{\text{P2}}(Prescript$

Schema of `Prescription`.

Why. Group-wise maximum of `Date` within each `Doctor_SSN` (Q5.13).

Trap / common mistake. Using the global maximum (without the `Doctor_SSN` equality) returns only the single most recent prescription in the hospital. The equality inside the σ is what makes it per-doctor.

Q8.25 **Find the names of patients who have received every medicine that the patient with SSN "123" has received.**

Answer. Let $R(\mathrm{PT}) = \pi_{\text{Patient_SSN},\ \text{TradeName}}\Big(\mathit{Prescription} \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} \mathit{Prescription_Medicine}\Big)$ be the "patient received medicine" relation, and let

$$R(\mathrm{P123}) = \pi_{\text{TradeName}}\big(\sigma_{\text{Patient_SSN} = \mathbf{123}}\big(\rho_{\mathrm{Q}}(R(\mathrm{PT}))\big)\big)$$

. Then the answer is

$$\pi_{\text{FirstName},\ \text{LastName}}\Big(\mathit{Patient} \underset{\text{SSN} = \text{Patient_SSN}}{\bowtie} (R(\mathrm{PT}) \div R(\mathrm{P123}))\Big)$$

$R(\mathrm{PT})$: (Patient_SSN, TradeName). $R(\mathrm{P123})$: (TradeName). The division returns (Patient_SSN), and the natural join with `Patient` on `SSN`... careful: the division result's attribute is named `Patient_SSN`, which is **not** the name of `Patient.SSN`. The join must be written as a theta join $\underset{\text{Patient_SSN} = \text{SSN}}{\bowtie}$, or the division result must be renamed first with $\rho_{\text{Patient_SSN} \leftarrow \text{SSN}}$.

Why. Same shape as Q4.4 and Q7.22: the quantified set is computed through two joins, and the division is performed on the derived binary relation. Patient 123 himself appears in the answer, since he received all of his own medicines.

Trap / common mistake. Two errors are common. (i) A **natural** join between the division result and `Patient` finds no shared attribute name at all, degenerating into a Cartesian product — always check that the attribute names match, or rename. (ii) If patient 123 has received no medicine, the divisor is empty and every patient who received any medicine is returned (vacuous truth).

Q8.26 **Find the names of doctors who have prescribed every medicine in $R(\mathrm{ReqMed})(\mathrm{TradeName})$.**

Answer.

$$\text{ame, LastName}\Big(\rho_{\mathrm{D}}(\mathit{Doctor}) \underset{\text{D.SSN} = \text{Doctor_SSN}}{\bowtie} \Big(\pi_{\text{Doctor_SSN},\ \text{TradeName}}\Big(\mathit{Prescription} \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} \mathit{Prescription_Medicine}\Big) \div R(\mathrm{Req}$$

(FirstName, LastName) of doctors.

Why. The binary relation must be derived by joining `Prescription` and `Prescription_Medicine`, because `Doctor_SSN` and `TradeName` never appear in the same stored relation.

Trap / common mistake. Dividing `Prescription_Medicine` directly fails: it has no `Doctor_SSN`. The join is the point of the question.

Q8.27 **Explain, using this schema, which of the following are expressible and why: (a) patients with at least two prescriptions; (b) patients with more prescriptions than their primary doctor; (c) patients whose prescriptions were all written by their primary doctor; (d) patients with the most prescriptions.**

Answer.

(a)	**Yes**	self-join of `Prescription` on `Patient_SSN` with $\neq$ on `Id` (pattern 14)
(b)	**No**	compares two computed counts; needs γ
(c)	**Yes**	pattern 5 with the complement: subtract patients having a non-primary prescription (Q8.4)
(d)	**No**	maximum of a computed count; needs γ

Why. (a) and (c) are Boolean combinations of existence statements over a **fixed** number of tuple variables. (b) and (d) require building a number per entity and comparing numbers — that is aggregation, and it is exactly what the classical algebra lacks.

Trap / common mistake. (b) looks deceptively similar to (a). "More than X" where X is a **computed count** is never expressible in classical RA, even when the two counts are over the same relation.

Part 9 — Advanced mixed questions combining multiple techniques

What this part fixes. Real exam questions are rarely one pattern. They stack a quantifier on a join on a negation. Each solution here names the **patterns** it stacks (§1.14), states the intermediate schemas, and shows where a naive composition breaks. The schemas used are Sailors–Boats–Reserves and Hospital–Prescription.

9.1 Two quantifiers in one sentence

Q9.1 Find the names of sailors with rating above 8 who have reserved every green boat.
Answer.

$$\pi_{\text{sname}}\big(\sigma_{\text{rating}>8}(Sailors) \bowtie \big(\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{color}=\textbf{green}}(Boats))\big)\big)$$

The division returns (sid); the selection on `Sailors` returns the full sailor schema; the natural join is on `sid`; the result is (sname).
Why. Patterns stacked: 8 (selection) + 6 (division) + 1 (join). The selection restricts the **answer**, not the quantification, so it is applied to `Sailors` outside the division.

Trap / common mistake. Applying the rating filter inside the dividend — $\pi_{\text{sid, bid}}\big(\sigma_{\text{rating}>8}(Sailors) \bowtie Reserves\big) \div \ldots$ — also works **here** because the join preserves `sid` and `bid`, but it is fragile: if the filter mentioned an attribute that did not survive the join, it would silently change the meaning. The intersection form (division result) $\cap\ \pi_{\text{sid}}\big(\sigma_{\text{rating}>8}(Sailors)\big)$ is the safest.

Q9.2 Find the names of sailors who have reserved a boat that no sailor with rating 10 has reserved.
Answer. Let $R(\text{Taken}) = \pi_{\text{bid}}\big(Reserves \bowtie \pi_{\text{sid}}\big(\sigma_{\text{rating}=10}(Sailors)\big)\big)$ be the boats reserved by at least one rating-10 sailor, and $R(\text{Free}) = \pi_{\text{bid}}(Boats) - R(\text{Taken})$ the boats nobody with rating 10 has reserved. Then

$$\pi_{\text{sname}}(Sailors \bowtie \pi_{\text{sid}}(Reserves \bowtie R(\text{Free})))$$

.
$R(\text{Taken})$ and $R(\text{Free})$: (bid). The join with `Reserves` gives (sid). Result: (sname).
Why. Patterns stacked: 2 (difference) + 1 (join) + 8 (selection). The universal-looking phrase "no sailor with rating 10" is **not** a universal quantifier over sailors — it is a negative existential over the boat's reservations. Read the English: there is no rating-10 sailor who reserved **this** boat.

Trap / common mistake. $\pi_{\text{bid}}(Boats) - \pi_{\text{bid}}(Reserves)$ would mean "never reserved by anyone", a much stronger condition. And dropping the intermediate $R(\text{Taken})$ step — writing $\pi_{\text{bid}}(Boats) - \pi_{\text{bid}}(Reserves) \times \ldots$ — is not even well-typed. Build the set in two named steps.

Q9.3 Find the names of sailors who have reserved every boat that has ever been reserved by a sailor younger than 20, and who are themselves older than 40.
Answer. Let $R(\text{YoungBoats}) = \pi_{\text{bid}}\big(Reserves \bowtie \pi_{\text{sid}}\big(\sigma_{\text{age}<20}(Sailors)\big)\big)$. Then

$$\pi_{\text{sname}}\big(\sigma_{\text{age}>40}(Sailors) \bowtie \big(\pi_{\text{sid, bid}}(Reserves) \div R(\text{YoungBoats})\big)\big)$$

.
$R(\text{YoungBoats})$: (bid). Division: (sid). Result: (sname).
Why. The divisor is a **derived** set computed through two joins — the most common shape in advanced questions. The outer selection is independent of the division and is applied to `Sailors`.

Trap / common mistake. If $R(\text{YoungBoats})$ is empty (no sailor younger than 20 has reserved anything), the division returns every sailor in the dividend's projection — and the outer selection then returns every sailor older than 40 who has made any reservation. Vacuous truth; state it.

9.2 Negation inside a universal

Q9.4 Find the names of sailors who have reserved only boats that have been reserved by sailor 22.
Answer.

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Reserves) - \pi_{\text{sid}}(Reserves \bowtie (\pi_{\text{bid}}(Boats) - \pi_{\text{bid}}(\sigma_{\text{sid}=22}(Reserves))))))$$

Innermost difference: (bid) — boats never reserved by 22. Next: (sid) — sailors with a reservation of such a boat. Outer difference: (sid). Result: (sname).
Why. Pattern 5 ("only X") with X itself defined by a negative condition (pattern 2). The subtracted term must be **sailors who reserved a boat not in 22's set**, which requires the double difference.

Trap / common mistake. Writing $\pi_{\text{sid}}(Reserves) - \pi_{\text{sid}}(\sigma_{\text{sid}=22}(Reserves))$ is a set of sailors, not a set of boats — a type error that is easy to make because both are "sets of ids". Check the **attribute names**, not just the domains.

Q9.5 Find the names of sailors who have not reserved any boat that a rating-1 sailor has reserved.
Answer.

$$\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(Reserves \bowtie \pi_{\text{bid}}(Reserves \bowtie \pi_{\text{sid}}(\sigma_{\text{rating}=1}(Sailors))))))$$

Innermost: (bid) of rating-1 sailors. Middle: (sid) — sailors who reserved any such boat. Outer difference: (sid).
Why. The "negative" is over a **set of boats**, and that set is itself computed by a join. The universe is all sailors, so sailors with no reservations are correctly included.

Trap / common mistake. The natural-join chain $Reserves \bowtie \pi_{\text{bid}}(\ldots)$ is on `bid`, which is correct; but a careless $Reserves \bowtie Reserves$ (unrenamed) is ambiguous. Rename when the same relation appears twice in one expression.

Q9.6 Find the ordered pairs of sailors (x, y) such that x has reserved every boat that y has reserved, and y has reserved at least one boat.
Answer. Let $R(\text{P}) = \pi_{\text{sid, bid}}(Reserves)$. The candidate pairs are $R(\text{Cand}) = \pi_{\text{x.sid, y.sid}}\big(\rho_{\text{x}}(R(\text{P})) \times \rho_{\text{y}}(R(\text{P}))\big)$. The pairs that **fail** are those for which some boat of y is missing from x: $R(\text{Miss}) = \pi_{\text{x.sid, y.sid}}\big(\rho_{\text{x}}(R(\text{P})) \times \rho_{\text{y}}(R(\text{P})) \times \rho_{\text{z}}(R(\text{P}))\big)$ restricted by $A(\text{z.sid}) = A(\text{y.sid})$, $A(\text{y.bid}) = A(\text{z.bid})$, $A(\text{x.bid}) \neq A(\text{z.bid})$. Then the answer is

$$R(\text{Cand}) - R(\text{Miss})$$

.
$R(\text{P})$: (sid, bid). $R(\text{Cand})$ and $R(\text{Miss})$: (x.sid, y.sid) — union compatible, so the difference is legal.
Why. "x covers y" is "¬∃ a boat of y that is not a boat of x". That negative existential over a **pair** of entities needs a third tuple variable ranging over the boat — the three-variable device of Q7.33. The projection of the missing-pair relation onto (x, y) is what eliminates the third variable.

The condition "y has reserved at least one boat" is automatic: $R(\text{Cand})$ was built from $R(\text{P})$, so a y with no boat never appears in it. Saying so earns the edge-case mark.

Trap / common mistake. Testing only the positive direction gives "x has some boat that y has", not "x covers y". The entire content of the query is in the subtracted term.

Q9.7 Find the sailor(s) with the highest rating among those who have reserved at least one red boat.
Answer. Let $R(\text{Red}) = \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color}\,=\,\textbf{red}}(Boats))$ and $R(\text{Cand}) = \sigma_{\text{rating}\,\geq 0}(Sailors) \bowtie R(\text{Red})$ — i.e. the sailors who qualify. Then apply the maximum-of-values pattern to $R(\text{Cand})$:

$$R(\text{Cand}) \bowtie \left(\pi_{\text{rating}}(R(\text{Cand})) - \pi_{\text{C1.rating}}\left(\sigma_{\text{C1.rating} < \text{C2.rating}}(\rho_{\text{C1}}(R(\text{Cand})) \times \rho_{\text{C2}}(R(\text{Cand})))\right)\right)$$

.
$R(\text{Red})$: (sid). $R(\text{Cand})$: schema of `Sailors`. Result: schema of `Sailors`.
Why. The selection and the existential are applied **first**, producing the candidate relation; the extreme is then computed **over the candidates**. This ordering is the whole point of "maximum after a selection" (Q5.12).

Trap / common mistake. Computing the global maximum and then intersecting with $R(\text{Red})$ answers a different question: "is the globally highest-rated sailor a red-boat reserver?" — usually empty. The extreme must be taken over the restricted relation.

Q9.8 Find the boat(s) reserved by the largest number of distinct sailors — or explain why you cannot.

Not expressible in classical relational algebra. "Largest number of distinct sailors" is a cardinality comparison. Not expressible.

Answer. Not expressible in classical relational algebra. With γ:

$$\gamma_{\text{bid; count(sid)} \leftarrow n}\left(\pi_{\text{bid, sid}}(\mathit{Reserves})\right)$$

, then the maximum of n, then a join back to `Boats` for the name.
Why. Note the difference from Q9.7: there the extreme was over a **stored attribute** (`rating`); here it is over a **computed count**. Stored values can be compared with $<$; counts cannot even be formed.

Trap / common mistake. A student may try "the boat that is not dominated by another boat", with domination meaning "has strictly more reserving sailors". That relation is not expressible, so the self-join has no predicate to write. If you catch yourself unable to state the σ condition, that is the signal.

Q9.9 Find the second-most-expensive medicine, and then the patients who received it.
Answer. Step 1: the maximum price is $R(\text{MaxP}) = \pi_{\text{UnitPrice}}(\mathit{Medicine}) - \pi_{\text{M1.UnitPrice}}(\sigma_{\text{M1.UnitPrice} < \text{M2.UnitPrice}}(\rho_{\text{M1}}(\mathit{Medicine}) \times \rho_{\text{M2}}(\mathit{Medicine})))$. Step 2: the second price is $R(\text{Second}) = \text{Max}(\pi_{\text{UnitPrice}}(\mathit{Medicine}) - R(\text{MaxP}))$, using Q5.1 on the reduced relation. Step 3: the medicines at that price are $\pi_{\text{TradeName}}(\mathit{Medicine} \bowtie R(\text{Second}))$. Step 4: the patients are

$$\text{irstName, LastName}\Big(\mathit{Patient} \underset{\text{SSN} = \text{Patient_SSN}}{\bowtie} \mathit{Prescription} \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} \mathit{Prescription_Medicine} \bowtie \pi_{\text{TradeName}}(\mathit{Medicine} \bowtie R(\text{Secon}$$

.
$R(\text{MaxP})$ and $R(\text{Second})$: (UnitPrice). Step 3: (TradeName). Step 4: the patient's two name attributes.
Why. Four patterns stacked: 8 (maximum), 10 (remove then maximum), 1 (join), 8 again (selection). Writing it as numbered steps is the safest exam style, and each step's schema is easy to check.

Trap / common mistake. "Second-most-expensive medicine" and "second-highest price" are the same only because price is attached to the medicine. If two medicines share the second price, **all** of them and all their patients are returned — ties are preserved by construction.

9.4 Queries about the absence of a relationship

Q9.10 Find the names of patients who have never been prescribed anything by their own primary doctor.

Answer. Let $R(\text{Rx}) = \pi_{\text{P.SSN}}\Big(\rho_{\text{P}}(Patient) \underset{\text{P.SSN} = \text{Patient_SSN}}{\bowtie} \sigma_{\text{Doctor_SSN} = \text{PrimaryDoctor_SSN}}(Prescription)\Big)$ be the set of patients who **have** been prescribed something by their own primary doctor. Then

$$\text{Answer} = \pi_{\text{FirstName, LastName}}\Big(Patient \underset{\text{SSN} = \text{S}}{\bowtie} \rho_{\text{S}}(\pi_{\text{SSN}}(Patient) - R(\text{Rx}))\Big)$$

$R(\text{Rx})$: (SSN) — note that the projection $\pi_{\text{P.SSN}}$ deliberately renames the result to `SSN` so that the difference is union compatible with $\pi_{\text{SSN}}(Patient)$. The difference also has schema (SSN), and ρ_{S} gives it a name to join on. Result: the patient's two names.

Why. Pattern 2, with a **correlated** witness: after joining `Patient` into the witness relation first, both `Doctor_SSN` and `PrimaryDoctor_SSN` sit in the same tuple, so the correlation is one attribute equality inside a single σ. This "join the correlated relation in first" move is the key technique of Part 9.

Trap / common mistake. A common over-complication uses two copies of `Prescription` — one to test "has any prescription" and one to test "by the primary doctor". Only the second test is needed. The opposite error is subtler: projecting the witness to `Patient_SSN` instead of `SSN` breaks union compatibility with $\pi_{\text{SSN}}(Patient)$, and some systems will accept it silently because both are single attributes over the same domain. Check the **names**.

Q9.11 Find the names of doctors who have prescribed for at least one patient who is not their own primary patient.

Answer.

$$\pi_{\text{D.FirstName, D.LastName}}\Big(\rho_{\text{D}}(Doctor) \underset{\text{D.SSN} = \text{Doctor_SSN}}{\bowtie} \sigma_{\text{PrimaryDoctor_SSN} \neq \text{Doctor_SSN}}\Big(Prescription \underset{\text{Patient_SSN} = \text{P.SSN}}{\bowtie} \rho_{\text{P}}(Patient)\Big)\Big)$$

The inner join of `Prescription` with `Patient` has schema (Id, Date, Doctor_SSN, Patient_SSN, FirstName, LastName, Address, DOB, PrimaryDoctor_SSN) — note that `Patient_SSN` and `SSN` are different names, so the theta join keeps both. After the σ, the tuple contains both the prescribing doctor and the patient's primary doctor, so the comparison is a single conjunct. Result: the doctor's two names.

Why. Existential over prescriptions, with a correlation between the prescription's doctor and the patient's primary doctor. Because the correlation attribute was brought into the same tuple by the join, no nesting is needed.

Trap / common mistake. This is **not** the complement of Q8.4. Q8.4 asks whether **all** of a patient's prescriptions come from the primary doctor; this asks whether **one** prescription does not. Different quantifiers, different answer sets. Examiners deliberately put the two side by side.

Q9.12 Find the names of patients who have received exactly one medicine (one distinct trade name), across all their prescriptions.

Answer.

$$\Big(\sigma_{\text{Q.Patient_SSN} = \text{R.Patient_SSN} \wedge \text{Q.TradeName} \neq \text{R.TradeName}}\Big(\rho_{\text{Q}}\Big(\pi_{\text{Patient_SSN, TradeName}}\Big(Prescription \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} Prescription_Medi$$

Then

$$\pi_{\text{FirstName, LastName}}\Big(Patient \underset{\text{SSN} = \text{Patient_SSN}}{\bowtie} R(\text{One})\Big)$$

.

$R(\text{One})$: (Patient_SSN). Result: patient names.

Why. "Exactly one" is "at least one" minus "at least two" (pattern 13). "At least two" requires two distinct trade names for the same patient — a self-join with equality on the patient and inequality on the medicine.

Trap / common mistake. If "exactly one medicine" is meant **per prescription** rather than across all prescriptions, the projection must keep `Prescription_Id` and the universe becomes pairs (patient, prescription). Read which noun "one" counts.

Q9.13 Find the sailors who have made exactly as many reservations as sailor 22 — or explain why this is not expressible.

Not expressible in classical relational algebra. Comparing two cardinalities needs counting. Not expressible.

Answer. Not expressible in classical relational algebra. With γ:

$$\gamma_{\text{sid; count}()\leftarrow n}(Reserves)$$

, then restrict to $A(\text{sid}) = 22$ to get n_0, then select the sids with $n = n_0$.
Why. The comparison is between two **computed numbers**. Even though the word "exactly" appears, it is not the fixed-numeral "exactly" of Q5.17 — the right-hand side is not a literal but a count that must be evaluated.

Trap / common mistake. The fixed-numeral special case **is** expressible: "sailors with exactly as many reservations as 22" stops being expressible only because 22's count is data-dependent. If the question said "sailors with exactly three reservations", it would be expressible (three witnesses). This contrast is the best short answer to "what does classical RA lack?"

Q9.14 Find the names of sailors who have reserved at least two boats that are red and at least one boat that is green.
Answer. Let $R(\text{Red2}) = \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats))$ and $R(\text{G1}) = \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} = \textbf{green}}(Boats)\big)$. The "at least two distinct red boats" condition is

$$R(\text{TwoRed}) = \pi_{\text{R1.sid}}\Big(\sigma_{\text{R1.sid} = \text{R2.sid} \wedge \text{R1.bid} \neq \text{R2.bid}}(\rho_{\text{R1}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats)) \times \rho_{\text{R2}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats)))\Big)$$

. Then the answer is

$$\pi_{\text{sname}}(Sailors \bowtie (R(\text{TwoRed}) \cap R(\text{G1})))$$

.
$R(\text{TwoRed})$ and $R(\text{G1})$: (sid). Result: (sname).
Why. Two different cardinality shapes — "at least two of X" (self-join) and "at least one of Y" (join) — combined by intersection on the key. This is the standard way to combine conditions with different quantifiers: compute each as a key set, then use a set operator.

Trap / common mistake. "At least two red **boats**" is not "at least two red **reservations**": a sailor who reserved the same red boat on two days has two reservations but one red boat. Projecting to `(sid, bid)` before the self-join is the fix, exactly as in Q3.8.

Part 10 — Query equivalence, incorrect expressions, counterexamples and exam traps

What this part fixes. The questions that carry the highest marks per line: "are these two expressions equivalent?", "find the error", "simplify", and "explain the difference between these two English sentences". §10.2 collects the eleven confusable pairs the brief asks for, each with the two expressions side by side.

10.1 Equivalence: the laws worth knowing

Two expressions are **equivalent** if they return the same relation on **every** instance. Counter-examples, not intuition, settle non-equivalence.

Law	Statement	Holds?
Cascade of selection	$\sigma_{\theta_1}\big(\sigma_{\theta_2}(R)\big) = \sigma_{\theta_1 \wedge \theta_2}(R)$	always
Commutativity of selection	$\sigma_{\theta_1}\big(\sigma_{\theta_2}(R)\big) = \sigma_{\theta_2}\big(\sigma_{\theta_1}(R)\big)$	always
Selection over union	$\sigma_\theta(R \cup S) = \sigma_{\theta(R)} \cup \sigma_{\theta(S)}$	always
Selection over difference	$\sigma_\theta(R - S) = \sigma_{\theta(R)} - \sigma_{\theta(S)}$	always
Selection over join	$\sigma_\theta(R \bowtie S)$ — cannot always be pushed into one side	only if θ mentions only that side
Projection cascade	$\pi_{L_1}\big(\pi_{L_2}(R)\big) = \pi_{L_1}(R)$ if $L_1 \subseteq L_2$	always
Projection and selection	$\pi_{L\left(\sigma_{\theta(R)}\right)} = \pi_{L\left(\sigma_{\theta\left(\pi_{L \cup \operatorname{attr}(\theta)}(R)\right)}\right)}$	always
Union: commutative, associative	$R \cup S = S \cup R$	always
Intersection: commutative, associative	$R \cap S = S \cap R$	always
Difference: neither	$R - S \neq S - R$	only when $R = S$
Difference is not associative	$(R - S) - T \neq R - (S - T)$	fails when R and T overlap
De Morgan (set form)	$R - (S \cup T) = (R - S) \cap (R - T)$	always
De Morgan (set form)	$R - (S \cap T) = (R - S) \cup (R - T)$	always
Join: commutative (natural)	$R \bowtie S = S \bowtie R$	always
Join: associative (natural)	$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$ when schemas allow	always for natural join
Division and union	$R \div (S_1 \cup S_2) \neq (R \div S_1) \cup (R \div S_2)$	wrong in general — see Q4.15
Division and intersection	$R \div (S_1 \cap S_2) = (R \div S_1) \cup (R \div S_2)$	true

10.2 The eleven pairs of confusable sentences

Each row gives the two English sentences, the two expressions, and the instance on which they differ. This is the section examiners lift questions from.

Pair 1 — "reserved a red boat" versus "reserved only red boats"

Sentence	Expression	Difference
Reserved a red boat	$\pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats))$	A sailor with one red and one blue reservation **qualifies**.
Reserved only red boats	$\pi_{\text{sname}}\Big(Sailors \bowtie \big(\pi_{\text{sid}}(Reserves) - \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} \neq \textbf{red}}(Boats)\big)\big)\Big)$	The same sailor **fails**, because of the blue one.

The quantifier changes from $\exists$ to $\forall$; the second expression is the first one **intersected with the absence of counter-examples**.

Pair 2 — "a red and a green boat" versus "both colours in one reservation"

Sentence	Expression	Difference
Reserved a red boat and a green boat	$\pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats)) \cap \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} = \textbf{green}}(Boats)\big)$	Two **different** reservations are fine; a sailor who reserved red in March and green in July qualifies.
Both colours in the same reservation	not meaningful in the 3-attribute `Reserves` schema (Q6.32); expressible only with a `Reservation`/ `Reservation_Item` schema	The standard schema cannot hold two boats in one reservation, so the honest answer names the schema limitation. On the "same day" reading, add $A(\text{R1.day}) = A(\text{R2.day})$.

Pair 3 — "has all required books" versus "has at least one required book"

Sentence	Expression	Difference
Has all required books	$\pi_{\text{MemberID, BookID}}(Borrow) \div R(\text{Required})$	Fails if even one is missing.
Has at least one required book	$\pi_{\text{MemberID}}(Borrow \bowtie R(\text{Required}))$	Succeeds with a single one.

Never "simplify" the first into the second. The division is the $\forall$; the join is the $\exists$.

Pair 4 — "not a primary doctor for any patient" versus "has never treated a patient"

Sentence	Expression	Difference
Not the primary doctor of any patient	$\pi_{\text{SSN}}(Doctor) - \pi_{\text{PrimaryDoctor_SSN}}(Patient)$	Uses the `Patient.PrimaryDoctor_SSN` column only.
Never treated any patient	$\pi_{\text{SSN}}(Doctor) - \pi_{\text{Doctor_SSN}}(Prescription)$	Uses the `Prescription` relation only.

A doctor who has written fifty prescriptions but is nobody's primary doctor satisfies the first and not the second; a primary doctor who has never written a prescription satisfies the second and not the first. Both directions occur in real data, which is why this pair is examinable.

Pair 5 — "no prescriptions by other doctors" versus "no prescriptions at all"

Sentence	Expression	Difference
No prescription by a doctor other than my primary doctor	pattern of Q8.4: all patients minus patients having a non-primary prescription	A patient with **zero** prescriptions **qualifies** — vacuously true.
Has no prescriptions at all	$\pi_{\text{SSN}}(Patient) - \pi_{\text{Patient_SSN}}(Prescription)$	Strictly the empty-

		prescription patients only.

The first is "A ∪ B" in the case table of Q8.4; the second is "A" alone. Write the case table in the exam — it is usually worth a mark by itself.

Pair 6 — "medicine occurs in more than 20 prescriptions" versus "total units exceed 20"

Sentence	Quantity compared	Expressible?
Occurs in more than 20 prescriptions	$\text{count}(\text{Prescription_Id})$ — a **row** count	No. Needs γ.
Total number of units exceeds 20	$\text{sum}(\text{NumOfUnits})$ — a **value** sum	No. Needs γ.

Two different aggregates, two different answers on the same data. Aspirin in 3 prescriptions of 100 units each: count 3, sum 300. Vitamin in 1 prescription of 300 units: count 1, sum 300. The same total, a completely different count.

Pair 7 — "second-highest distinct cost" versus "second record in descending order"

Sentence	Meaning	Expressible?
Second-highest distinct cost	the second element of $\pi_{\text{cost}}(R)$ after removing the maximum (Q5.5)	Yes
Second record when sorted by cost descending	a **position** in an ordering of tuples	No — relations have no order, and ties make the position undefined

With costs 100, 100, 90: the second-highest distinct cost is 90; the second record is 100 (if ties are listed) or 90 (if ties collapse) — the phrase itself is ambiguous. Only the distinct-value reading is well defined in classical RA.

Pair 8 — "maximum cost overall" versus "maximum cost per doctor"

Sentence	Expression	Difference
Overall maximum	$R(\text{T}) - \pi_{\text{T1.attr}}(\sigma_{\text{T1.attr} < \text{T2.attr}}(\rho_{\text{T1}}(R(\text{T})) \times \rho_{\text{T2}}(R(\text{T}))))$	No equality on a group attribute in the σ.
Maximum per doctor	$R(\text{T}) - \pi_{\text{T1.attr, T1.doc}}(\sigma_{\theta}(\rho_{\text{T1}}(R(\text{T})) \times \rho_{\text{T2}}(R(\text{T})))) \text{ with } \theta = A(\text{T1.doc}) = A(\text{T2.doc}) \wedge A(\text{T1.attr}) < A(\text{T2.attr})$	Two changes: the conjunct that ties the two tuples to the **same group**, and the projection, which must list the **full** schema of $R(\text{T})$ — here `attr` and `doc`, not `attr` alone.

Pair 9 — "at least two related records" versus "at least two distinct related entities"

Sentence	Expression	Difference
At least two reservations	self-join of `Reserves` with $\neq$ on `bid` or `day`	Counts **rows**.
At least two distinct boats	self-join of $\pi_{\text{sid, bid}}(Reserves)$ with $\neq$ on `bid`	Counts **entities**.

On a schema whose key is (sid, bid, day), the same boat reserved on two days gives two rows and one entity. Read the noun after "two".

Pair 10 — "every record satisfies a condition" versus "there is a record satisfying it"

Sentence	Expression	Difference
Every reservation is red	$\pi_{\text{sid}}(Reserves) - \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} \neq \text{red}}(Boats)\big)$	$\forall$: no counter-example
Has a red reservation	$\pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \text{red}}(Boats))$	$\exists$: one witness

The two use the **same** σ with the condition negated, and the second is not a subset of the first: a sailor with a red and a blue boat is in the second and not the first.

Pair 11 — "same set of books" versus "same number of books"

Sentence	Expressible?	Why
Borrowed exactly the same set of books	Yes — two-directional coverage (Q7.33)	It is a Boolean combination of universals
Borrowed the same **number** of books	No	It compares two cardinalities — needs γ

The word "same" is innocent-looking and hides either a set equality (expressible) or a cardinality equality (not). This is the single most reliable discriminator question in the syllabus.

10.3 Error detection: eighteen expressions, each wrong in exactly one way

Q10.1 $\pi_{\text{sname}}(Reserves)$ **— find the error.**
Answer. `Reserves` has schema (sid, bid, day); it has no `sname`. The projection names an attribute outside the operand's schema — a type error.

> **Trap / common mistake.** The repair depends on the intent: to get names of sailors who reserved **something**, join with `Sailors` first.

Q10.2 $\sigma_{\text{rating} > 7}(Sailors)$ **— find the error.**
Answer. `rating` is numeric and `7` is a string. Domain mismatch inside a comparison.

Q10.3 $\pi_{\text{sid, sname}}(Sailors) - \pi_{\text{sid}}(Reserves)$ **— find the error.**
Answer. Not union compatible: two attributes against one. Project both sides to (sid), or add `sname` to both sides by first joining.

Q10.4 $Sailors \times Sailors$ **— find the error.**
Answer. Ambiguous attribute names: the product would have two attributes called `sid`, and the definition of the product requires disjoint attribute sets. Rename at least one copy with ρ.

Q10.5 $Reserves \div \pi_{\text{bid}}(Boats)$ **— find the error.**
Answer. Not a type error — it is well-typed and **returns the wrong thing**: schema (sid, day), meaning "sailors who reserved every boat on a single day". Project to (sid, bid) first.

Q10.6 $\sigma_{\text{color} = \text{red}}(\pi_{\text{bid}}(Boats))$ **— find the error.**
Answer. `color` has been projected away. Select **before** projecting.

Q10.7 $\pi_{\text{sname}}(Sailors) - \pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \text{red}}(Boats))$ **— find the error.**
Answer. Not a type error, but semantically wrong: `sname` is not a key. Two different sailors called Horatio cancel each other. Subtract on `sid`.

Q10.8 $\pi_{\text{sid}}(\sigma_{\text{bid}=101 \wedge \text{bid}=103}(Reserves))$ — **find the error.**
Answer. One tuple cannot have two values for `bid`; the result is always empty. Use two tuple variables, or an intersection of two projections.

Q10.9 $\sigma_{\text{S1.rating} < \text{S2.rating}}(Sailors)$ — **find the error.**
Answer. `S1` and `S2` are not bound: no ρ has introduced them, and a selection cannot compare two tuples of the same relation. The self-product with two renames is missing.

Q10.10 $\pi_{\text{sid}}(Sailors \bowtie Reserves) - \pi_{\text{sid}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \text{red}}(Boats))$ — **is this "sailors who never reserved a red boat"?**
Answer. Yes for the reservers, but **no** for sailors with no reservations: they are in neither operand, so they are not returned. The question's universe must be $\pi_{\text{sid}}(Sailors)$, not $\pi_{\text{sid}}(Sailors \bowtie Reserves)$.

Trap / common mistake. This is the single most common "looks right, is subtly wrong" expression in the whole subject. The visible symptom is that sailors with zero reservations silently disappear.

Q10.11 $\pi_{\text{sname}}(Sailors \bowtie Reserves)$ — **find the error, if any.**
Answer. No error. The natural join merges the shared `sid`; the projection is well-typed. It returns the names of sailors who made at least one reservation.

Trap / common mistake. Watch the difference from $\pi_{\text{sname}}(Sailors \bowtie Boats)$: `Sailors` and `Boats` share no attribute name, so that expression is a **Cartesian product** — every sailor paired with every boat. Legal, and almost never intended.

Q10.12 $\rho_{\text{X}}(Sailors) \cup Sailors$ — **find the error, if any.**
Answer. No error: ρ changes only the relation name, not the schema, so the operands are union compatible. The result takes the **left** operand's relation name, $R(\text{X})$.

Trap / common mistake. Renaming to a name already in scope — $\rho_{\text{Sailors}}(Sailors)$ used together with `Sailors` — creates genuine ambiguity. Use fresh names.

Q10.13 $\pi_{\text{rating}}(Sailors \bowtie Reserves) - \pi_{\text{rating}}(Sailors)$ — **what does this compute?**
Answer. Nothing meaningful: the left operand has no `rating` (the natural join does keep it, since `Reserves` has no `rating`) — so actually this **is** well-typed and computes "ratings of sailors who have reserved, minus all ratings", which is always empty, because every reserving sailor is a sailor. The expression is a disguised empty relation.
Why. A useful reminder that well-typedness and meaningfulness are different. If an expression looks suspiciously empty, check whether one side's set is a subset of the other's.

Q10.14 $\sigma_{\text{age}>30}(Sailors) - \sigma_{\text{rating}>5}(Sailors)$ — **find the error.**
Answer. No type error (both operands are `Sailors`), and the meaning is "sailors older than 30 who do not have a rating above 5" — a legitimate but rarely intended query. The **likely** intent was a conjunction, which is one selection with $\wedge$.

Trap / common mistake. Set difference of two selections on the same relation is $\sigma_{\theta_1 \wedge \neg\theta_2}$, never $\sigma_{\theta_1 \wedge \theta_2}$. Adding conditions is $\wedge$ inside one σ; subtracting sets is a different operation.

Q10.15 $\pi_{\text{sid}}(Reserves \div \pi_{\text{bid}}(Boats))$ — **find the error.**
Answer. The division already returns (sid), so the outer projection is redundant but legal. The real trap is the inner one: `Reserves div pi_(bid)(Boats)` has schema (sid, day), so the projection returns `sids` of sailors who reserved all boats **on one day**, which is the wrong query (Q4.13).

Q10.16 $\gamma_{\text{bid; count(sid)}}\big(\pi_{\text{bid, sid}}(Reserves)\big)$ **— is this relational algebra?**
Answer. No. γ is the **extended** grouping/aggregation operator. It is not part of the classical algebra of $\pi, \sigma, \rho, \cup, -, \times$ (and the derived $\cap, \bowtie, \div$). In an exam that says "classical relational algebra only", writing γ is normally accepted only as a **supplementary** answer after stating that the query is not expressible classically.

Q10.17 **Is `SELECT sname FROM Sailors WHERE rating > 7` a relational algebra expression?**
Answer. No — it is SQL. Its relational-algebra equivalent is $\pi_{\text{sname}}\big(\sigma_{\text{rating} > 7}(Sailors)\big)$. The examiner asks for the algebra, not the SQL; translating SQL into algebra is a legitimate **first step**, but the SQL itself is not the answer.

Q10.18 $\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(Reserves) \cap \pi_{\text{sid}}\big(\sigma_{\text{age} < 20}(Sailors)\big)$ **— find both errors.**
Answer. (i) Precedence: in the absence of brackets $\cap$ binds tighter than $-$, so this parses as $\pi_{\text{sid}}(Sailors) - \big(\pi_{\text{sid}}(Reserves) \cap \pi_{\text{sid}}\big(\sigma_{\text{age} < 20}(Sailors)\big)\big)$ — probably not the intent. (ii) The phrase "$-$ before $\cap$" is genuinely ambiguous across textbooks, so always bracket.
Why. There is no universal precedence agreement; the safe rule is to parenthesise every compound expression. This is why every expression in this book is fully bracketed.

10.4 Simplification

Q10.19 **Simplify** $\pi_{\text{sname}}\big(\sigma_{\text{rating} > 7}\big(\sigma_{\text{age} < 40}(Sailors)\big)\big)$**.**
Answer.

$\pi_{\text{sname}}\big(\sigma_{\text{rating} > 7 \wedge \text{ age} < 40}(Sailors)\big)$
Why. Cascade of selection (§10.1). Both conditions are tuple-local to the same relation, so they combine into one σ.

Q10.20 **Simplify** $\sigma_{\text{rating} > 7}(Sailors \bowtie Reserves)$**.**
Answer.

$$\sigma_{\text{rating} > 7}(Sailors) \bowtie Reserves$$

, provided `rating` is in `Sailors` only — it is. Pushing the selection to the leaf reduces the intermediate relation and is always safe when the attribute survives the join.

Trap / common mistake. Pushing a selection **through** a projection is not always safe; pushing through a union, intersection or difference is safe; pushing through a division is not.

Q10.21 **Simplify** $(\pi_{\text{sid}}(Sailors) \cup \pi_{\text{sid}}(Sailors)) - \pi_{\text{sid}}(Reserves)$**.**
Answer.

$$\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(Reserves)$$

. Union is idempotent under set semantics: $R \cup R = R$.

Trap / common mistake. Under **bag** semantics (SQL's UNION ALL) this simplification is invalid. Classical relational algebra is set-based, so it holds. State the semantics you are assuming.

Q10.22 **Simplify** $\pi_{\text{sid}}\big(\pi_{\text{sid, bid}}(Reserves)\big)$**.**
Answer.

$$\pi_{\text{sid}}(Reserves)$$

. Projection cascade: a projection of a projection equals the projection onto the smaller attribute list, provided that list is a subset of the inner one.

Q10.23 **Is $\sigma_{\text{rating}>7}(Sailors) \cup \sigma_{\text{rating}\leq 7}(Sailors)$ simplifiable?**
Answer. Yes — to $Sailors$. The two selections partition the relation, and their union recovers it.
Why. This is the set analogue of the law of excluded middle; it is worth remembering as a way to **prove** that a partition-based expression is correct.

10.5 Why certain simplifications are unsound

Attempted simplification	Status	Counterexample or reason
$\sigma_{\theta(R \bowtie S)} = \sigma_{\theta(R)} \bowtie S$	safe iff θ mentions only R's attributes	if θ mentions S's attributes, the right side is a type error or a different filter
$\pi_{L(\sigma_{\theta(R)})} = \sigma_{\theta(\pi_{L(R)})}$	unsafe	θ may mention an attribute not in L
$\pi_{L(R \bowtie S)} = \pi_{L(R)} \bowtie \pi_{L(S)}$	unsafe in general	join attributes may be projected away, turning the join into a product
$\sigma_{\theta(R \div S)} = \sigma_{\theta(R)} \div S$	unsafe	θ may mention attributes that division removes
$(R \div S) \div T = R \div (S \times T)$	true when schemas allow	worth proving once; it is the associativity of division
$R \div (S_1 \cup S_2) = (R \div S_1) \cup (R \div S_2)$	**false**	Q4.15; dividing by a bigger set is harder, so the left side is smaller
$R \div (S_1 \cap S_2) = (R \div S_1) \cup (R \div S_2)$	true	both sides quantify "every element of the smaller set"; prove by cases
$R - (S - T) = (R - S) \cup (R \cap T)$	true	useful in error detection

Learn the four "**unsafe**" rows. They are the source of nearly every "is this simplification valid?" question.

10.6 Fixed cardinality is expressible; computed cardinality is not

> **The rule.** Let k be a **numeral written in the query**. Then "at least k", "at most k", "exactly k" are expressible, using k distinct tuple variables. If the bound is a **value that must be computed** (an attribute, a count of another set, a sum), the query is not expressible.

Worked example: "sailors with exactly two distinct boats". $R(\text{Two}) = \pi_{\text{R1.sid}}\Big(\sigma_{\text{R1.sid} = \text{R2.sid} \wedge \text{R1.bid} \neq \text{R2.bid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves))\Big)$ $R(\text{Three}) = \pi_{\text{R1.sid}}\Big(\sigma_{\text{R1.sid} = \text{R2.sid} \wedge \text{R2.sid} = \text{R3.sid} \wedge \text{R1.bid} \neq \text{R2.bid} \wedge \text{R2.bid} \neq \text{R3.bid} \wedge \text{R1.bid} \neq \text{R3.bid}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves) \times \rho_{\text{R3}}(Reserves))\Big)$

$$\text{Exactly2} = R(\text{Two}) - R(\text{Three})$$

Why the same trick fails for "more than 20". The expression would need twenty-one variables and, worse, its **shape** would change with the numeral. A query language is a fixed set of operators; "more than n" for arbitrary n requires a fixpoint or a counting operator. This is the precise sense in which classical relational algebra is weaker than SQL.

Q10.24 **Which is expressible: "sailors with at least two reservations", "sailors with more reservations than sailor 22", "sailors with exactly five reservations"?**
Answer. First: yes (two variables). Second: no (compares a computed count with another computed count — needs γ). Third: yes in principle (five variables), no in the syllabus convention, which treats any query whose structure depends on a numeral as "requires counting".
Why. The bright line is not the **word** "exactly" but whether the bound is a literal in the query text or a value that must be computed.

10.7 Queries that cannot be expressed: the full list

Category	Example	Why not
Counting	"patients with more than 20 prescriptions" (Q8.3)	cardinality is not a term
Summing	"total units prescribed" (Q8.3)	sum is not a term
Averaging	"department with the highest average salary" (Q5.9)	average is not a term
Maximum of a count	"most-borrowed book" (Q7.24)	needs counting first
Comparison of counts	"patients with more prescriptions than their doctor" (Q8.27b)	needs two counts
Ranking by position	"the second record in descending order" (§10.2 pair 7)	relations have no order
Deterministic choice among ties	"one of the oldest sailors"	requires an order or an arbitrary choice
Arithmetic on attribute values	"employees whose salary exceeds twice the department average"	arithmetic over aggregates
Transitive closure	"all employees who report to X, directly or indirectly"	needs a fixpoint; not first-order
Conditional/derived columns	"salary minus 10% as a new attribute"	no arithmetic in the output schema in classical RA

Every entry in this table has the same cure: an extended operator (γ for the first six, a fixpoint operator for the ninth, an extended projection for the last). Saying **which** extension is needed is worth as much as saying "not expressible".

10.8 A ten-point checklist for the exam hall

1. Did I write the **answer schema** first?
2. Is every $\cup$ between operands with the **same attribute list**?
3. Did I **rename** one copy of any relation that appears twice?
4. Is every attribute in a σ or π actually **in the operand's schema**?
5. Did I subtract on the **key**, not on a descriptive attribute?
6. Did I **project before dividing**, and is the divisor exactly the quantified set?
7. Are **ties** preserved (strict inequalities, joins rather than tuple subtraction)?
8. Did I handle the **empty** case: empty divisor, empty witness set, no records at all?
9. Is the query **expressible** at all — and if not, did I say which operator is missing?
10. Did I **bracket** every compound expression?

Part 11 — Practice paper — questions only

How to use it. Close the book. Time yourself. This paper is modelled on the MANIT Bhopal end-semester pattern (one compulsory section of short answers, plus a choice of long questions), but the **wording** is new — no question is copied verbatim from a previous paper. The answer key starts on the page referenced in Part 12.

Instructions

Paper	Relational Algebra and Relational Calculus — Section B
Time	3 hours
Maximum marks	80
Schemas	Sailors–Boats–Reserves, Library (full schema), Hospital–Prescription. All are as defined in Part 1.2 and restated at the head of each question.
Marks	Section I: 10 × 2 = 20. Section II: 4 × 5 = 20. Section III: 2 × 10 = 20 (internal choice). Section IV: 1 × 20 = 20 (case study, internal choice).
Rules	Classical relational algebra only. No SQL. No aggregate functions. State the schema of the answer, and of each named intermediate relation. Marks are awarded for the schema even when the expression is imperfect.

Section I — short answers (2 marks each, all compulsory)

> **A1** Define union compatibility. State precisely why it is required by $\cup$, $\cap$ and $-$ but not by $\times$.

> **A2** Give the relational algebra expression for "the names of sailors who have reserved boat 103", and state the schema of the intermediate relation after the join.

> **A3** Is $\pi_{\text{sname}}(Sailors) - \pi_{\text{sname}}\big(\sigma_{\text{rating}\,<8}(Sailors)\big)$ a correct way to find the names of sailors with rating at least 8? Justify your answer in two sentences.

> **A4** Write the division $Reserves \div \pi_{\text{bid}}(Boats)$ and state its result schema. Explain in one sentence what it computes, and why that is not "sailors who reserved all boats".

> **A5** Using the hospital schema, explain why `Patient` and `Doctor` cannot be joined with a natural join in the query "patients whose primary doctor is John Smith".

> **A6** State whether "the medicine prescribed in the largest total number of units" is expressible in classical relational algebra. Name the operator that would be required.

> **A7** Give two different expressions, one using $\div$ and one not, for "sailors who reserved every boat".

> **A8** What is the result of $\pi_{\text{color}}(Boats)$ when sixty boats are red and forty are blue? Explain in terms of set semantics.

> **A9** Write the expression for "the boat(s) most recently reserved" — or state why it needs a maximum over a date and give the expression for the latest date alone.

> **A10** Define the semijoin and the antijoin, say whether either is part of the classical algebra, and give the classical expression for each.

Section II — short problems (5 marks each, attempt any four)

> **B1** **(Sailors)** Find the names of sailors who have reserved a red boat but have never reserved a green boat. Give the expression, the schema at each step, and the edge case of a sailor with no reservations.

B2 **(Sailors)** Find the sids of sailors whose rating is the second-highest rating in the relation. Preserve ties in the rating value, and explain why "second-highest rating" and "rating of the second sailor" differ.

B3 **(Library)** Find the members who have borrowed every book written by any author whose name is "Navathe". State the dividend and divisor schemas explicitly.

B4 **(Hospital)** Find the names of patients who have received at least one medicine from every generic category, where `Medicine.GenericFlag = 'Y'` marks a generic medicine, and each such medicine is treated as its own category. Does the question even make sense? If not, repair it and answer the repaired question.

B5 **(Sailors)** Find the pairs of sailors who have reserved exactly the same set of boats. Give the expression in the two-directional form and explain why one direction alone is insufficient.

B6 **(Mixed)** A student writes $\pi_{\text{sname}}(\sigma_{\text{color} = \textbf{red}}(Sailors \bowtie Reserves \bowtie Boats))$. Identify every error, in the order in which an evaluator would encounter them, and write the corrected expression.

Section III — long answers (10 marks each, attempt any two)

C1 **(Sailors)** For each of the following, write the expression and say in one line which pattern (§1.14) you used. (a) sailors who have reserved all red boats; (b) sailors who have reserved no red boat; (c) sailors who have reserved only red boats; (d) sailors who have reserved at least one red boat and at least one green boat. Then explain, in a short paragraph, why (b) and (c) are not complements of each other.

C2 **(Hospital)** Using the hospital schema: (a) list the names of doctors who have prescribed for patients they are not the primary doctor of; (b) list the names of patients all of whose prescriptions were written by their primary doctor; (c) explain, with a case table, the difference between (b) and "patients with no prescriptions by other doctors". State which reading your expression implements.

C3 **(Library)** The librarian wants "the most borrowed book". (a) Show that this is not expressible in classical relational algebra and say exactly which operator is missing. (b) Write the extended-algebra expression that answers it. (c) Write the classical expression for the closely related query "the book borrowed by the largest number of **distinct** members" and say whether it is expressible. (d) Write the classical expression for "the book with the highest price" (assume a `Price` attribute on `Book`) and explain why **that** one is expressible.

C4 **(Sailors)** Prove or disprove each statement, with a counter-example where it fails. (a) $R \div (S_1 \cup S_2) = (R \div S_1) \cup (R \div S_2)$. (b) $\sigma_{\theta(R \bowtie S)} = \sigma_{\theta(R)} \bowtie \sigma_{\theta(S)}$. (c) $\pi_{\text{sid}}(Reserves) - \pi_{\text{sid}}(Sailors) = \varnothing$ for every instance of the sailors schema. (d) $R \cap S = R - (R - S)$.

Section IV — case study (20 marks, attempt any one)

D1 **Hospital, integrated.** A hospital quality report needs the following five answers. Write all five expressions, name each intermediate relation with its schema, and state for each whether it is expressible in classical relational algebra.

- The names of patients who have seen (i.e. have a prescription from) more than three distinct doctors, together with the number of such doctors — **the count is optional**; if it is not expressible, say so and give what is.
- The names of doctors who have prescribed at least one medicine to every patient for whom they are the primary doctor.

- The names of patients who have been prescribed at least one medicine that their primary doctor has never prescribed to anyone.
- The trade names of medicines that have been prescribed by every doctor who has prescribed anything.
- The names of the patient(s) whose most recent prescription is the most recent prescription in the hospital.

D2 **Sailors, integrated.** Answer the following, naming each intermediate relation and its schema, and flag any part that is not expressible.

- The names of sailors who have reserved every boat reserved by any sailor with a rating lower than their own.
- The sids of sailors who have reserved at least one boat on every date on which boat 103 was reserved.
- The boat(s) with the highest number of reservations — expressible or not?
- The names of sailors who have never reserved a boat that is older than their own rating — note that this question contains a domain error; identify it and answer the repaired question.
- Using only π, σ and $\times$, express "sailors who have reserved all boats".

Marking scheme, in one paragraph

Section I rewards precision: a correct expression with no schema statement typically loses one of the two marks. Section II rewards the **edge cases** — the empty divisor, the sailor with no reservations, the tie. Section III expects every intermediate relation to be named, and expects you to say which reading of an ambiguous sentence you are implementing. Section IV is where the "not expressible" discussion earns its marks: a bare "cannot be done" without the name of the missing operator scores at most a third of the part.

Diagnostic. If you can do Section I in 20 minutes and Section II in 40, your notation is sound. If Section III(C3) or Section IV(D1.1) still feels uncertain, re-read §1.11 and §10.6 — the **expressibility frontier** is where most students lose the largest single block of marks, because they answer a counting question with an extreme-value pattern.

Part 12 — Answer key, marking scheme and coverage checklist

Purpose. Compact answers to the practice paper (Part 11), followed by the coverage checklist that proves every category required by the brief has been covered, with a pointer to where. Use the checklist as a revision index, not as a list of topics to memorise separately.

12.1 Answer key — Section I

A1 R and S are union compatible when they have the same number of attributes and the i-th attribute of R has the same domain as the i-th of S, for every i; names need not match, and the result takes the names of the left operand. $\times$ needs no such condition because it produces **all** combinations and merges no attribute: its result schema is the union of the two attribute sets, and it is defined for any pair of relations with disjoint attribute names.

A2 $\pi_{\text{sname}}(Sailors \bowtie \sigma_{\text{bid}=103}(Reserves))$. Intermediate schema after the join: $(\text{sid}, \text{sname}, \text{rating}, \text{age}, \text{bid}, \text{day})$.

A3 No. `sname` is not a key: a sailor with rating 7 and the same name as a sailor with rating 9 appears on both sides and cancels. The correct form subtracts on `sid`.

A4 Result schema (sid, day) — "reserved every boat on one single day". It is not "sailors who reserved all boats" because `day` is retained: the quantification is per day.

A5 `Patient` and `Doctor` share `SSN`, `FirstName` and `LastName`. A natural join equates all three, i.e. it looks for patients who **are** the doctor. The intended join is `Patient.PrimaryDoctor_SSN = Doctor.SSN`, an equijoin on differently-named attributes.

A6 Not expressible. It requires the aggregation operator γ with `sum`.

A7 With division: $\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(Boats)$. Without: $\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}\big(\pi_{\text{sid}}(Sailors) \times \pi_{\text{bid}}(Boats) - \pi_{\text{sid, bid}}(Reserves)\big)$.

A8 Two rows: one for `red`, one for `blue`. π returns a **set**, so the 60 red boats collapse to a single `red` tuple.

A9 The latest date is $\pi_{\text{day}}(Reserves) - \pi_{\text{R1.day}}\big(\sigma_{\text{R1.day} < \text{R2.day}}(\rho_{\text{R1}}(Reserves) \times \rho_{\text{R2}}(Reserves))\big)$. The boat(s) associated with it are found by joining that value back to `Reserves`; if several boats share the latest date, all are returned.

A10 Semijoin $R \ltimes S = \pi_{\text{attr}(R)}(R \bowtie S)$; antijoin $R \leftarrow S = R - \pi_{\text{attr}(R)}(R \bowtie S)$. Neither is classical; both are shorthands to be expanded in a classical-RA answer.

12.2 Answer key — Section II

B1

$$\pi_{\text{sname}}\big(Sailors \bowtie \big(\pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats)) - \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} = \textbf{green}}(Boats)\big)\big)\big)$$

. Steps: `Red` $= \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats))$, schema (sid); `Green` likewise; difference (sid); natural join with `Sailors` on `sid`; result (sname). A sailor with no reservations is in neither `Red` nor `Green`, so he is **not** returned — which is correct for "reserved a red boat but not a green one".

B2 Let `M` be the maximum rating (Q5.1) and `Rest` $= \pi_{\text{rating}}(Sailors) -$ M. The second-highest is the maximum of `Rest`, i.e. Rest $- \pi_{\text{R1.rating}}\big(\sigma_{\text{R1.rating} < \text{R2.rating}}(\rho_{\text{R1}}(\text{Rest}) \times \rho_{\text{R2}}(\text{Rest}))\big)$. "Second-highest rating" is a **value**; "rating of the second sailor" needs an ordering of rows, which relations do not have.

B3 Dividend $\pi_{\text{MemberID, BookID}}(Borrow)$. Divisor $\pi_{\text{BookID}}\Big(WrittenBy \underset{\text{AuthorID} = \text{A.AuthorID}}{\bowtie} \sigma_{\text{A.AuthorName} = \textbf{Navathe}}(\rho_{\text{A}}(Author))\Big)$. Result (MemberID).

B4 As stated it does not make sense: "every generic category" treats each medicine as a category of its own, so the query degenerates to "received every generic medicine", which **is** answerable. Repaired: dividend $\pi_{\text{Patient_SSN, TradeName}}\Big(Prescription \underset{\text{Id} = \text{Prescription_Id}}{\bowtie} Prescription_Medicine\Big)$, divisor $\pi_{\text{TradeName}}\big(\sigma_{\text{GenericFlag} = \textbf{Y}}(Medicine)\big)$, result (Patient_SSN). The lesson: a division question is only well posed when the divisor describes a genuine finite set.

B5 Two-directional coverage (Q7.33 / Q9.6). One direction alone returns pairs (a, b) where b has borrowed everything a did — a **superset** relation, not equality. Both directions are needed because set equality is a conjunction of two inclusions.

B6 The errors, in evaluation order: (i) `Sailors` has no `color`, so the selection is not bound to the right relation — it must be applied to `Boats`; (ii) as written the selection is applied **after** the joins, which is legal but then `color` is unqualified and ambiguous in principle — qualify it; (iii) the projection after the join is fine, so the repaired expression is $\pi_{\text{sname}}(Sailors \bowtie Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats))$, or equivalently $\pi_{\text{sname}}(\sigma_{\text{Boats.color} = \textbf{red}}(Sailors \bowtie Reserves \bowtie Boats))$. A third, subtler issue: if the intent was "reserved **only** red boats", the expression answers the wrong quantifier.

12.3 Answer key — Section III

C1 (a) $\pi_{\text{sname}}\big(Sailors \bowtie \big(\pi_{\text{sid, bid}}(Reserves) \div \pi_{\text{bid}}(\sigma_{\text{color} = \textbf{red}}(Boats))\big)\big)$ — pattern 6. (b) $\pi_{\text{sname}}(Sailors \bowtie (\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats))))$ — pattern 2. (c) $\pi_{\text{sname}}\Big(Sailors \bowtie \Big(\pi_{\text{sid}}(Reserves) - \pi_{\text{sid}}\Big(Reserves \bowtie \sigma_{\text{color} \neq \textbf{red}}(Boats)\Big)\Big)\Big)$ — pattern 5. (d)

$\pi_{\text{sname}}\big(Sailors \bowtie \big(\pi_{\text{sid}}(Reserves \bowtie \sigma_{\text{color} = \textbf{red}}(Boats)) \cap \pi_{\text{sid}}\big(Reserves \bowtie \sigma_{\text{color} = \textbf{green}}(Boats)\big)\big)\big)$ — pattern 3. (b) and (c) are not complements: a sailor with no reservations satisfies **neither** "reserved a red boat" nor "reserved a red boat but not a green one"; and a sailor with red **and** blue satisfies neither (b) nor (c) either — he is excluded from (c) by the blue boat, and from (b) because he **did** reserve red. The two sets overlap, so they cannot be complements.

C2 (a) $\pi_{\text{D.FirstName, D.LastName}}\Big(\rho_{\text{D}}(Doctor) \underset{\text{D.SSN} = \text{Doctor_SSN}}{\bowtie} \sigma_{\text{PrimaryDoctor_SSN} \neq \text{Doctor_SSN}}\Big(Prescription \underset{\text{Patient_SSN} = \text{P.SSN}}{\bowtie} \rho_{\text{P}}(Patient)\Big)\Big)$. (b) patients minus patients having a prescription by a non-primary doctor (Q8.4). (c) the case table of Q8.4: (b) as literally worded returns A ∪ B; the stricter "has prescriptions, all by the primary doctor" returns B only. State the reading.

C3 (a) Not expressible; missing operator γ (grouping with `count`). (b) $\gamma_{\text{BookID; count(MemberID)}\leftarrow n}(Borrow)$, then the maximum-of-`n` pattern, then join to `Book` for the title. (c) "Borrowed by the largest number of distinct members" is also a count of distinct entities, so also not expressible — a common trap, because "distinct" sounds like a set operation but the **comparison** is still between counts. (d) With a `Price` attribute, $Book - \pi_{\text{B1.BookID, B1.Title, B1.Publisher, B1.Category, B1.Price}}(\sigma_{\text{B1.Price} < \text{B2.Price}}(\rho_{\text{B1}}(Book) \times \rho_{\text{B2}}(Book)))$ — expressible because price is a **stored value**, so it can be compared with $<$ directly.

C4 (a) False (Q4.15). (b) False in general: if θ mentions attributes of both R and S, $\sigma_{\theta(R)} \bowtie \sigma_{\theta(S)}$ is ill-typed or empty while the left side is not; the identity holds only when θ is a conjunction of an R-part and an S-part. (c) True: every reserving `sid` is a sailor's `sid` (referential integrity), so the difference is always empty — a good check on foreign keys. (d) True: $R - (R - S) = R \cap S$ (§1.10).

12.4 Answer key — Section IV (outline)

D1 (1) "More than three distinct doctors" is **expressible** (four tuple variables, Q8.22); the accompanying **count** is not — say so and give the four-variable expression. (2) Expressibility is subtle: "every patient for whom they are the primary doctor" is a quantified set derived from `Patient`; if a doctor has **no** primary patients, the condition is vacuously true. The expression is a difference against a violation set, exactly as in Q8.14. (3) Two nested differences: medicines of the patient minus medicines of the primary doctor. (4) $\pi_{\text{Doctor_SSN, TradeName}}(Prescription \bowtie Prescription_Medicine) \div \pi_{\text{Doctor_SSN}}(Prescription)$ — a clean division with a derived dividend. (5) Maximum over `Date` (Q5.13), then join back: expressible.

D2 (1) A division whose divisor varies per sailor — not directly expressible as one division; use the counter-example method with a **three-variable** device (Q9.6), subtracting the "sailor x has a boat of a lower-rated sailor that x lacks" pairs. (2) $\pi_{\text{sid, day}}(Reserves) \div \pi_{\text{day}}(\sigma_{\text{bid} = 103}(Reserves))$ — a clean division. (3) Not expressible (count of reservations per boat). (4) Domain error: `age` and `rating` are different domains, and "a boat that is older than a rating" is a category mistake. The repaired question is presumably "sailors who have never reserved a boat whose **bid** is lower than their rating" — still a domain error; the meaningful repair is "…a boat that any sailor with a lower rating has reserved" (Q9.5). (5) $\pi_{\text{sid}}(Sailors) - \pi_{\text{sid}}\big(\pi_{\text{sid}}(Sailors) \times \pi_{\text{bid}}(Boats) - \pi_{\text{sid, bid}}(Reserves)\big)$.

12.5 Coverage checklist

Every row of the brief is listed with the questions that discharge it. Tick the box when you can reproduce the **pattern** from memory, not the example.

Ref	Required category	Where covered	Done
C1	Selection and projection	§1.4–1.5; Q2.1–Q2.12; Q6.1–Q6.8	☐
C2	Rename and aliasing	§1.6; Q2.13–Q2.14	☐
C3	Union, intersection, difference; union compatibility	§1.7; Q3.1–Q3.14; Q6.9–Q6.14	☐
C4	Cartesian product and theta join	§1.8; Q2.15–Q2.20	☐
C5	Equijoin and natural join	§1.8; Q2.17–Q2.18	☐
C6	Multi-relation joins	Q2.18; Q6.4; Q7.4; Q8.9	☐
C7	Self-joins	Q3.15–Q3.20; Q6.15–Q6.22	☐
C8	At least one / at least two / exactly one / none	Q3.8–Q3.10; Q5.17–Q5.19; Q5.23–Q5.27	☐
C9	NOT, NEVER, WITHOUT, ONLY, EXCEPT	Q3.3–3.7, 3.21–3.26; Q6.11–6.14, 6.26, 6.31, 6.33, 6.35	☐
C10	ALL, EVERY, EACH; universal quantification	Part 4 in full; Q6.23–Q6.27, Q6.34, Q6.37	☐
C11	Division and equivalent formulations	§1.9–1.10, §4.2; Q4.1–Q4.22	☐
C12	Same versus different records	Q3.15–3.19; Q6.15–6.22; Q7.33; Q8.21	☐
C13	Date comparisons	Q2.8; Q3.9, 3.21; Q5.13; Q7.25–7.26; Q8.24	☐
C14	Conditional and group-wise maximum/minimum	Q5.8, 5.12–5.16; Q8.24	☐
C15	Nested expressions	Q9.1–Q9.14	☐
C16	Query equivalence	§10.1; Q10.19–Q10.23; C4 of the practice paper	☐
C17	Error detection	§10.3, Q10.1–Q10.18; B6 of the practice paper	☐
C18	Union compatibility and schema matching	§1.7; Q10.3, Q10.10, Q10.18	☐
C19	Expression simplification	§10.4–10.5; Q10.19–Q10.23	☐
C20	Fixed-cardinality conditions without aggregates	§10.6; Q5.17–Q5.19; Q8.22	☐
C21	Queries not expressible under the restrictions	§10.7; Q5.9–5.10, 5.16, 5.19, 5.27⟶; Q7.24; Q8.3, 8.27; Q9.8, 9.13	☐
C22	Maximum / minimum attribute values	§5.1; Q5.1–5.4, 5.11	☐
C23	Records with maximum / minimum	Q5.3–5.4, 5.6, 5.11	☐
C24	Second-highest, third-highest, k-th highest	Q5.5–5.7, 5.22, 6.30, 9.9	☐
C25	Tied maxima and minima	Q5.3, 5.6, 5.14; Q6.28	☐
C26	Global versus group-wise maximum	Q5.8, 5.13, 5.15; Q8.24; §10.2 pair 8	☐
C27	Most expensive treatment per doctor / cheapest per patient	Q5.8, 5.11, 5.13; Q8.16, 8.20, 8.24	☐
C28	Highest salary per department	Q5.8, 5.23	☐
C29	Latest and earliest dates	Q3.9, 3.21; Q5.13; Q7.25–7.26; Q8.24	☐
C30	Maximum after a selection / after removing records	Q5.12, 5.21; Q9.7, 9.9	☐
C31	The eleven confusable sentence pairs	§10.2, pairs 1–11	☐

Ref	Required category	Where covered	Done
C32	Sailors and Boats: all 37 required queries	Part 6, Q6.1–Q6.37	☐
C33	Library: full and simplified schema, 33 queries	Part 7, Q7.1–Q7.33	☐
C34	Hospital: the five exam questions	Q8.1–Q8.5	☐
C35	Hospital: 20+ additional queries	Q8.6–Q8.27	☐
C36	Assumptions on keys and NULLs	§1.16; Q7.30; Q8.4	☐
C37	When aggregation or extensions are required	§1.11, §10.6–10.7; Q5.9, 5.16, 5.27; Q7.24; Q8.3	☐

12.6 Known traps, in one page — the last thing to read before the exam

1. **The universe is part of the question.** "Never reserved a red boat" must start from $\pi_{\text{sid}}(Sailors)$, not from $\pi_{\text{sid}}(Reserves)$.
2. **Set operations go on keys.** `sname` and `Title` are not keys; subtracting names deletes innocent rows and keeps wrong ones.
3. **Both sides of $-, \cup, \cap$ must have the same attribute list.** Write it out before you write the operator.
4. **A selection cannot see another tuple.** Comparisons between two rows need ρ plus $\times$.
5. **For self-joins with $\neq$ use $\times$.** A natural join of a relation with its renamed copy equates everything.
6. **Divide only after projecting.** The dividend is exactly (answer attributes $\cup$ divisor attributes).
7. **The divisor is the set you quantify over** — filter **inside** it, not around it.
8. **Extremes: value first, then join.** That is what preserves ties.
9. **Standardise the projection list to the target's schema order** whenever you subtract whole tuples.
10. **Counts, sums, averages, "more than n", "same number as", "most" are not classical.** Say so, name γ, and then — if the question allows — give the extended expression.
11. **Bracket everything.** There is no universal precedence.
12. **State your assumptions** about ties, NULLs, keys and the empty case. Examiners award marks for each one you name, and they cannot award marks for what you did not write.

End of the question bank.
Every expression in this book is classical relational algebra: π, σ, ρ, $\cup$, $-$, $\times$, and the derived $\cap$, $\bowtie$, $\div$. No SQL, no aggregates, no shortcuts that cannot be expanded.